

Algorithmische Komposition in der Filmmontage oder wie ich darüber denke

Masterarbeit
zur Erlangung des Grades
Master of Fine Arts (M.F.A.)
im Studiengang Montage
an der Filmuniversität Babelsberg KONRAD WOLF

1. Gutachterin: Prof. Marlis Roth
2. Gutachterin: Gesa Marten

Vorgelegt von:

Jonathan Frank
Fuldastraße 42
12045 Berlin
0163 68 123 86
JonathanFrank@gmx.net
Matrikelnummer: 5542

Abgabetermin: 29. Juni 2020

2. Auflage





Abb. 1: Ein Baum auf dem Tempelhofer Feld. Diese Aufnahme habe ich für eines meiner ersten algorithmischen Experimente, den audiovisuellen Automaten, genutzt.

So wie es in der Kunst ist, ist es in der Welt.

Inhaltsverzeichnis

Seite 6	Einleitung
Seite 15	Eine assoziative Definitionskette
Seite 16	Niklas Luhmann und die Montage
Seite 20	Algorithmus und Improvisation
Seite 24	Eine kleine Geschichte der algorithmischen Filmkomposition
Seite 33	»Pure Data« und das Programmieren
Seite 37	Die diagrammatische Partitur
Seite 38	Markov-Ketten
Seite 51	Der Montage-Automat
Seite 58	Der Montage-Automat – eine Sequenz
Seite 92	Der Montage-Automat und meine Erkenntnisse
Seite 94	Der Montage-Automat als Quelltext
Seite 116	Zellulärer Automat

Seite 124	»Pure Data« Experimente – ein Katalog
Seite 134	»AudioVideo«
Seite 138	»Handmadeproductions«
Seite 141	Über künstlerische Forschung
Seite 150	Abschließende Gedanken
Seite 154	Literaturverzeichnis
Seite 159	Abbildungsverzeichnis
Seite 163	Eidesstattliche Erklärung

Einleitung

In meiner Masterarbeit werde ich der Frage meiner Bachelorarbeit: Wie entscheide ich mich in der Filmmontage? Weiter nachgehen. Mein Ansatz ist, die Entscheidungsfindung in der Montage mit dem Begriff der Improvisation zu erklären. Inspiriert wurde ich dabei, unter anderem, von dem Buch »Prinzip Improvisation« von Christopher Dell.

»Der Terminus Improvisation stammt vom lateinischen *improvisus* und bedeutet *unvorhergesehen, unvermutet* oder auch *ohne Vorbereitung*. Da dieser Begriff demnach das Unvermutete umfasst, es umschreibt, umschreibt es auch das, was es noch nicht gibt. Da wiederum das, was es noch nicht gibt, selten Gegenstand gründlicher Untersuchung ist, hat sich der Begriff Improvisation inzwischen derart verwässert, dass er heute einer Neudefinition Bedarf. Eingedenk dessen, dass eine Verdinglichung von Improvisation ihrem Charakter widerspricht. Improvisation ist genau die Differenz zwischen Denken und Handeln, zwischen Gedachtem einerseits und Erhandeltem und Erfühltem andererseits. Wir können Improvisation nur umschreiben, können uns nur in eine Haltung der Improvisationsbereitschaft begeben. Alles Weitere rufen die Situationen und die daraus resultierenden Prozesse hervor.«¹

Ich behaupte, dass die Improvisation in der Filmmontage wesentlicher Bestandteil des kreativen Schaffensprozesses ist. Die Improvisation wiederum findet innerhalb von Strukturen statt und ist in der Lage innerhalb dieser Strukturen etwas Neues zu erschaffen. Die Strukturen, auf die ich mich in der Improvisation beziehe, habe ich in meiner Bachelorarbeit metaphorisch mit musikalischen Notationen in Form von diagrammatischen Partituren (Abb. 2) verglichen. Diagrammatische Partituren (graphical scores) lassen sich auch mit der Programmiersprache »Pure Data«, die ich für meine Programmierexperimente genutzt habe, und auf die ich noch eingehen werde, erstellen:

¹ Christopher Dell: Prinzip Improvisation. Köln: Verlag der Buchhandlung Walther König, 2002. S. 17.

»The original idea in developing Pd [»Pure Data«] was to make a real-time computer music performance environment like Max, but somehow to include also a facility for making computer music scores with user-specifiable graphical representations. This idea has important precedents in Eric Lindemann's Animal and Bill Buxton's SSSP. An even earlier class of precedents lies in the rich variety of paper scores for electronic music before it became practical to offer a computer-based score editor. In this context, scores by Stockhausen (Kontakte, Studie II) and Yuasa (Toward the Midnight Sun) come most prominently to mind, but also Xenakis's Mycenae-alpha, which, although it was realized using a computer, was scored on paper and only afterward laboriously transcribed into the computer.«²

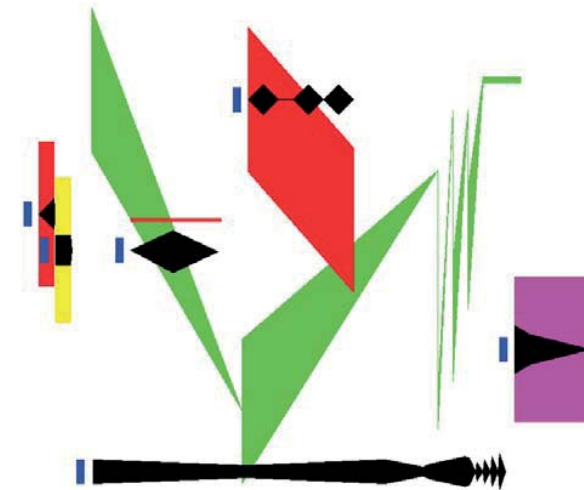


Abb. 2: Eine mit »Pure Data« erstellte diagrammatische Partitur (doc/4.data.structures/07.sequencer.pd). »This patch shows an example of how to use data collections as musical sequences (with apologies to Yuasa and Stockhausen). Here the black traces show dynamics and the colored ones show pitch. The fatness of the pitch traces give bandwidth. Any of the three can change over the life of the event.«

² Miller Puckette: Using Pd as a score language. San Diego: University of California, 2007. S. 1.

Nun werde ich versuchen, die Begriffe Programm und Algorithmus als die Strukturen zu beschreiben, innerhalb derer ich mich improvisatorisch in der Montage bewege. Dabei kann die diagrammatische Partitur als eine Möglichkeit gesehen werden, ein Programm lesbar zu machen.

Sowohl die Filmmontage, als auch die Montage im Allgemeinen, sind für mich das improvisatorische, spielerische Entdecken und Herausarbeiten von Strukturen aus dem Material, das mir innerhalb eines Projekts zur Verfügung steht. Ob das im Austausch mit der Regie oder alleine passiert, macht in der Arbeitsweise einen großen Unterschied, spielt aber bei meinen Überlegungen keine Rolle, da das mögliche Wechselspiel von Programm und Improvisation nicht von der Anzahl der Beteiligten abhängt.

Ich reagiere improvisierend immer wieder neu auf das Material und die daraus entstehenden Sequenzen, ohne einem vorher festgelegten Schema zu folgen. Dabei sind meine Entscheidungen nicht beliebig, da das Material nur bestimmte Möglichkeiten der Interpretation zulässt. Ich bekomme etwas geschenkt und meine Aufgabe ist, zu entscheiden, ob und wie es modifiziert werden soll. Entweder im Austausch mit der Regie und anderen Beteiligten oder für mich alleine. Ich vergleiche das gerne mit der musikalischen Improvisation, die ebenso gemeinsam als auch alleine möglich ist. Bei der gemeinschaftlichen Improvisation reagieren alle nicht nur auf das Material, sondern außerdem auf die anderen Beteiligten.

Mit der algorithmischen Filmmontage will ich eine spezielle Form des Programms in der Montage untersuchen. Ein Versuch etwas mit vorher festgelegten Regeln, also ohne improvisatorischen Spielraum, zu generieren. Um durch das Weglassen der Improvisation vielleicht etwas über sie zu erfahren. Wobei das Festlegen der Algorithmen, also das Programmieren von Computerprogrammen, durchaus im improvisatorischen Modus stattfindet, und nur das Generieren selbst einem determinierten Ablauf folgt. Das Improvisatorische verschiebt sich von der Montage zum Programmieren des Programms, das diese dann ausführt.

Der Begriff des Programms bedeutet, nach dem griechischen Ursprung des Wortes *Próγραμμα*, das Vorgeschriebene. Eigentlich das gleiche wie eine Vorschrift, aber doch ganz anders konnotiert. Weil *vor-geschrieben* ist alles, was schon da ist, auf das ich mich mit meiner Erfahrung, die auch *vor-geschrieben* ist, da ich sie bereits gemacht habe, beziehen kann. Also in der Filmmontage zunächst das Material. Aus dem dann in der Montage die Sequenzen, die Ordnung des Materials in den Bins (Behälter / Kasten), sowie Notizen und Aufzeichnungen entstehen, auf die ich mich dann wieder neu beziehen kann. Das *Vorgeschriebene* befindet sich in ständiger Veränderung, da es sich ständig *fort-schreibt*.

Auf diese umfassende Definition des Programms bin ich erst durch Niklas Luhmanns Idee des sich selbst programmierenden Kunstwerks gekommen, auf das ich noch näher eingehen werde. Ist somit auch das Algorithmische essenzieller Bestandteil der Montage? Ursprünglich hatte ich mich jedenfalls mit algorithmischer Filmmontage mehr auf die algorithmische Komposition in der Musik bezogen und bei dem Begriff des Programms an ein Computerprogramm gedacht, das die programmierten Algorithmen ausführt:

»Als Algorithmische Komposition (AK) bezeichnet man jene Kompositionsverfahren, bei denen die Partitur durch einen automatischen, mathematisch beschreibbaren Prozess oder Algorithmus erzeugt wird.

Im Prinzip lässt sich jedes Musikstück als eine Folge von Zahlen darstellen: Ist es bei einem Instrument möglich, die Tonhöhe sowie die Anschlagsstärke und -dauer einer Note zu variieren, dann ist jede Note mit drei Zahlen darstellbar.

AK ist etwas vereinfacht die Entwicklung von Regeln, die solche musikalisch interpretierbaren Zahlenfolgen erzeugen. In der heutigen Praxis ist das meist die Entwicklung eines Computerprogramms; Computer sind jedoch nicht zwingend erforderlicher Bestandteil der AK.«³

³ https://de.wikipedia.org/wiki/Algorithmische_Komposition
[22.05.2020].

Sehr ähnlich dieser Definition der algorithmischen Komposition ist die folgende Definition der generativen Kunst:

»Als generative Kunst kann man jene künstlerischen Praktiken/Verfahren bezeichnen, bei denen der/die KünstlerIn einen Prozess in Gang setzt, bei dem durch das Abarbeiten des von ihm/ihr vorgegebenen Regelwerkes ein Kunstwerk entsteht (oder zu einem Kunstwerk beigetragen wird). Dabei können die Regeln entweder die Form natürlicher Sprache haben, es kann sich aber auch um ein Computerprogramm oder andere Mechanismen handeln.«⁴

Unerheblich, ob von einem Menschen oder einem Computer ausgeführt, kann ein Programm nicht improvisieren, es kann die Improvisation durch den Menschen nur zulassen. Es ist allerdings möglich, den Prozess der Improvisation, zum Beispiel durch die Generierung von Markov-Ketten (auf die ich noch eingehen werde), mit einem Computerprogramm zu simulieren. Meist wird dabei die menschliche Entscheidung in der Improvisation durch Zufallsprozesse und vorher berechnete Wahrscheinlichkeiten nachgeahmt.

Eine weitere Fragestellung, die sich aus meinen Überlegungen ergeben hat: Ist es möglich, ein Computerprogramm zu schreiben, das in der Lage ist, die Struktur von Filmen zu lernen und diese auf anderes Filmmaterial anzuwenden, sodass sich die gelernte Struktur in dem generierten Ergebnis ausdrückt? Aus der theoretischen Idee ist das praktische Kernstück meiner Masterarbeit, der Montage-Automat, entstanden. Ein Versuch, mit algorithmischen Methoden filmische Strukturen zu generieren.

Neben dem Montage-Automaten sind weitere audiovisuelle Experimente entstanden, die ich ausschnittsweise in Form eines Katalogs noch vorstellen werde. Teils haben sie zu den Überlegungen, die zum Montage-Automaten geführt haben beigetragen, teils sind es eigenständige Ansätze. Ich würde sie größtenteils der algorithmischen Komposition, der generativen Kunst und der Computerkunst zuordnen:

⁴ Philip Galanter: Zitiert von der Mailingliste eu-gene. http://artwarez.org/projects/nagBOOK/texte/sarah_de.html [22.05.2020].

»Digitale Kunst oder Digitalkunst, oft gleichbedeutend mit Computerkunst gebraucht, sind im allgemeinen Sprachgebrauch Sammelbegriffe für Kunst, die digital mit dem Computer erzeugt wird. Im engeren Sinn ist es Kunst, die nur durch die spezifischen Eigenschaften digitaler Medien möglich geworden ist, zum Beispiel die Zählbarkeit aller Information, ihre Trennbarkeit von einem bestimmten Datenträger oder den Einsatz von Algorithmen. – Erst in den 1990er-Jahren wurde der Ausdruck digitale Kunst gebräuchlich.«⁵

Über das, was für mich sinnvoll ist, werde ich auch noch etwas schreiben. Das ist gar nicht so einfach. Kann zum Beispiel eine Tonfolge oder ein Rhythmus sinnvoll sein? Jedenfalls ist es in der Musik und der abstrakten Kunst schwieriger von Bedeutung zu sprechen, als zum Beispiel im Dokumentarfilm oder der Erzählung. Wahrscheinlich weil das Begriffliche fehlt und der Sinn, wenn man davon sprechen kann, über die Struktur entsteht.

Der Tonumfang einer Tonfolge kann in der Musik eingegrenzt sein von einer Tonleiter, die Akkorde von einer Harmonie, und der Rhythmus von einem Takt. Innerhalb dieser Reduktion von Komplexität kann man sich improvisatorisch frei bewegen und so innerhalb der Strukturen neue Strukturen erzeugen. Das Musikalische lässt sich dabei aus unterschiedlichen Gründen einfacher algorithmisch generieren und beschreiben (notieren) als das Filmische. Es sei denn, man arbeitet auch im Film ohne begriffliche Sprache, und visuell auf einer ähnlich abstrakten Ebene wie die Musik. Zum Beispiel dadurch, dass man einer Farbe oder einem Bild eine Zahl zuordnet, und diese rhythmisch in der Zeit anhand einer Formel strukturiert.

Aber zumindest, wenn man von einem Film mit klassischer Erzählstruktur ausgeht, das heißt, mit Protagonisten, Dialog und Handlung, dann ist es im Gegensatz zur Musik aufgrund der ungleich größeren Komplexität in der Praxis kaum möglich, dessen Struktur in eine algorithmische Formel zu fassen. Schon die Anzahl der möglichen Ausdrucks-

⁵ https://de.wikipedia.org/wiki/Digitale_Kunst [22.05.2020].

formen ist im Film sehr viel größer als in der Musik. Neben der Musik selbst kann der Film weitere Elemente wie das Visuelle (die Bildsprache), den Text und das Verbalen enthalten. Alles das trägt durch die Interpretation und Verknüpfung der Montierenden zur Struktur des Films bei. Und selbst wenn sich eine Erzählstruktur formalisieren lässt, wie es zum Beispiel in der »Der Heros in tausend Gestalten«⁶ von Joseph Campbell versucht wird, kann man daraus zwar eine Struktur ableiten, aber ohne die künstlerische Bearbeitung und Variation dieser Strukturen durch eigene Ideen ist es dennoch nicht möglich, ein neues Werk zu schaffen. Das wiederum ist in der Musik nicht anders, auch da gibt es Strukturen, die in immer wieder neuen Variationen erzeugt und miteinander kombiniert werden können.

Man kann musikalische Begriffe wie Rhythmus, Melodie und Harmonie auch auf die Filmmontage übertragen. Dabei bleiben sie aber zumindest teilweise Metaphern, die es nicht einfacher machen, das Filmische algorithmisch auszudrücken. Gilles Deleuze hat dazu geschrieben:

»Jedes Bewegungs-Bild drückt in Bezug auf die Gegenstände, zwischen denen sich die Bewegung herstellt, das sich verändernde Ganze aus. Folglich muss schon die Einstellung als potentielle Montage und das Bewegungs-Bild als Matrix oder Zelle der Zeit betrachtet werden. Unter diesem Gesichtspunkt ist die Zeit abhängig von der Bewegung und ihr zugehörig. Man kann sie nach der Art der antiken Philosophie als die Zahl der Bewegung bestimmen. Die Montage wäre demnach ein Zahlenverhältnis, das gemäß der intrinsischen Natur der Bewegungen variiert, die in jedem Bild und in jeder Einstellung anzutreffen sind. Eine gleichförmige Bewegung in der Einstellung gemahnt an einen einfachen Takt, während vielfältige und differenzierte Bewegungen an einen Rhythmus, intensive Bewegungen (wie Licht und Wärme) an eine Klangfarbe, und die Gesamtheit aller Einstellungsmöglichkeiten an eine Harmonie erinnern. Von daher Eisensteins Unterscheidungen zwischen

⁶ Joseph Campbell: Der Heros in tausend Gestalten. Frankfurt: Insel Taschenbuch, 1999.

einer metrischen, rhythmischen, tonalen und harmonischen Montage.«⁷

Ist der Sinn eines Bildes immanent, oder konstruieren wir ihn erst? Ich musste mir tatsächlich auch noch mal die Frage stellen: Was ist ein Algorithmus? Hat das auch etwas Rhythmische? Wo ist da die Verbindung? - Falls es eine gibt.

Ein Algorithmus ist für mich eine vorher festgelegte Entscheidungsfolge oder Spielregel (bei der durchaus mehrere Möglichkeiten erlaubt sein können). Eigentlich die klassische if → then → else Folge. Wenn man zum Beispiel einer Tonhöhe eine Farbe oder eine Form zuordnet, ist das dann schon algorithmisch? (if) Wenn Ton F gespielt wird, (then) zeige einen blauen Kreis, (else) ansonsten zeige eine rote Fläche. Oder noch simpler: Immer auf die rhythmische eins zu schneiden wäre nach meiner Definition auch schon die Umsetzung eines Algorithmus, also ein ausgeführtes Programm. Auch der Film an sich folgt durch die regelmäßige Abfolge seiner Einzelbilder einer Algorithmik, die zeitliche Länge eines Bildes berechnet sich dabei durch die Formel: $1 / \text{Bildrate} = \text{Bildlänge in Sekunden}$. Die mögliche Komplexität der Algorithmen ist, in der Filmmontage auch abhängig vom Rohmaterial, wahrscheinlich nach oben offen.

Die Improvisation findet im Rahmen des Algorithmus statt, beziehungsweise lassen sich dessen Spielregeln durchaus brechen. Kann man schon die klassische 3 Akt Struktur mit ihren Wendepunkten oder die griechische Tragödie als eine algorithmische Struktur sehen? Das Drehbuch als einen Algorithmus für den Film? Das Rohmaterial als Algorithmus für die Filmmontage?

Auch habe ich mich oft gefragt, was ich da eigentlich mache: Kunst, Montage, Handwerk, Programmieren, Forschung, Wissenschaft, künstlerische Forschung, Werkzeuge bauen, Automaten bauen? Um für mich die Frage nach der künst-

⁷ Gilles Deleuze: Das Zeit-Bild - Kino 2. Frankfurt: Suhrkamp, 1997. S. 54.

lerischen Forschung zu beantworten. Und ich habe versucht, meine eigene Position darzulegen. Ganz kurz gesagt glaube ich, dass ich ein Künstler bin, der forscht und Technologien nutzt. Wissenschaftler bin ich nicht, auch wenn ich von der Wissenschaft (und der Natur) fasziniert bin, und mich von ihr inspirieren lasse. Auch nutze ich die Mathematik, aber ich habe nicht vor neue (mathematische) Formeln zu finden, dazu wäre ich auch nicht in der Lage. Technologien nutze ich einerseits als fertige Werkzeuge (die Schere, den Schneidetisch, das Schnittprogramm), andererseits baue ich mir Werkzeuge, die ich meine zu benötigen, und die es noch nicht gibt. Und ich baue Automaten, Werkzeuge die generieren. Da man mit den Werkzeugen, die man baut und nutzt das künstlerische Ergebnis wesentlich beeinflusst, ist auch das Werkzeuggebauen selbst Teil des künstlerischen Prozesses.

Eine assoziative Definitionskette

Zur besseren Orientierung, mit welchen Begriffen ich mich beschäftige und wie sie miteinander zusammenhängen könnten, habe ich spielerisch eine Definitionskette gebildet:

Ein Muster ist eine Struktur mit sich regelmäßig wiederholenden Elementen. Ein Rhythmus ist die regelmäßige Wiederkehr von Elementen. Eine Struktur ist eine Zusammenfügung von Elementen. Eine Komposition ist eine Zusammenstellung von Elementen. Eine Komposition kann in jedem künstlerischen Medium erstellt werden. Eine Komposition kann notiert werden oder improvisatorisch aus dem Moment entstehen. Eine Notation ist das Festhalten von Bewegungsabläufen. Eine Sequenz ist eine lineare Abfolge. Auch eine notierte Komposition kann improvisatorisch entstanden sein. Eine Komposition hat eine Struktur und kann Muster haben. Montage ist Komposition. Ein Prozess ist ein Verlauf oder eine Entwicklung. Dramaturgie ist prozessual und strukturiert. Ein Diagramm ist eine grafische Darstellung von Daten, Sachverhalten oder Informationen. Ein Algorithmus ist eine eindeutige Handlungsvorschrift zur Lösung eines Problems oder einer Klasse von Problemen. Nichtdeterministische Algorithmen können im Allgemeinen mit keiner realen Maschine direkt umgesetzt werden. Nicht determinierte Algorithmen erlauben Improvisation. Ein Programm ist eine konkrete Form des Algorithmus. Eine Arbeit ist ein Kunstwerk. Eine Position ist eine Arbeit. Ein Stück ist ein Kunstwerk. Ein Kunstwerk ist ein Programm. Sinn ist Bedeutung. Der künstlerische Prozess kann das Kunstwerk sein. Musik ist ein organisiertes Schallereignis. Intentionalität ist die Fähigkeit, sich auf etwas zu beziehen.

Niklas Luhmann und die Montage

Mich mit dem Begriff der Kunst beschäftigend bin ich auf Niklas Luhmann gestoßen, der in »Die Kunst der Gesellschaft« im Rahmen seiner soziologischen Systemtheorie das Kunstwerk als sich selbst programmierend beschreibt:

»Das ›Wesen‹ der Kunst ist die Selbstprogrammierung der Kunstwerke.«⁸

Auch, weil ich Niklas Luhmann anfangs falsch verstanden habe, musste ich zunächst bei dem Begriff der Selbstprogrammierung an die von meinen Computerprogrammen algorithmisch generierte Kunst denken. Von seinem Verständnis, dass sich jedes Kunstwerk selbst programmiert, außer vielleicht die Computerkunst, da sie generiert, anstatt das Reagieren zu ermöglichen, habe ich erst später erfahren.

»Niklas Luhmann nimmt in seiner Gesellschaftstheorie der Kunst keine Rücksicht auf geisteswissenschaftliche Vorbehalte gegenüber Begriffen wie ›Maschine‹ und ›Programm‹. Das ›Programm‹ eines Kunstwerks ist kein Plan, der in einem getrennten Ausführungsprozess strikt zu befolgen ist, sondern enthält Entscheidungen, die Künstler im Verlauf der Herstellung treffen. In Ausführungsschritten entscheiden Künstler, welche Wahl zwischen Möglichkeiten ›passend‹ an die bereits getroffenen Entscheidungen anschließt, und schränken sich so weiter ein: ›[...] der Formfindungsprozess [läuft] nicht wie eine programmierte Maschine, sondern eher wie eine historische Maschine ab [...], also wie eine Maschine, die sich durch den gerade erreichten eigenen Zustand determinieren läßt‹⁹. Der Begriff ›Form‹ steht für ›strikte Kopplungen‹ und der Begriff ›Medium‹ für ›lose Kopplungen‹. Entscheidungen, die Künstler im Prozess der Herstellung fällen, sind Selbst-

8 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 332.

9 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 314.

beschränkungen (strikt) vor dem Horizont der medienbedingten Möglichkeiten (lose).«¹⁰

Diese Beschreibung des künstlerischen Schaffensprozesses kann man auch auf den Film und insbesondere auf die Montage übertragen. So ist die Montagesequenz ein Programm, also ein ausgeführter Algorithmus, das sich bis zum vollendeten Film selbst programmiert. Damit ist ein anderer, allgemeinerer Ansatz der algorithmischen Montage beschrieben. Ich meine aber, dass es sich dabei im Gegensatz zur computergenerierten Montage um nicht-determinierte Algorithmen handelt. Die erste Formfestlegung wird nicht zwangsläufig das Ergebnis bestimmen, es ist innerhalb gewisser Grenzen offen.

»Ein Kunstwerk entsteht durch eine erste Formfestlegung, an die sich weitere anschließen. ›Am Anfang ist die Differenz, der Einschnitt einer Form, die das weitere zu regulieren beginnt.«¹¹ Die erste Klangfolge, der erste Satz, die ersten Tanzschritte, der erste Pinselstrich usw. präfigurieren das Folgende, und mit jeder hinzukommenden Form wird der Bereich der noch möglichen Anschlüsse eingeschränkt. Das Kunstwerk gibt sich selbst sein Gesetz, es limitiert sich selbst zunehmend. Mehr und mehr entzieht es sich selbst der Willkür. Auto-Komposition ist jedoch nur möglich, wenn das Kunstwerk schon angefangen hat. Über seinen Anfang kann das Kunstwerk aber nicht verfügen. Das Kunstwerk braucht eine erste Unterscheidung, um aus der Welt einen Bereich bestimmbarer Komplexität auszuschneiden.«¹²

Wo ist die erste Formfestlegung des Kunstwerks in der Montage? In der Malerei ist es der erste Strich. In der Musik der erste Ton. In der Montage könnte es das erste

10 Thomas Dreher: Das Kunstwerk, Weltkunst und Intermedia Art - kritische berichte, Bd. 36, Nr. 4. Weimar: Jonas Verlag, 2008. S. 54.

11 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 45.

12 Markus Koller: Die Grenzen der Kunst - Luhmanns gelehrte Poesie. Wiesbaden: VS Verlag für Sozialwissenschaften, 2007. S. 50.

Bild in der Sequenz sein, das erste Bild, das man aus dem vorhandenen Material sieht, oder das gesamte Rohmaterial.

Auch wichtig ist, mir zu betonen, dass Niklas Luhmann mit der Selbstprogrammierung eine Freiheit in der Kunst beschreibt:

»Das Kunstwerk arbeitet mit seiner Grundlosigkeit, von allem Anfang an. Deshalb ist es ein kompliziertes Gebilde und nicht nach Regeln, schon gar nicht durch gutgemeinte Willensakte herstellbar. Kunstwerke, die in der ›Kultur‹ so selbstverständlich für verfügbar, also für besprechbar, für kritisierbar gehalten werden, sind angesichts dieser Selbstverständlichkeit (und auch angesichts jener, mit der manche Künstler etwas, nur weil es von ihrer Hand stammt, für Kunst gehalten haben wollen) geradezu spöttisch komplex. Für diese Komplexität findet Luhmann die schönsten Worte: Kunst sei auch insofern ein ›Medium der Kommunikation, als sie nicht festlegt, wie Künstler und Betrachter durch das Kunstwerk gekoppelt werden, aber andererseits doch garantiert, daß es dabei nicht beliebig zugeht‹¹³.«¹⁴

Luhmann bezieht sich mit seiner soziologischen Systemtheorie auf »The Laws of Form«¹⁵ von George Spencer Brown. Von diesem stammt die Aussage »draw a distinction«, auf die sich Niklas Luhmann mit der ersten Formfestlegung eines Kunstwerks bezieht. Auf George Spencer Brown beziehen sich auch einige Kybernetiker, unter anderem Max Bense und seine Informationsästhetik, bei denen es um Systeme, aber auch um eine Art der konstruktivistischen Weltanschauung geht. Das möchte ich vor allem deshalb erwähnen, weil es zeigt, wie sich unterschiedliche Disziplinen gegenseitig bereichern können und miteinander verknüpft sind.

¹³ Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp 1997. S. 76.

¹⁴ Franz Schuh: Schöpfung ohne Zentrum, 1996. https://www.zeit.de/1996/10/Schoepfung_ohne_Zentrum/komplettansicht [22.5.2020]

¹⁵ George Spencer Brown: Laws of Form – Gesetze der Form. Leipzig: Bohnmeier Verlag, 1997.

Schnell habe ich geahnt, dass sich mir mit den Ideen von Niklas Luhmann ein gewaltiger Themenkomplex erschließt, den ich leider nur fragmentarisch anreißen kann, und von dem ich manches nur im Ansatz verstehen werde. Trotzdem habe ich das Gefühl, dass seine Gedanken und Theorien ein inspirierendes Werkzeug für das Ausformulieren meiner Ideen sind. Ich werde versuchen einzelne seiner Gedanken in meine Arbeit einzuflechten, mit der Hoffnung, eine Ahnung von dem Komplex zu vermitteln, ohne dabei beliebig zu sein oder Dinge zu behaupten, zu denen ich nicht in der Lage bin.

Algorithmus und Improvisation

Mit algorithmischer Komposition in der Filmmontage (algorithmischer Filmmontage) meine ich das algorithmisch zeitliche Anordnen von Filmmaterial. Der Unterschied zur algorithmischen Filmkomposition besteht darin, dass algorithmische Filmkomposition auch die Erzeugung der Bilder und Töne enthalten kann, während sich die algorithmische Filmmontage auf die zeitliche Strukturierung dieser beschränkt.

Ich habe viel Literatur über algorithmische Komposition in der Musik gefunden. Dagegen kaum Literatur, die sich mit der Algorithmik im Film beschäftigt. Das liegt vermutlich daran, dass in der Musik das Formelhafte offensichtlich ist, und dass es die Musik und damit auch deren algorithmische Komposition schon länger gibt. Dennoch gibt es bereits seit 1920, anfangend mit dem abstrakten oder absoluten Film, über den Flickerfilm um 1960, bis hin zu aktuellen Experimenten des Machine-Learning mit künstlichen neuronalen Netzen, Versuche Film algorithmisch zu strukturieren. Meist ohne, dass der Begriff der algorithmischen Filmmontage dabei explizit genutzt wird.

»Der abstrakte Film oder absolute Film ist eine experimentelle Filmbewegung der Avantgarde, die in den 1920er Jahren entstand. Er löste sich von den erzählerischen Strukturen des von Literatur und Fotografie geprägten Handlungsfilms und stellte eine rein visuelle Wirkung durch die rhythmisierende Strukturierung von Farbe und abstrakten Formen in den Vordergrund. Die Einflüsse kamen daher vor allem aus der Malerei und Musik. Die bekanntesten Vertreter des absoluten Films waren Oskar Fischinger, Viking Eggeling, Hans Richter und Walter Ruttmann. Die Beiträge des französischen Kinos zum abstrakten Film wurden unter dem Schlagwort Cinéma pur bekannt.«¹⁶

¹⁶ https://de.wikipedia.org/wiki/Absoluter_Film [22.5.2020].

Der Begriff der algorithmischen Montage (algorithmic editing) wurde 2002 von Lev Manovich geprägt. Ich habe mir die Freiheit genommen, das Wort Editing mit dem Begriff der Montage zu übersetzen:

»Explicitly, algorithmic editing refers to any method of editing based on direct procedural approaches. In other words, algorithmic editing can be seen as a technique for cutting and reassembling raw footage by following a schema or score. Here is an example of a simple, one line algorithm that could be used to algorithmically edit a film. Sequentially use every odd frame from one sequence of film and every even frame from another sequence of film to assemble a new film which alternates between the odd and the even frames. The resulting algorithmically edited flicker film would rapidly alternate between the two sequences. Creating this film using two filmstrips would be difficult without the labored use of an optical printer. On the other hand, producing this film from two video sequences would be difficult without the use of a script or specially made plug-in; nevertheless, a script for this algorithm would only consist of two or three lines of code.«¹⁷

Ähnlich wie die Improvisation kann man auch das Algorithmische in vielen Bereichen des Lebens finden. Auch wenn man die Improvisation hauptsächlich in der Musik und den Algorithmus hauptsächlich in der Mathematik und in der Informatik verortet:

»[...] we can say that human culture consists of millions of interconnected algorithms that help us communicate, share knowledge and believe in the existence of a stable and somewhat predictable reality. In this scenario, all kinds of activities are involved, from the execution of menial work, such as tying shoelaces, to highly complex activities, like eye surgery; from the very sensual, like wine tasting, to the highly intellectual, like writing a book. For each of these situations, algorithms provide an

¹⁷ Clint Enss: A Brief History of Algorithmic Editing, 2018. <https://medium.com/janbot/a-brief-history-of-algorithmic-editing-732c3e19884b> [22.5.2020].

actionable blueprint for our individual experiences. They suggest what to do, when and how to react in order to reach the intended goal and share the knowledge that comes from such experiences. Even highly spiritual activities, such as praying, meditating, or making love are shaped by social algorithms that help us confirm our experiences with others, empathize and feel connected.«¹⁸

Nach dieser allgemeinen Definition kann man sagen, das Algorithmus und Improvisation unmittelbar voneinander abhängen. Oder besser gesagt, es braucht nicht-determinierte algorithmische Strukturen für die Improvisation. Also Algorithmen, die sich durch eine Offenheit auszeichnen, die mit einem Computer nicht umsetzbar ist. Nach Christopher Dell könnte man sagen, Handlung entsteht aus Struktur und Struktur entsteht aus Handlung. Dabei kann die Struktur des Kunstwerks als das Algorithmische und die Handlung der Künstler als eine Zusammensetzung algorithmischer und improvisatorischer Anteile gesehen werden:

»Wenn Handlung Strukturen herstellt, können wir dann noch Strukturen so fassen, wie wir gemeinhin gewohnt sind sie zu fassen, und wie sie auch Giddens fasst? Nämlich als außerhalb von der Zeit stehend. Das ist essenziell, eine Struktur muss [nach Anthony Giddens] außerhalb von der Zeit stehen, sie muss auch außerhalb von der Handlung stehen, denn sonst ist sie keine Struktur als Ressource, auf die wir zurückgreifen können, um überhaupt handeln zu können. Wie ist es aber dann bestellt, wenn Strukturen aus Handlungen entstehen, also im Zeitverlauf eingebettet sind? Wie kommen die Strukturen in die Zeit hinein, wie können wir uns zu den Strukturen so verhalten, dass die Strukturen uns befähigen in dem wie wir handeln, wenn Handeln überhaupt den Raum herstellt?«¹⁹

18 Pablo Núñez Palma: An algorithmic mindset for filmmaking, 2018. <https://medium.com/janbot/an-algorithmic-mindset-for-filmmaking-572a04c4cd04> [22.5.2020].

19 Christopher Dell: Raumwandler, 2012. <https://www.youtube.com/watch?v=ExCCmp9NI58> (Position 11:14) [22.5.2020].

Für das Zulassen der Improvisation muss der Algorithmus (des sich selbst programmierenden Kunstwerks) nicht-determiniert sein. Er muss zumindest an bestimmten Stellen bewusste (nicht durch den Zufall bestimmte) Entscheidungen zulassen, die unterschiedliche Resultate ermöglichen. Ein Beispiel dafür ist, wenn mehrere Menschen aus dem gleichen Filmmaterial etwas montieren. Es werden sicherlich unterschiedliche Ergebnisse entstehen, wahrscheinlich, ohne dass es eine objektiv beste Version gibt. Wenn man aber sagt, dass die persönliche Geschichte der Montierenden, und nicht nur das Material, Teil der Montage sind, so würde unter denselben Umständen vielleicht auch wieder das gleiche Ergebnis entstehen.

Ein anderer Ansatz ist, das Algorithmische im Film über bereits bestehende Strukturen zu suchen, zum Beispiel über Genres oder Stile (oder über die Untertitelstruktur, wie im Fall des Montage-Automaten). Pablo Núñez Palma bezeichnet auch die Reise des Helden als Algorithmus:

»When it comes to the kind of stories that populate cinema, one of the most famous algorithms is the so-called Hero's Journey, or monomyth. This algorithm is centered on a main character, called the hero, who proceeds through a sequence of life-changing adventures wherein he confronts his deepest inner fears, eventually returning home as a stronger and wiser person.«²⁰

Eine besondere Form des Algorithmischen ist die algorithmische Komposition. Mit dieser habe ich mich im Wesentlichen mit meinen Programmierexperimenten auseinandergesetzt. Sie ist das Anordnen von Film- und Tonmaterial nach mathematischen Formeln. Auch dabei spielt die Improvisation wieder eine Rolle, allerdings mehr im Programmieren, also im Erstellen des Algorithmus, als in der Generierung des Kunstwerks selbst.

20 Pablo Núñez Palma: An algorithmic mindset for filmmaking, 2018. <https://medium.com/janbot/an-algorithmic-mindset-for-filmmaking-572a04c4cd04> [22.5.2020].

Eine kleine Geschichte der algorithmischen Filmkomposition

Auf den folgenden Seiten stelle ich einige Beispiele aus der Geschichte der algorithmischen Filmkomposition in chronologischer Reihenfolge vor. Ich habe mich dazu entschieden, die Beispiele größtenteils anhand von Notationen zu veranschaulichen, da sie mehr über den Aufbau des algorithmischen Films verraten, als ein einzelner Screen-shot es könnte. Auch haben die Notationen eine Nähe zur musikalischen Komposition. Die vorgestellten Arbeiten sind zwischen den Jahren 1922 und 1971 entstanden und wurden ohne einen Computer erstellt. Dies wiederum erklärt auch, warum die Notationen für den Herstellungsprozess notwendig waren, oder zumindest die Arbeit erheblich erleichterten. Man konnte nicht wie heute mit einem Computer durch Verändern von Parametern quasi in Echtzeit auf das Material und die Struktur reagieren (das gilt natürlich nur bedingt, da auch das Programmieren sehr viel Zeit beanspruchen kann). Stattdessen musste das erdachte Konzept, also der Algorithmus, handwerklich am Schneidetisch umgesetzt werden. Damit will ich nicht sagen, dass die Notation in Zeiten des Computers nicht mehr notwendig wäre. Das kann nach wie vor für die Entwicklung der Arbeit und ein Verstehen von Zusammenhängen sehr nützlich sein. Aber wenn ein Computerprogramm vorliegt, ist es um ein vielfaches einfacher damit Variationen zu erzeugen. Erste Computerprogramme für den Filmschnitt sind, wie die Computerkunst, erst um 1990 herum entstanden. Für die klassische Filmmontage sind diese Programme inzwischen sehr ausgereift. Ich persönlich habe alles, was ich brauche, und vieles von dem man vor wenigen Jahrzehnten noch geträumt hat. Dabei geht es im Wesentlichen um eine Vereinfachung der handwerklichen Vorgänge in der Montage. Für die algorithmische Filmkomposition wiederum schreibe ich die Computerprogramme selbst, da sie als unsichtbare Notationen spezifisch für das generierte Resultat sind. Also quasi den Vorgang der Montage nicht ermöglichen, sondern, im Rahmen ihrer Möglichkeiten, ersetzen.

»Komp. I/22« und »Komp. II/22« von Werner Graeff aus dem Jahr 1922

Die Arbeiten von Werner Graeff möchte ich deshalb hervorheben, weil sie meine einzigen Funde sind, die den Begriff der Partitur und der Komposition bezogen auf den Film nutzen. Möglicherweise auch deshalb, weil es zur Zeit ihrer Entstehung noch kein ausgeprägtes filmisches Vokabular gab. Dennoch finde ich die von ihm gewählten Bezeichnungen für seine Arbeiten sehr treffend.

»Um genauer zu erklären, wie ich das meinte, entwarf ich meine erste »Filmpartitur I/22«. Es handelte sich um die grafische Notierung sehr einfacher Vorgänge in bestimmtem Tempo und Rhythmus. Als Zeiteinheit – sozusagen als Takt – legte ich 3/4 Sekunden fest. Da jeweils immer nur eine einzelne Farbe auf der Leinwand erscheinen sollte, so hätte man kurze Stücke Film rot oder blau oder gelb einfärben können. (>Virage« wurde das damals genannt), worauf die Komposition aneinanderzustückeln wäre. Jedenfalls war das eine Möglichkeit ... (ideal fand ich sie nicht)«²¹

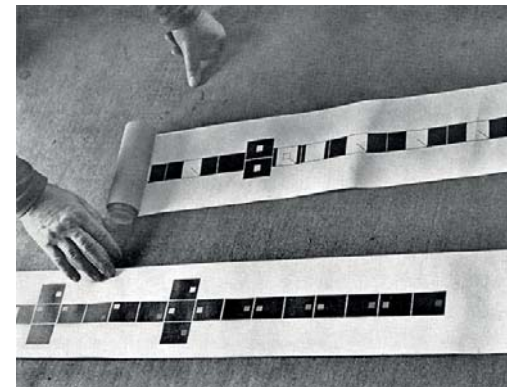


Abb. 3: Werner Graeff mit Siebdrucken seiner Filmpartituren.

²¹ Werner Graeff: Film als Film. 1910 bis Heute, 1977 / 1978.
<http://www.wernergraeff.de/film/film.html> [22.5.2020].

von Sziga Vertov aus dem Jahr 1929

Dieses Beispiel ist das Einzige, das eher dem Dokumentarfilm zuzuordnen ist. Aber auch hier ist der Ansatz ein experimenteller und der Rhythmus, also das musikalische, spielt in der Montage eine große Rolle.

»I work in the field of the poetic documentary film. That's why I feel so close to both the folk songs and the poetry of Majakovskij.«²²

Interessant finde ich die Darstellung des Diagramms (Abb. 4) zur Veranschaulichung der Friseursequenz des Films »Man with a Movie Camera«. Sie erinnert mich an diagrammatische Partituren aus dem musikalischen Bereich, mit denen ich mich in meiner Bachelorarbeit in Bezug auf das Improvisatorische beschäftigt habe. Ebenso könnte das Diagramm auch den Programmablaufplan eines Computerprogramms mit eindeutigen Handlungsanweisungen darstellen.

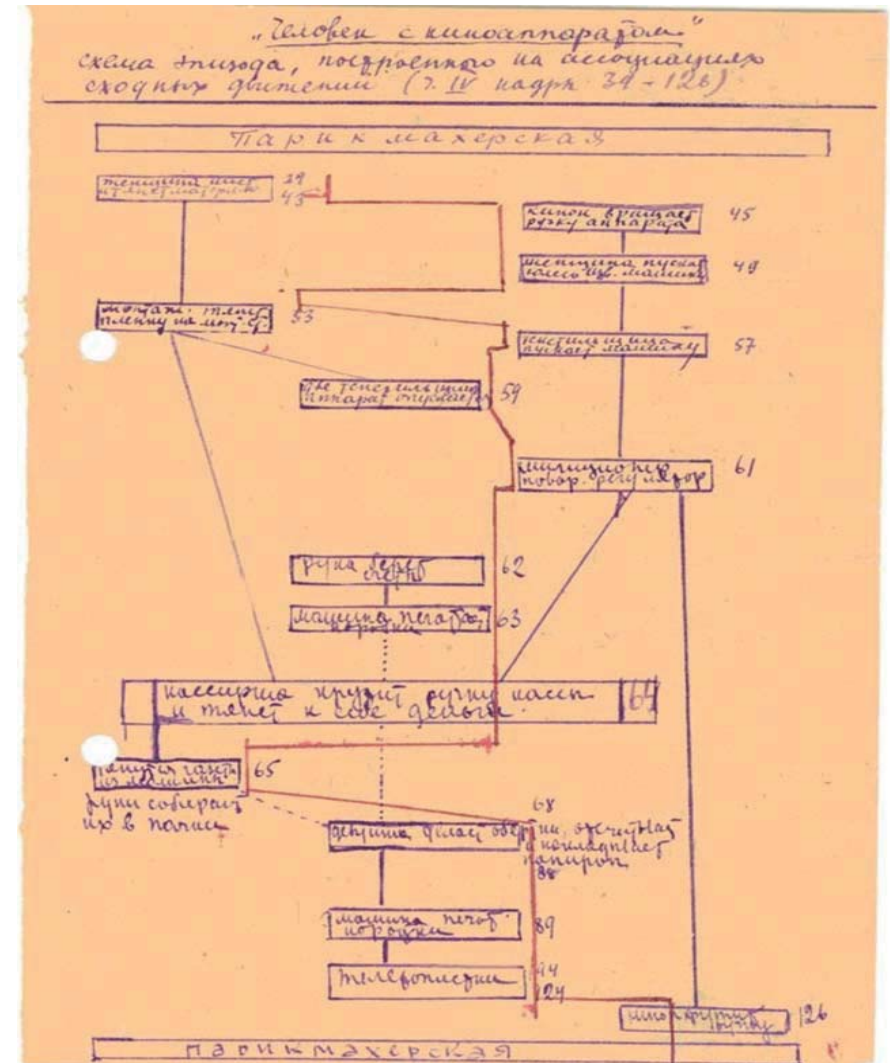


Abb. 4: Montagediagramm für die Friseursequenz in »Man with a Movie Camera« von Sziga Vertov. Der Text in den nummerierten Kästen beschreibt den Bildinhalt, während die Linie dazwischen den Rhythmus der Montage beschreibt.

22 Dziga Vertzov: Film Culture Reader – The Writings of Dziga Vertov. Berkeley: University of California Press, 1984. S. 366.

»Tönende Ornamente« von Oskar Fischinger aus dem Jahr 1932

»Between ornament and music persist direct connections, which means that Ornaments are Music. If you look at a strip of film from my experiments with synthetic sound, you will see along one edge a thin stripe of jagged ornamental patterns. These ornaments are drawn music -- they are sound: when run through a projector, these graphic sounds broadcast tones or a hitherto unheard of purity, and thus, quite obviously, fantastic possibilities open up for the composition of music in the future. Undoubtedly, the composer of tomorrow will no longer write mere notes, which the composer himself can never realize definitively, but which rather must languish, abandoned to various capricious reproducers. Now control of every fine gradation and nuance is granted to the music-painting artist, who bases everything exclusively on the primary fundamental of music, namely the wave -- vibration or oscillation in and of itself.«²³

Oskar Fischingers »Tönende Ornamente« ist eines seiner audiovisuellen Experimente, bei denen die Bild- und Toninformation identisch ist. Die auf das Filmmaterial aufgebrachten Muster, die man auch als Notationen sehen kann, werden sowohl von einem Bild- als auch von einem Tonprojektor abgetastet und wiedergegeben. Hier findet man das Algorithmische in der Zuordnung von Form und Klang, sie sind unmittelbar voneinander abhängig. Auch der Aufbau der Muster selbst hat etwas Algorithmisches, da sie den physikalischen Gesetzen der Klangsynthese folgen, und erst durch eine regelmäßige Wiederholung konstante Klänge und Rhythmen entstehen können.

²³ Oskar Fischinger: Sounding Ornaments, 1932. <http://www.center-forvisualmusic.org/Fischinger/SoundOrnaments.htm> [22.5.2020].

Ein weiteres Zitat von Oskar Fischinger aus dem Vorspann des Films »An Optical Poem« aus dem Jahr 1938. Interessant finde ich das er seine Arbeit nicht Kunst, sondern wissenschaftliches Experiment nennt. Vermutlich liegt es auch daran, dass der Tonfilm zu dieser Zeit ein verhältnismäßig neues Medium war, dessen Möglichkeiten erst noch erforscht werden mussten:

»To most of us music suggests definite mental images of form and color. The picture you are about to see is a novel scientific experiment - its object is to convey these mental images in visual form.«²⁴

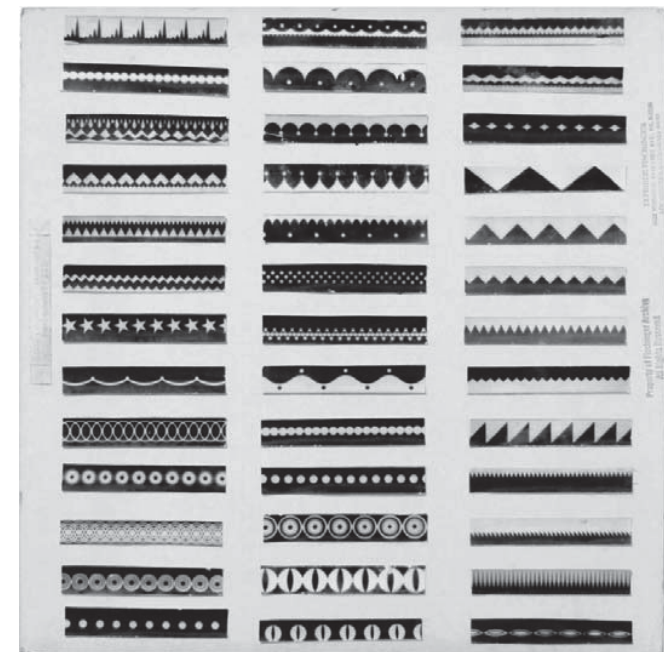


Abb. 5: Filmstreifen aus Oskar Fischingers »Tönende Ornamente«.

²⁴ Oskar Fischinger: An Optical Poem, 1938. <https://www.youtube.com/watch?v=6Xc4g00FFLk> (Position 00:35) [22.5.2020].

**»6 / 64 Mama und Papa«
von Kurt Kren aus dem Jahr 1964**

»Shot/countershot sequences alternate, lumping back and forth between single (!) frames, they turn the Actionist turmoil into ornaments, rigid geometrical patterns, the equivalent in time to what Mondrian used to distill on canvas in space.«²⁵

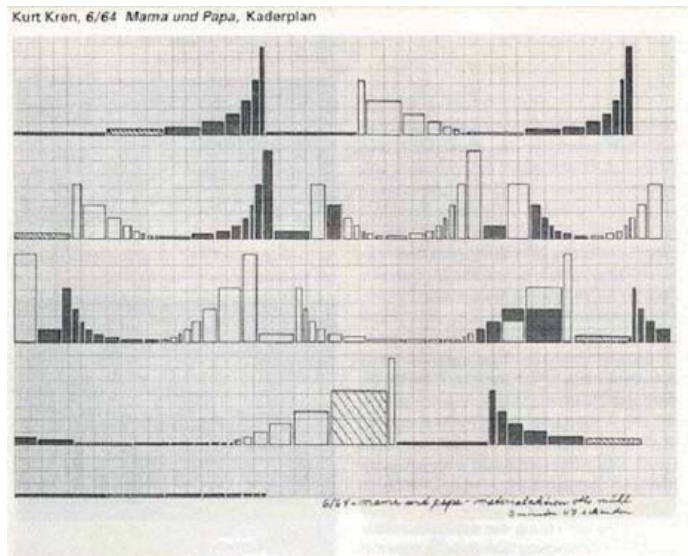


Abb. 6: Montagediagramm (Kaderplan) für »6 / 64 Mama und Papa« von Kurt Kren.

²⁵ Peter Tscherkassky: Mama and Papa. <https://kurtkren.info/> [22.5.2020].

**»The Flicker«
von Tony Conrad aus dem Jahr 1965**

»Ein Film der nur aus abwechselnden schwarzen und weißen Filmbildern besteht. Bei der Projektion erzeugen die Hell- und Dunkelsequenzen, die in veränderlichen Rhythmen alternieren, stroboskopische und flimmernde Effekte. Diese können beim Betrachten optische Eindrücke wie Farb- oder Formensehen hervorrufen. Der Film stimuliert dabei physiologische anstelle psychologischer Eindrücke indem er nicht die Sinne anspricht, sondern direkt neurale Reaktionen auslöst. Tony Conrad, der sich intensiv mit der Physiologie des Nervensystems beschäftigt hat, hat mit »The Flicker« eine Ikone des strukturellen Films geschaffen, die ganz ohne narrative oder reproduzierende Bilder auskommt. Denn diese werden nicht durch das Auge aufgenommen, sondern erst im Gehirn produziert.«²⁶

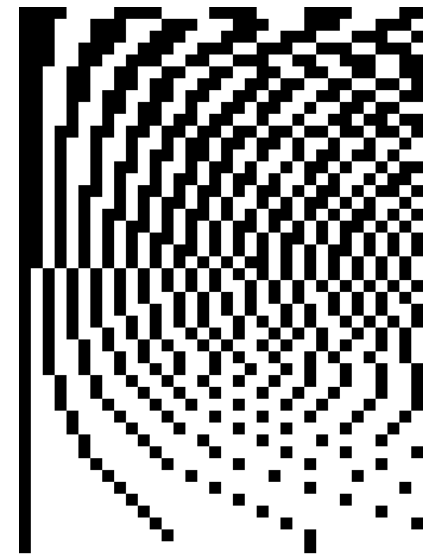


Abb. 7: Plan für die Abfolge der schwarzen und weißen Bilder in Tony Conrads Film »The Flicker«.

²⁶ Heike Helfert: Tony Conrad »The Flicker«. <http://www.medienkunstnetz.de/werke/the-flicker/> [22.5.2020].

»Synchrony« von Norman McLaren aus dem Jahr 1971

Da ich zu diesem Film keine Notationen fand, habe ich einen Screenshot zur Veranschaulichung gewählt. Gleichzeitig ist der Screenshot in diesem Fall eine Notation im musikalischen Sinn, da das experimentelle Verfahren des Films »Synchrony«, wie bei Oskar Fischingers »Tönende Ornamente«, darin besteht, dass das Bildmaterial dem Tonmaterial entspricht. Es wird bei der Projektion sowohl in sichtbare, als auch in hörbare Frequenzen umgewandelt. Im Gegensatz zu Oskar Fischinger sieht Norman McLaren seine Experimente nicht (mehr) als wissenschaftlich an:

»Music has had a great effect on me and in quite an old-fashioned way since I have found some music ›inspiring‹. Certain music had the power of bringing up before my mind's eye certain kinds of moving imagery and some of my earlier films were attempts to put such imagings onto film. But although I may have once thought that I was, if only intuitively, giving a visual translation of the music in my film in a way that might conceivably have some logical scientific basis, I tend now to consider that naive.«²⁷

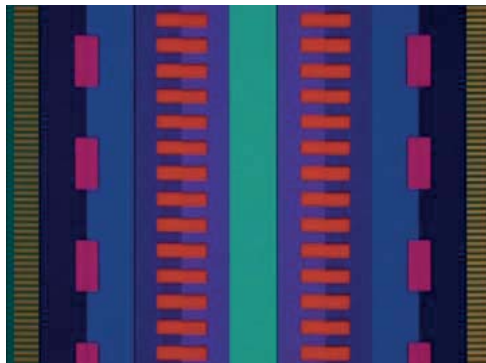


Abb. 8: Screenshot aus »Synchrony« von Norman McLaren.

²⁷ Norman McLaren: Letter to Thea Goldberg, 1973, S. 3. NFB Archives. Terence Dobsen: The Film Work of Norman McLaren. Canterbury: University of Canterbury, 1994. S. 198.

»Pure Data« und das Programmieren

An dieser Stelle gehe ich auf die von mir genutzten Programmiersprachen und Programmierumgebungen ein. Dabei beschränke ich mich hauptsächlich auf »Pure Data«, weil damit das Programmieren für mich angefangen hat, und es nach wie vor mein zentrales Werkzeug in der algorithmischen Kunst ist.

Ich bin erst durch meine Masterarbeit zum Programmieren gekommen und auch das war nicht von Anfang an geplant. Ursprünglich wollte ich mich mit der algorithmischen Komposition nur theoretisch auseinandersetzen, mir einen fiktiven Montage-Automaten als Gegenstück zur Improvisation ausdenken. Durch ein Seminar über Interfaces bei Gesa Marten habe ich dann erste Versuche mit der Programmiersprache »Pure Data« unternommen. Dabei habe ich Videoeinstellungen Codes zugeordnet und auf Karten angebracht. Durch das Anordnen der Karten lässt sich, dadurch dass die Codes von einer Kamera gelesen werden, mit dem Programm eine Videosequenz erstellen. Auch wenn es nur ein halbwegs funktionierender Prototyp war, hat es mir die Angst vor dem Programmieren genommen, und mich auf die Idee gebracht, mich im Rahmen meiner Masterarbeit auch praktisch mit der algorithmischen Komposition zu beschäftigen.

In der folgenden Zeit habe ich viele Abende mit meinem Bruder Johannes, der zu dieser Zeit in Berlin gewohnt hat, und Ingo Stock verbracht und über das Programmieren diskutiert sowie verschiedene Projekte, wie den Melodie-Generator, den Markov-Generator oder den Montage-Automaten, realisiert. Das war, abgesehen von den entstandenen Ergebnissen, eine wichtige und schöne Zeit des Lernens, die mir das Programmieren als eine Form der künstlerischen Ausdrucksweise näher gebracht hat.

Alle meiner im Rahmen dieser Arbeit vorgestellten Experimente, bis auf die zweite Version des Montage-Automaten, habe ich mit »Pure Data« programmiert:

»Pure Data (Abkürzung: Pd) ist eine datenstromorientierte Programmiersprache und Entwicklungsumgebung, die visuelle Programmierung benutzt. Sie wird vor allem zur Erstellung von interaktiver Multimedia-Software eingesetzt, etwa für Software-Synthesizer in der elektronischen Musik. [...] Pure Data wurde in den 1990ern von Miller Puckette entwickelt, um damit interaktive Computermusik zu erzeugen. In seinem Umfang und seinen Zielen ist Pure Data dem ursprünglichen Max sehr ähnlich, das ebenfalls von Puckette entwickelt wurde und der Vorgänger des kommerziellen MSP ist. Im Gegensatz zu Max/MSP handelt es sich bei Pd um freie/Open-Source-Software. Pd besitzt eine aktive Entwickler-Community.«²⁸

Neben der Erzeugung und Strukturierung von Klang, lässt sich mit »Pure Data« auch visuelles Material generieren und anordnen. Dazu werden zusätzlich Bibliotheken, wie zum Beispiel »Gem« oder »Ofelia« benötigt.

Die visuelle Darstellungsform der Patches genannten Programme hat es mir erleichtert, ein Verständnis vom Programmieren zu bekommen. Man arbeitet mit Objekten und Nachrichten, die mit virtuellen Kabeln verbunden werden. So deutet schon die visuelle Erscheinung des Programms, ähnlich einer Notation, dessen Datenfluss und Funktionsweise an.

Wichtig ist außerdem die Internet-Gemeinschaft der »Pure Data« Nutzer und Entwickler. Sowohl für die Entwicklung von »Pure Data« selbst, als auch zum Austausch über konkrete Projekte, spezielle technische Probleme, oder künstlerische Resultate. Dabei bin ich vor allem in dem Forum »Pure Data Patch Repository«²⁹ aktiv, es gibt aber auch weitere Kommunikationskanäle wie Mailing-Listen oder eine Facebook-Gruppe. Das alles ersetzt natürlich nicht die Qualität des persönlichen Gesprächs und Austauschs, hat aber dennoch auch bei mir schon zu produktiven Kollaborationen und Inspirationen geführt. Die Online-Kommunikationskanäle und Kollaborationen sind natürlich nichts,

²⁸ https://de.wikipedia.org/wiki/Pure_Data [22.5.2020].

²⁹ <https://forum.pdpatchrepo.info> [22.5.2020].

was für die Programmiersprache »Pure Data« spezifisch ist, auch für die Filmmontage und fast jeden anderen Bereich gibt es das. Dennoch nimmt die Internet-Gemeinschaft in meiner Arbeit mit der algorithmischen Kunst, besonders auch das erwähnte »Pure Data« Forum, einen deutlich höheren Stellenwert ein, als das in der Filmmontage der Fall ist.

Die erste nicht datenstromorientierte, sondern textbasierte Programmiersprache, mit der ich mich beschäftigt habe, ist »Python«. Mein Bruder Johannes hat vorgeschlagen, sie für die zweite Version des Montage-Automaten zu nutzen. »Python« wird viel im Bereich der wissenschaftlichen Datenverarbeitung und dem Machine-Learning eingesetzt und ist deshalb für die Aufgaben des Montage-Automaten gut geeignet. Auch für zukünftige künstlerische Projekte, wie dem Experimentieren mit künstlichen neuronalen Netzen, könnte »Python« für mich interessant sein.

Durch die Arbeit mit der »Pure Data« Bibliothek »Ofelia« von Zack Lee, die es seit 2018 gibt, bin ich mit einer Reihe weiterer Sprachen, wie »C++« (»openFrameworks«), »Java Script« und »Lua«, in Berührung gekommen. Zu kleinen Teilen bin ich inzwischen auch in die Entwicklung von »Ofelia« involviert, beziehungsweise sind Ideen von mir eingeflossen:

»Ofelia is a Pd external which allows you to use openFrameworks and Lua within a real-time visual programming environment for creating audiovisual artwork or multimedia applications such as games. openFrameworks is an open source C++ toolkit for creative coding. Lua is a powerful, efficient, lightweight, easy-to-learn scripting language. Pure Data(Pd) is a real-time visual programming language for multimedia. Thanks to Lua scripting feature, you can do text coding directly on a Pd patch or through a text editor which makes it easier to solve problems that are complicated to express in visual programming languages like Pd. And unlike compiled languages like C/C++, you can see the result immediately as you change code which

enables faster workflow. Moreover, you can use openFrameworks functions and classes within a Lua script.»³⁰

Die Bibliothek »Ofelia« erweitert »Pure Data« um einige, für mich sehr interessante, Möglichkeiten. Mit ihr lassen sich zum Beispiel komplexe Bedienoberflächen und Visualisierungen programmieren. Sie ist außerdem für die algorithmische Filmmontage und Videoverarbeitung sehr gut geeignet. Ebenso lassen sich Programme als ausführbare Dateien erstellen, das bedeutet »Pure Data« mit Erweiterungen muss nicht installiert sein, um die Programme ausführen zu können. Und es lassen sich »Pure Data« Programme über die Bibliothek »Ofelia« mit »Emscripten«³¹ in Internetseiten einbinden.

30 <https://github.com/cuinjune/Ofelia> [22.5.2020].

31 <https://emscripten.org> [22.5.2020].

Die diagrammatische Partitur

Computerprogramme können zur Veranschaulichung in Form eines Programmablaufplans oder Flussdiagramms dargestellt werden. Die Abläufe erinnern mich an musikalische Notationen in Form von diagrammatischen Partituren. Auch in der datenstromorientierten Programmierung ist das Diagrammatische noch deutlich zu erkennen, während es im Textbasierten hinter einer verschriftlichten Codierung verschwindet.

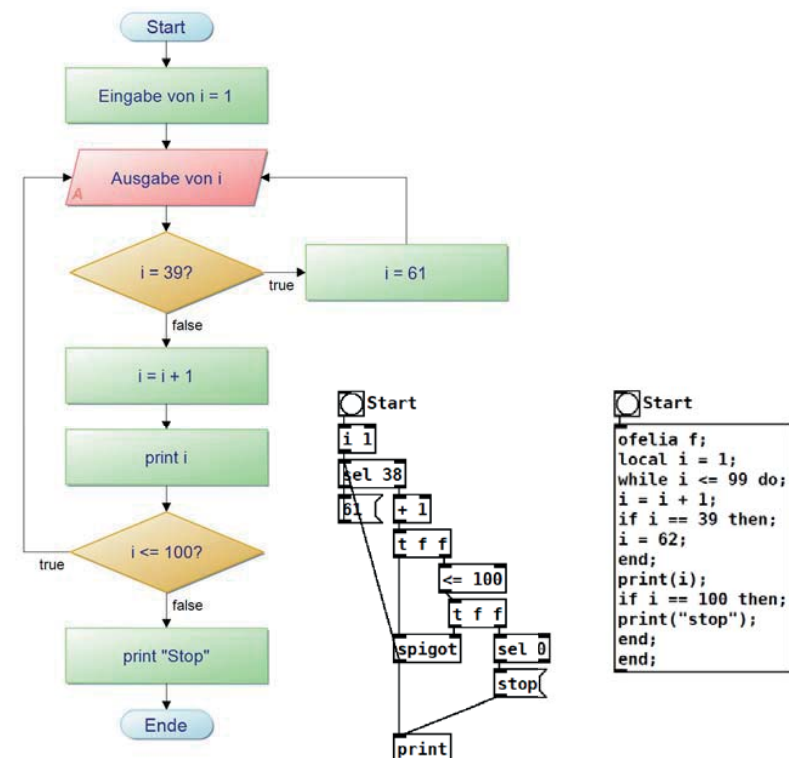


Abb. 9: Ein Programm als Programmablaufplan, datenstromorientierter Programmierung und textbasierter Programmierung (von links nach rechts).

Markov-Ketten

Am Rande eines Seminars des Performancekünstlers, Komponisten und Autoren des Computerprogramms für interaktive Performance »Isadora«³², Mark Coniglio, habe ich mit ihm über meine Idee gesprochen, im Rahmen meiner Masterarbeit einen Montage-Automaten zu programmieren. Ich dachte dabei an künstliche neuronale Netzwerke, aber er meinte, ich könne mich als Grundlage zunächst mit Markov-Ketten als stochastischem Verfahren zur Erzeugung von Sequenzen beschäftigen.

»Markov chains are a fairly common, and relatively simple, way to statistically model random processes. They have been used in many different domains, ranging from text generation to financial modeling. A popular example is SubredditSimulator³³, which uses Markov chains to automate the creation of content for an entire subreddit. Overall, Markov Chains are conceptually quite intuitive, and are very accessible in that they can be implemented without the use of any advanced statistical or mathematical concepts. They are a great way to start learning about probabilistic modeling and data science techniques.«³⁴

Wir, genauer gesagt Ingo Stock, mein Bruder Johannes und ich, wollten uns, aufgrund der Annahme, dass die Relation von Tönen einfacher zu ermitteln ist als die von Bildern, mit diesem Verfahren zuerst im musikalischen Versuchen. Die Relation der Töne haben wir zunächst auf die Tonabstände, Intervalle, beschränkt. Anhand von Tonfolgen haben wir dann die Wahrscheinlichkeiten der Folgetöne berechnet, um durch aneinanderreihen der Töne neue Tonfolgen, sogenannte Markov-Ketten zu bilden.

³² <https://troikatronix.com> [22.5.2020].

³³ <https://www.reddit.com/r/SubredditSimulator> [22.5.2020].

³⁴ Devin Soni, <https://towardsdatascience.com/introduction-to-markov-chains-50da3645a50d> [22.5.2020].

Aus dieser ersten Beschäftigung mit Markov-Ketten ist ein längeres Projekt entstanden, das sich über mehrere Versionen und Ansätze entwickelt hat und nun mehr oder weniger abgeschlossen ist. Das Ergebnis sind unterschiedliche Markov-Generatoren, die zum Beispiel aus MIDI-Noten musikalische Markov-Ketten erzeugen können.

Ich beschreibe den Markov-Prozess anhand der gegebenen Tonfolge A-C-C-D-A-D-C-D-D-A mit der Länge 10 und den drei unterschiedlichen Tönen A, C und D. Die Zustände, aus denen sich die Markov-Kette berechnet, sind die Variablen, die in der Folge beobachtet werden, in diesem Fall die unterschiedlichen Töne. Die Ordnung der Markov-Kette beschreibt die Länge der Sequenz, aus der sich die Wahrscheinlichkeiten der Folgetöne ergeben.

Die Wahrscheinlichkeit, dass ein bestimmter Ton als Folgeton eintritt, liegt zwischen 0 und 1. Dabei bedeutet 0 das Ereignis tritt in keinen Fall ein und 1 das Ereignis tritt in jedem Fall ein. Die Summe aller Wahrscheinlichkeiten der möglichen Folgetöne / Zustände beträgt 1.



Abb. 10: Bei einer Markov-Kette erster Ordnung wird die Wahrscheinlichkeit der Folgetöne anhand des vorherigen Tons bestimmt.

ÜBERGANGSMATRIX

		VON:		
		A	C	D
NACH:	A	$\begin{pmatrix} 0 & 0 & 1/2 \\ 1/2 & 1/3 & 1/4 \\ 1/2 & 2/3 & 1/4 \end{pmatrix}$		
	C			
	D			

$A \rightarrow C = 1/2, A \rightarrow D = 1/2$
 $C \rightarrow C = 1/3, C \rightarrow D = 2/3$
 $D \rightarrow A = 1/2, D \rightarrow C = 1/4, D \rightarrow D = 1/4$

Abb. 11: Eine Übergangsmatrix zur Berechnung von Markov-Ketten erster Ordnung.

Mit einer Übergangsmatrix (Abb. 11) kann man anhand von Zeilen und Spalten die Wahrscheinlichkeit des nächsten Zustands einer Markov-Kette ablesen. Dabei bezeichnet die Spalte den derzeitigen Zustand und die Zeile den darauf folgenden. Während die Übergangsmatrix eine Grundlage für weitere Berechnungen ist, dient das Prozessdiagramm (Abb. 12) einer besseren Veranschaulichung.

Verschriftlicht bedeuten die Übergangsmatrix und das Prozessdiagramm:

Wenn der aktuelle Zustand A ist, wird der nächste Zustand mit einer Wahrscheinlichkeit von 1/2 Zustand C sein, und mit einer Wahrscheinlichkeit von 1/2 Zustand D sein.

Wenn der aktuelle Zustand C ist, wird der nächste Zustand mit einer Wahrscheinlichkeit von 1/3 Zustand C bleiben, mit einer Wahrscheinlichkeit von 2/3 Zustand D sein.

Wenn der aktuelle Zustand D ist, wird der nächste Zustand mit einer Wahrscheinlichkeit von 1/2 Zustand A sein, mit einer Wahrscheinlichkeit von 1/4 Zustand C sein, und mit einer Wahrscheinlichkeit von 1/4 Zustand D bleiben.

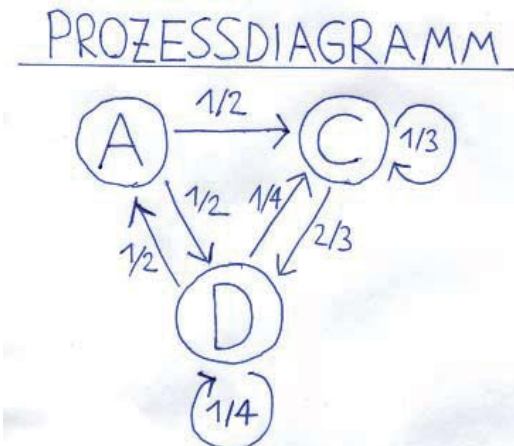


Abb. 12: Ein Prozessdiagramm zur Veranschaulichung der Folgewahrscheinlichkeiten für Markov-Ketten erster Ordnung.

Eine Markov-Kette höherer Ordnung wird nun daraus, wenn sich die Wahrscheinlichkeit des nächsten Zustands aus einer Folge mehrerer vergangener Zustände berechnet. Dabei bezeichnet die Ordnung die Anzahl der betrachteten Zustände. Je höher die Ordnung der Markov-Kette, desto weniger Möglichkeiten der Verkettung gibt es, und umso ähnlicher wird das generierte Material dem Ursprünglichen sein. Zum Beispiel ist die Frage bei einer Markov-Kette zweiter Ordnung, gebildet aus Tonhöhen: Wie groß ist die Wahrscheinlichkeit, aufgrund der letzten zwei Töne, die gespielt wurden, dass ein bestimmter Folgeton gespielt wird?



Abb. 13: Bei einer Markov-Kette zweiter Ordnung wird die Wahrscheinlichkeit der Folgetöne anhand der zwei vorherigen Töne bestimmt.

ÜBERGANGSMATRIX

VON:

	AA	AD	AC	CA	CC	CD	DA	DC	DD	
NACH:	A	(0	0	0	0	0	1/2	0	0	1)
	C	(0	1	1	0	0	0	0	0	0)
	D	(0	0	0	0	1	1/2	1	1	0)

$AC \rightarrow C = 1$
 $AD \rightarrow C = 1$
 $CC \rightarrow D = 1$
 $CD \rightarrow A = 1/2, CD \rightarrow D = 1/2$
 $DA \rightarrow D = 1$
 $DC \rightarrow D = 1$
 $DD \rightarrow A = 1$

Abb. 14: Eine Übergangsmatrix zur Berechnung von Markov-Ketten zweiter Ordnung.

Darüber hinaus kann ein Zustand einer Markov-Kette nicht nur aus einer Variablen bestehen, sondern auch aus mehreren. Wieder auf eine Tonfolge bezogen, können das zum Beispiel Akkorde, also mehrere gleichzeitig gespielte Töne, sein. Aber auch Lautstärke, Klang oder andere Parameter können zusätzliche Variablen sein.



Abb. 15: Es können auch mehrere Variablen für die Berechnung von Folgewahrscheinlichkeiten einer Markov-Kette genutzt werden.

$$\begin{aligned}
 &A \rightarrow C = 1 \\
 &C \rightarrow A = 1 \\
 &C \rightarrow C = 1 \\
 &A \rightarrow D = 1 \\
 &C \rightarrow D = 1 \\
 &D \rightarrow A = 1 \\
 &D \rightarrow A = 1/2, D \rightarrow C = 1/2 \\
 &A \rightarrow D = 1 \\
 &A \rightarrow D = 1 \\
 &D \rightarrow C = 1/2, D \rightarrow D = 1/2 \\
 &C \rightarrow D = 1 \\
 &C \rightarrow D = 1
 \end{aligned}$$

Abb. 16: Folgewahrscheinlichkeiten für Markov-Ketten erster Ordnung mit zwei Variablen am Beispiel einer duofonen Tonfolge (Abb. 15).

Eine Markov-Kette erster Ordnung, am Beispiel der Tonfolge A-C-C-D-A-D-C-D-D-A:

```
A C D A D D A D D A C C D D A C D C C C D D C D C C D C D
C D C D C D C C D D A C D A C D D D C D C D A D D A D C D
A D A C D C C D A C C C D C D A D A C D A C D C D A D C D
D C D A D C C D C D C D C
```

Eine Markov-Kette zweiter Ordnung, am Beispiel der Tonfolge A-C-C-D-A-D-C-D-D-A:

```
A C C D A D C D A D C D A D C D D A D C D D A D C
D D A D C D D A D C D D A D C D A D C D D A D C
D A D C D A D C D A D C D D A D C D D A D C D D
A D C D D A D C D A D C D
```

Eine Markov-Kette dritter Ordnung, am Beispiel der Tonfolge A-C-C-D-A-D-C-D-D-A:

```
A C C D A D C D D A
```

Die Markov-Kette dritter Ordnung entspricht der ursprünglichen Tonfolge, und bricht nach einem Durchgang ab, da auf die letzte Sequenz mit der Länge drei (D-D-A) kein Ton in der Originalfolge folgt. Um die Markov-Kette trotzdem fortzusetzen, könnte man den ersten oder einen zufälligen Ton der Tonfolge wählen.

Während sich die generierte Tonfolge als Markov-Kette erster Ordnung im Vergleich zum Original relativ beliebig anhört, wird eine Markov-Kette dritter Ordnung schon keinerlei Variation mehr aufweisen. In diesem Beispiel hat die Markov-Kette zweiter Ordnung das subjektiv beste Verhältnis von bekannter und neuer Struktur, von Ordnung und Chaos. Je mehr Ausgangsmaterial vorhanden ist, umso mehr Variation wird es vermutlich auch bei Markov-Ketten höherer Ordnung geben. Was sich in der Theorie recht trocken anhört, hat in den praktischen Experimenten zu interessanten, teils unerwarteten musikalischen Ergebnissen geführt.

Auf den folgenden Seiten stelle ich vier der im Rahmen meiner Masterarbeit entstandenen Markov-Generatoren vor. Wie die meisten meiner Computerprogramme sind sie mit der datenstromorientierten Programmiersprache und Entwicklungsumgebung »Pure Data« entstanden.

Markov-Generator 1

Dieser Markov-Generator ist eines der ersten Projekte, die ich zusammen mit Ingo Stock umgesetzt habe. Das Programm kann MIDI-Dateien einlesen oder aufnehmen, um aus den daraus berechneten Markov-Wahrscheinlichkeiten neue Abfolgen zu bilden.³⁵

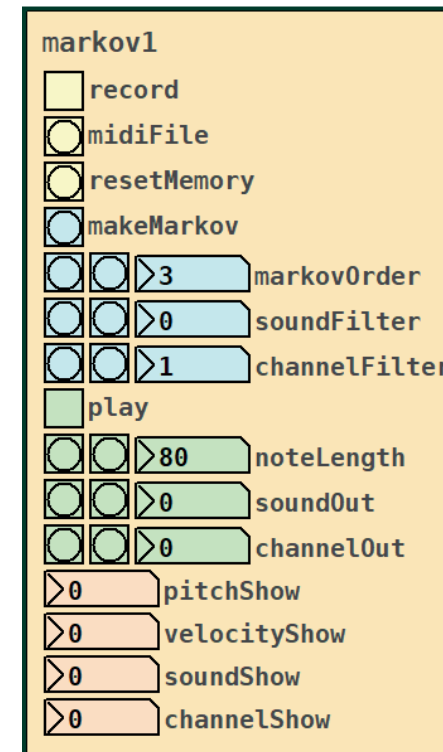


Abb. 17: Ausschnitt der Bedienoberfläche des Markov-Generator 1. Modul für einen MIDI-Kanal.

³⁵ <https://forum.pdpatchrepo.info/topic/10791/markovgenerator-a-music-generator-based-on-markov-chains-with-variable-length> [22.5.2020].

Markov-Generator 2

»A universal abstraction to create Markov chains of any order out of progressions of floats, symbols or lists. Using lists it can be used to create polyphonic Markov chains.

The source material is interpreted as a loop, so that the first item may be played after a chain of the last items.

To create the Markov chains, send some material to the right inlet, set the order and send ›make‹ to the left inlet. Afterwards the result can be played by sending bangs to the left inlet.

Once the material is stored inside the abstraction, the order can be changed and new chains can be created using ›make‹ again. New material can be added at any time. To incorporate it into the Markov chains, send ›make‹ again.

›reset‹ sets the player to the first position, ›random‹ sets it to a random position and ›clear‹ deletes all stored information. The order is automatically clipped to between 1 and the length of the source material.«³⁶

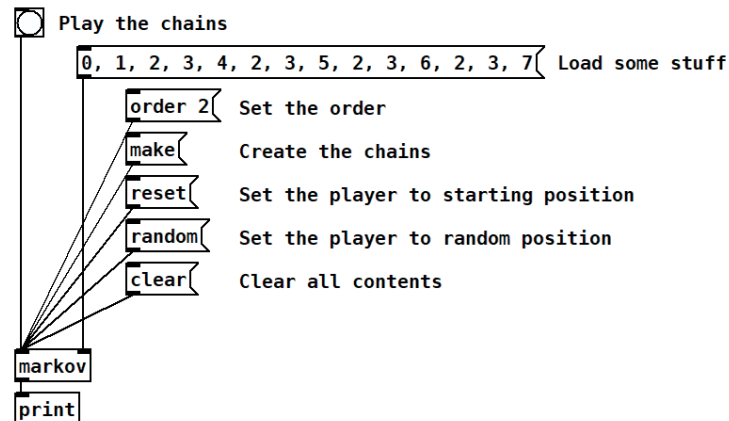


Abb. 18: Bedienoberfläche des Markov-Generator 2.

³⁶ Beschreibung des Programms von Ingo Stock. <https://forum.pdpatchrepo.info/topic/12147/midi-into-seq-and-markov-chains/44> [22.5.2020].

Markov-Generator 3

Dieser Markov-Generator ordnet die Samples einer Klangdatei in Form einer Markov-Kette an. Die Klangdatei kann innerhalb des gelben Rechtecks eingezeichnet werden. Eine Variante dieses Programms ordnet nach demselben Verfahren Tonfolgen in Form von MIDI-Noten an.³⁷



Abb. 19: Bedienoberfläche des Markov-Generator 3.

³⁷ <https://forum.pdpatchrepo.info/topic/11859/lua-ofelia-markov-generator-patch-abstraction> [22.5.2020].

Markov-Generator 4

Dies ist die zum Zeitpunkt des Verfassens dieser Arbeit aktuellste Version des Markov-Generators. Auch dieser ordnet wie der Markov-Generator 1 die Noten von MIDI-Dateien in Form einer Markov-Kette an. Dabei wird als zusätzliche Information die Länge der Töne und die Pausen zwischen den Tönen mit einberechnet.³⁸

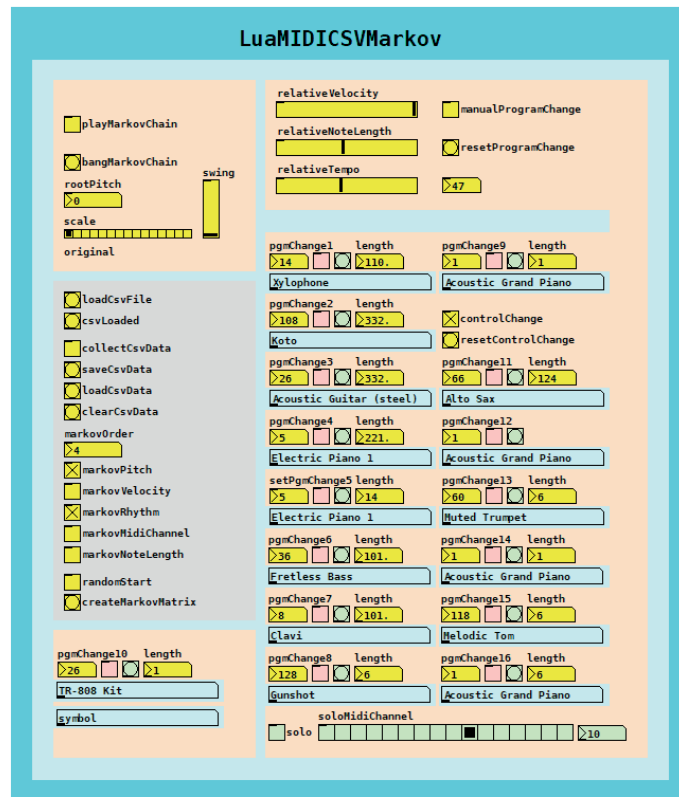


Abb. 20: Bedienoberfläche des Markov-Generator 4.

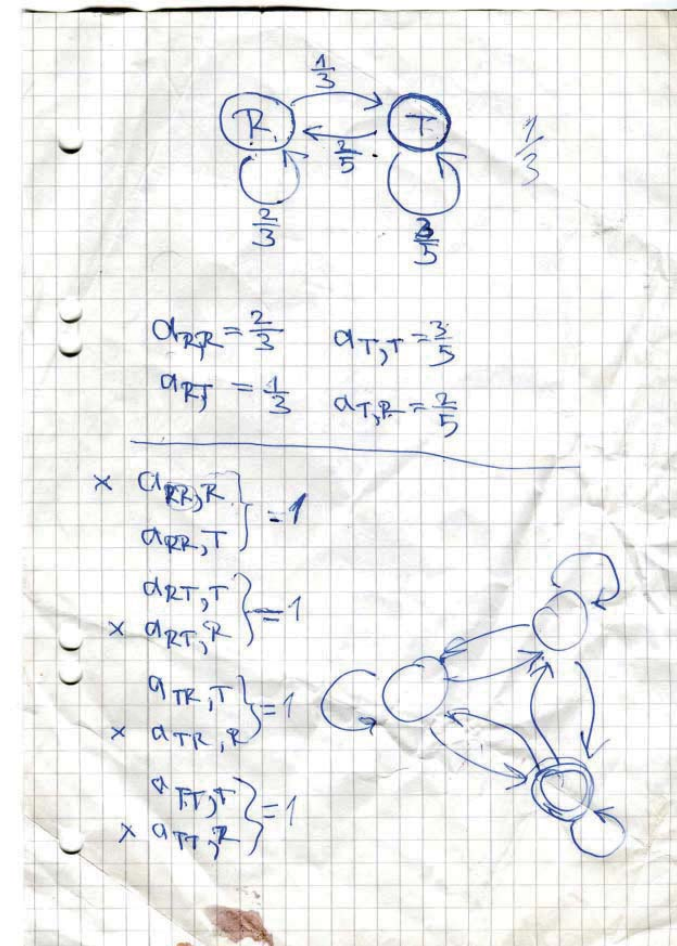


Abb. 21: Ein Ausschnitt eines Versuchs von meinem Vater Rolfdieter, mir das Prinzip der Markov-Kette zu erklären. Dass er und mein Bruder Johannes Mathematiker sind inspiriert mich auch im künstlerischen Schaffen, auch wenn mein mathematisches Verständnis im Vergleich sehr begrenzt ist. Ohne die mathematische Hilfe meines Bruders wäre der Montage-Automat in der jetzigen Form nicht entstanden, ebenso wenig aber ohne meine künstlerischen Ideen.

³⁸ <https://forum.pdpatchrepo.info/topic/11864/lua-midi-markov/4> [22.5.2020].

»Imagination is a pervasive structuring activity by means of which we achieve coherent, patterned, unified representations. It is indispensable for our ability to make sense of our experience, to find it meaningful. The conclusion ought to be, therefore, that imagination is absolutely central to human rationality, that is, to our rational capacity to find connections, to draw inferences, and to solve problems.«³⁹

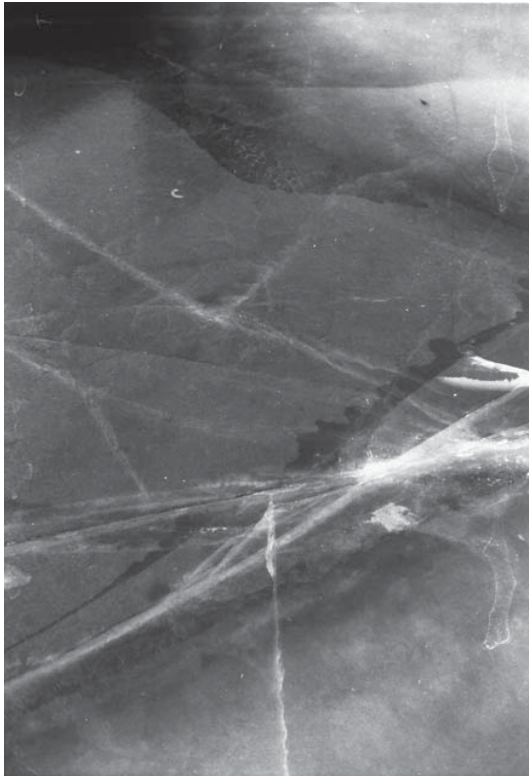


Abb. 22: Strukturen einer Eisfläche als Metapher für Störungen und Verbindungen.

³⁹ Mark Johnson: *The Body in the Mind - The Bodily Basis of Meaning, Imagination and Reason*. Chicago: The University of Chicago Press, 1987. S. 168.

Der Montage-Automat

Durch die Beschäftigung mit den Markov-Generatoren habe ich mich gefragt, ob sich eine ähnliche Methode auch für die Berechnung der Folgen von Bildern oder Szenen eines Films entwickeln lässt. Aus diesen Überlegungen ist der Montage-Automat entstanden. Wie die Markov-Generatoren berechnet auch der Montage-Automat den Folgezustand anhand der vorhergehenden Zustände. Ein Zustand ist im Fall des Montage-Automaten ein Untertitel. Da es unwahrscheinlich ist, dass ein gleicher Untertitel mehrmals innerhalb eines Films vorhanden ist, reicht das Markov-Verfahren alleine nicht aus, um Folgen zu berechnen, sondern es muss zum Beispiel mit Ähnlichkeiten gearbeitet werden. Außerdem unterscheidet der Montage-Automat, im Gegensatz zu den Markov-Generatoren, zwischen gelerntem und anzuordnendem Material, die Untertitel der generierten Folge sind in der Regel nicht im gelernten Material enthalten.

Für das Lernen der filmischen Struktur über die Untertitel habe ich mich entschieden, weil sie für die Struktur des Films durchaus ein Gewicht haben. Aber auch, weil Untertitel in großen Mengen bereits vorhanden sind und von dem Montage-Automaten nur noch eingelesen werden müssen. Zusätzlich oder stattdessen, könnte man den Montage-Automaten die Struktur des Materials auch nach anderen Kategorien, wie zum Beispiel der Handlung im Bild, der Schnittfrequenz oder der Örtlichkeit lernen lassen.

Natürlich kann über die Struktur der Untertitel nicht die Struktur der Erzählung des Films gelernt werden, sondern es werden neue Zusammenhänge, assoziative Wort-Verkettungen geschaffen.

Auf den folgenden Seiten stelle ich zwei aufeinander aufbauende Algorithmen des Montage-Automaten vor.

Montage-Automat – Algorithmus 1

Die erste Version des Montage-Automaten sucht den zum zuletzt angeordneten Untertitel ähnlichsten Untertitel im gelernten Material, springt dann von diesem zum folgenden Untertitel im gelernten Material und sucht dazu wieder den ähnlichsten Untertitel im anzuordnenden Material, um diesen als Nachfolger in die generierte Sequenz einzusetzen (Abb. 23).

Das Ähnlichkeitsmaß eines Untertitels zu einem anderen wird berechnet, indem die Summe der identischen Wörter beider Untertitel durch die Summe der Wörter des Untertitels geteilt wird. Dabei werden Wörter die mehrfach innerhalb des Untertitels auftauchen nur einfach gewertet.

Der Programmablauf:

1. Ausgehend von einem der anzuordnenden Untertitel wird der ihm ähnlichste Untertitel des Lernmaterials gewählt.
2. Ausgehend von dem gewählten Untertitel des Lernmaterials wird der im Lernmaterial folgende Untertitel gewählt.
3. Ausgehend von dem folgenden Untertitel des Lernmaterials wird der ähnlichste Untertitel des zu ordnenden Filmmaterials gewählt.

Damit ist der folgende Untertitel angeordnet und der Programmablauf startet von Neuem.

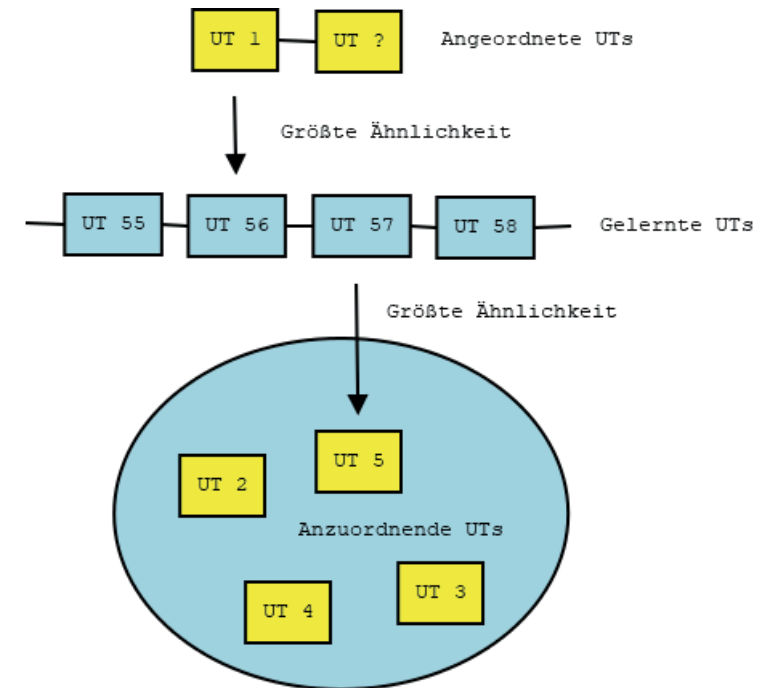


Abb. 23: Schematische Darstellung der Funktionsweise der ersten Version des Montage-Automaten (Algorithmus 1).

Montage-Automat – Algorithmus 2

Die erste Version des Montage-Automaten habe ich, wie die meisten anderen Programme, mit »Pure Data« programmiert. Die zweite, aktuelle Version, habe ich, mit der Hilfe meines Bruders und Ideen von Ingo Stock, mit der Programmiersprache »Python« programmiert. Die Folgen werden mit dem Algorithmus 2 nicht über Untertitelähnlichkeiten berechnet, sondern die Sequenz ergibt sich aus dem gewichteten Häufigkeitswert von Folgewörtern innerhalb des Lernmaterials. Ein Folgewort ist ein Wort, das auf ein bestimmtes Wort eines vorhergehenden Untertitels folgt. Je öfter ein Folgewort innerhalb des Lernmaterials vorhanden ist, umso höher ist dessen Häufigkeitswert. Ein Grund diese Methode zu wählen war die Überlegung, so die Charakteristik der Untertitelstruktur des Lernmaterials besser abbilden zu können. Außerdem ist es mit dieser Version möglich, mehrere Untertiteldateien gleichzeitig als Lernmaterial zu verwenden. Die gewichteten Häufigkeitswerte der Folgewörter unterschiedlicher Filme können zum Beispiel addiert werden.

Der Algorithmus 2 besteht aus zwei Matrizen, der Signifikanz-Matrix und der Folge-Matrix (Abb. 24 – 25). Aus ihnen berechnet sich die Reihenfolge der anzuordnenden Untertitel. Es wird der Untertitel mit dem höchsten Wert in der Folge-Matrix als Folgeuntertitel gewählt. Bereits angeordnete Untertitel dürfen nicht nochmals verwendet werden. Gibt es keinen Untertitel mit einem Folge-Matrix Wert größer 0 wird ein zufälliger Untertitel als Folgeuntertitel gewählt. Wird der in der Originalfolge letzte Untertitel gewählt, endet die Sequenz.

Die Signifikanz-Matrix:

Je öfter ein Wortpaar in aufeinanderfolgenden Untertiteln des Lernmaterials vorhanden ist, umso höher der entsprechende Eintrag in der Signifikanz-Matrix. Der Wert lässt sich unterschiedlich gewichten. So können z. B. Füllwörter (»stop words«) wie »und«, »ich« usw. vernachlässigt, oder andere Wörter hervorgehoben werden.

Die Folge-Matrix:

Für jede Kombination aus zwei Untertiteln des anzuordnenden Materials werden die Gewichtungen der Folgewörter aus der Signifikanz-Matrix in der Folge-Matrix abgebildet und aufsummiert.

Die Generierung der Reihenfolge:

Als Nachfolger in der neuen Sequenz wählt der Automat den Untertitel, der in der Folge-Matrix, bezogen auf seinen Vorgänger, den höchsten Wert hat.

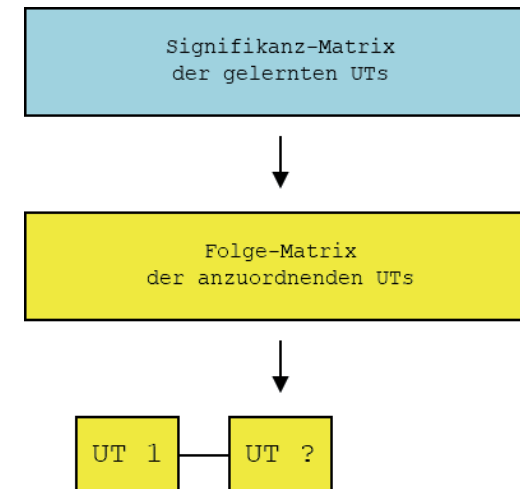


Abb. 24: Schematische Darstellung der Funktionsweise der zweiten Version des Montage-Automaten (Algorithmus 2).

Mit den Darstellungen auf der folgenden Seite (Abb. 25) veranschauliche ich, an einem Beispiel mit vier gegebenen Untertiteln ICH BIN HIER - DU BIST HIER - ICH WAR HIER - DU WARST HIER im Lernmaterial und vier gegebenen Untertiteln ICH SCHREIBE HIER - HIER IST HIER - DAS WAR SO - DU UND ICH UND WIR im anzuordnenden Material, wie die Signifikanz-Matrix und die Folge-Matrix berechnet werden.

Die Signifikanz-Matrix beinhaltet die gewichteten Folgehäufigkeiten der Wörter des Lernmaterials von Untertitel zu Untertitel der Originalfolge. In diesem Beispiel hat jedes Wort die Gewichtung 1. Der Wert in der Signifikanz-Matrix entspricht also der Anzahl, wie oft das Wort einer Zeile in einem Untertitel enthalten ist, der auf einen Untertitel folgt, der das Wort der Spalte enthält. Da das Wort HIER dreimal in der Untertitelfolge des Lernmaterials auf das Wort HIER folgt, bekommt diese Wortfolge demnach den Wert 3 zugeordnet.

In der Folge-Matrix werden die Werte der Signifikanz-Matrix für alle darin enthaltenen Wortkombinationen, bezogen auf den Ausgangsuntertitel, für jeden anzuordnenden Untertitel aufsummiert. Die Summe wird durch die Anzahl der Wörter des anzuordnenden Untertitels geteilt / normiert und anschließend wird der Untertitel mit dem höchsten Folge-Matrix Wert als Folgeuntertitel gewählt.

Aufgrund des Lernmaterials folgt der Untertitel HIER IST HIER mit dem höchsten Wert von $10/3$ in der Folge-Matrix auf den Untertitel ICH SCHREIBE HIER.

SIGNIFIKANZ-MATRIX

Untertitelfolge des Lernmaterials:
 ICH BIN HIER → DU BIST HIER → ICH WAR HIER → DU WARST HIER

Wörter der Untertitel →

Wörter der Folgeuntertitel →

	ICH	BIN	HIER	DU	BIST	WAR	WARST
ICH	0	0	1	1	1	0	0
BIN	0	0	0	0	0	0	0
HIER	2	1	3	1	1	1	0
DU	2	1	2	0	0	1	0
BIST	1	1	1	0	0	0	0
WAR	0	0	1	1	1	0	0
WARST	1	0	1	0	0	1	0

FOLGE-MATRIX

Ausgangsuntertitel
 ↓ ICH
 SCHREIBE
 HIER

Anzuordnende Untertitel

	HIER IST HIER	DAS WAR SO	DU UND ICH UND WIR
ICH	2 0 2	0 0 0	2 0 0 0 0
SCHREIBE	0 0 0	0 0 0	0 0 0 0 0
HIER	3 0 3	0 1 0	2 1 0 0 0

$\frac{10}{3}$ $\frac{1}{3}$ $\frac{5}{5}$

Abb. 25: Berechnung der Signifikanz-Matrix und der Folge-Matrix.

Der Montage-Automat – eine Sequenz

Exemplarisch zeige ich auf den folgenden Seiten die ersten 28 Untertitel einer mit dem Montage-Automaten generierten Sequenz. Ich habe den Film »Alphaville« von Jean-Luc Godard als Beispiel gewählt, weil sich der Film mit künstlicher Intelligenz beschäftigt, und dadurch mit seiner Thematik eine Nähe zum Montage-Automaten aufweist. Die Stadt Alphaville wird von dem Computersystem Alpha 60 verwaltet, während die Poesie und die Gefühle verpönt sind.

»Lemmy Caution, ein Privatdetektiv, kommt in die futuristische Stadt Alphaville, um nach dem vermissten Agenten Henry Dickson zu suchen. Die Stadt steht unter der Kontrolle von Professor von Braun und wird von einem Computersystem namens Alpha 60 verwaltet. Liebe, Dichtung und Gefühle sind verfehlt. Die Ächtung führt zu einer unmenschlichen und entfremdeten Gesellschaft. Caution lässt sich auf seiner Suche von Natascha, der Tochter des Professors von Braun, helfen.«⁴⁰

Da es mit dem Montage-Automaten in der jetzigen Form nicht möglich ist, die Struktur von Filmen erkennbar auf anderes Material abzubilden, habe ich einen experimentellen Ansatz gewählt und die Untertitel auf sich selber angewandt. Das Lernmaterial und das anzuordnende Material sind identisch. Dabei spielt die Vermutung, dass sich das angeordnete Material in der Reihenfolge dem Lernmaterial ähneln müsste, wenn der Algorithmus gut funktioniert, eine Rolle. Sozusagen als Test. Außerdem habe ich versucht, durch die manuelle Gewichtung einzelner Wörter assoziative Untertitelketten zu erzeugen.

Die Idee der unterschiedlichen Wortgewichtung wird mit dem Begriff der Stoppwörter auch in Suchmaschinen, die ebenso wie der Montage-Automat mit Textähnlichkeiten arbeiten, eingesetzt:

⁴⁰ https://de.wikipedia.org/wiki/Lemmy_Caution_gegen_Alpha_60 [22.5.2020].

»Als »Stoppwort« werden Wörter bezeichnet, die bei der Analyse eines Textes durch den Algorithmus einer Suchmaschine keine bzw. eher eine vernachlässigende Bedeutung haben. Beispiele für möglicherweise von Suchmaschinen als Stoppwörter deklarierte Wörter sind z.B. der, die, das, ein, eine, einer oder auch ihr, ihre, ihrer.«⁴¹

Mit dem Montage-Automaten lassen sich unterschiedliche Gewichtungen für sehr wichtige, wichtige, unwichtige und sonstige Wörter angeben. Dabei entsprechen die unwichtigen Wörter der Idee der Stoppwörter, die wichtigen und sehr wichtigen Wörter können kreativ genutzt werden. Es gibt offizielle Listen für Stoppwörter in unterschiedlichen Sprachen. Für dieses Beispiel habe ich stattdessen die häufigsten für die Verkettung redundanten Wörter gewählt. Was tatsächlich redundant ist, lässt sich schwer verallgemeinern, und muss von Fall zu Fall entschieden werden. Zur einfacheren Gewichtung der Wörter werden die häufigsten Wörter und Ihre Anzahl in Form einer Liste vom Montage-Automaten angezeigt. Ich habe versucht, die sehr wichtigen und die wichtigen Wörter so zu wählen, dass der Inhalt der generierten Untertitelfolge möglichst einen Bezug zum Thema der Erkenntnis aufweist.

Zwischen den Screenshots der generierten Sequenz (Abb. 27 – 54) habe ich mit Zitaten gearbeitet, die einen Bezug zu dem Thema meiner Masterarbeit haben. Um künstlerisch in die generierte Sequenz einzugreifen und sie weiterzudenken.

⁴¹ <https://www.seo-united.de/glossar/stoppwort> [22.5.2020].

Die hundert häufigsten Wörter des Films »Alphaville« und Ihre Anzahl. Eine Liste der häufigsten Wörter der Untertitel des Lernmaterials wird vom Montage-Automaten ausgegeben und ist hilfreich für die Wahl der Wortgewichtung:

the, 225	this, 31	»ill«, 19	has, 12
you, 179	my, 29	will, 19	y, 12
of, 121	was, 27	he, 19	es, 12
i, 115	with, 27	because, 19	light, 12
to, 104	be, 26	alpha, 18	come, 12
a, 100	or, 26	when, 17	say, 12
in, 79	sir, 25	see, 17	those, 12
and, 71	here, 25	as, 17	word, 12
is, 68	do, 24	by, 16	words, 12
it, 64	like, 24	60, 16	us, 12
what, 53	they, 24	»youre«, 15	afraid, 12
are, 52	mr, 23	yes, 15	»whats«, 11
that, 49	outlands, 23	who, 15	about, 11
know, 46	have, 22	an, 15	man, 11
me, 44	so, 22	them, 15	were, 11
no, 41	from, 22	which, 14	time, 10
for, 39	where, 22	can, 14	get, 10
your, 39	very, 21	never, 14	thank, 10
one, 38	why, 21	now, 13	going, 10
we, 37	love, 21	well, 13	miss, 10
»its«, 36	must, 20	go, 13	there, 10
»dont«, 35	all, 19	vonbraun, 13	understand, 10
»im«, 34	alphaville, 19	him, 13	night, 10
not, 34	johnson, 19	too, 12	professor, 10
but, 32	if, 19	at, 12	control, 9

Wortgewichtung der generierten Untertitelfolge:

Sehr wichtige Wörter: **know, understand, secret, conscience, think, memory**

Gewichtung der sehr wichtigen Wörter: 20

Wichtige Wörter: **why, because, word, words, number**

Gewichtung der wichtigen Wörter: 5

Unwichtige Wörter: the, you, of, i, to, a, in, and, is, it, what, are, that, me, no, for, your, one, we, »its«, »don't«, not, but, this, my, was, with, be, or, sir, here, do, like, they, mr, outlands, have, so, from, where, very, must, all, alphaville, johnson, if, »ill«, will, he, alpha, when, see, as, by, 60, »youre«, yes, who, an, them, which, can, never, now, well, go, vonbraun, him, too, at, es, light, come, say, those, us, afraid, »whats«, about, man, were, get, thank, going, miss, there, night, professor, control, said, »ive«, only, our, his, their, men, day, on, her, natasha, death, »thats«, »theres«

Gewichtung der unwichtigen Wörter: 0,1

Gewichtung der restlichen Wörter: 1

Auf der folgenden Seite sind die ersten 28 Untertitel der generierten Untertitelfolge des Films »Alphaville« in Textform abgedruckt. Ich habe diese, von den Bildern losgelöste, Form gewählt, da auch der Montage-Automat nur über das textliche der Untertitel generiert. Dabei wird deutlich, dass der Algorithmus durchaus Strukturen abbilden kann, und zwar dadurch, dass es mehrere Untertitelfolgen gibt, die der Originalsequenz entsprechen. In dieser Folge sind es die Untertitelsequenzen **1 - 4, 600 - 601, und 707 - 709**. Natürlich entfernt sich das Ergebnis durch die erfolgte individuelle Gewichtung der Wörter zusätzlich von der Originalfolge. Weiterhin lässt sich auch beobachten, wie sich die höher gewichteten Wörter durch die Sequenz ziehen, und von dem Algorithmus bevorzugt wieder aufgegriffen werden, ohne sie dabei stur abzuarbeiten.

1 - Sometimes, reality is too complex
 2 - for oral communication
 3 - But legend embodies it in a form
 4 - which enables it to spread all over the world
 728 - You send spies to sabotage the rest of the world
 664 - You see, you do **know** the outlands
 675 - It's believed to be secrets, but really it's no-
 thing
 587 - I don't **know** what
 658 - Where? I don't **know**
 365 - I don't **know** ... 45
 888 - They're **words** I don't **know**
 63 - How do you **know**?
 600 - Yes, secrets of planning, atomic secrets, secrets
 of **memory**
 601 - Now what are you looking for? This town is driving
 me nuts!
 603 - Are you a complete idiot? The **word** I'm looking for,
 obviously ...
 599 - Don't you **know** what a **secret** is?
 887 - I don't **know** what to say
 433 - A meaningless reply. We **know** nothing
 693 - No, I **know** what that is: it's sensuality
 669 - Of course I **know** him. Don't be stupid.
 273 - You really don't **know** what it means?
 807 - I don't **know** what became of him
 808 - ...we believe that we **know** the **number** two ...
 809 - Central Palace, South Zone, behind Raw Materials
 Station
 589 - Let's go
 319 - Don't budge
 229 - I **know** it well
 432 - It's already begun

Eine Liste der 28 ersten Untertitel, der vom Montage-Automaten generier-
 ten Sequenz. Untertitelnummern, die der Originalfolge entsprechen sowie
 wichtige und sehr wichtige Wörter sind fett gedruckt.

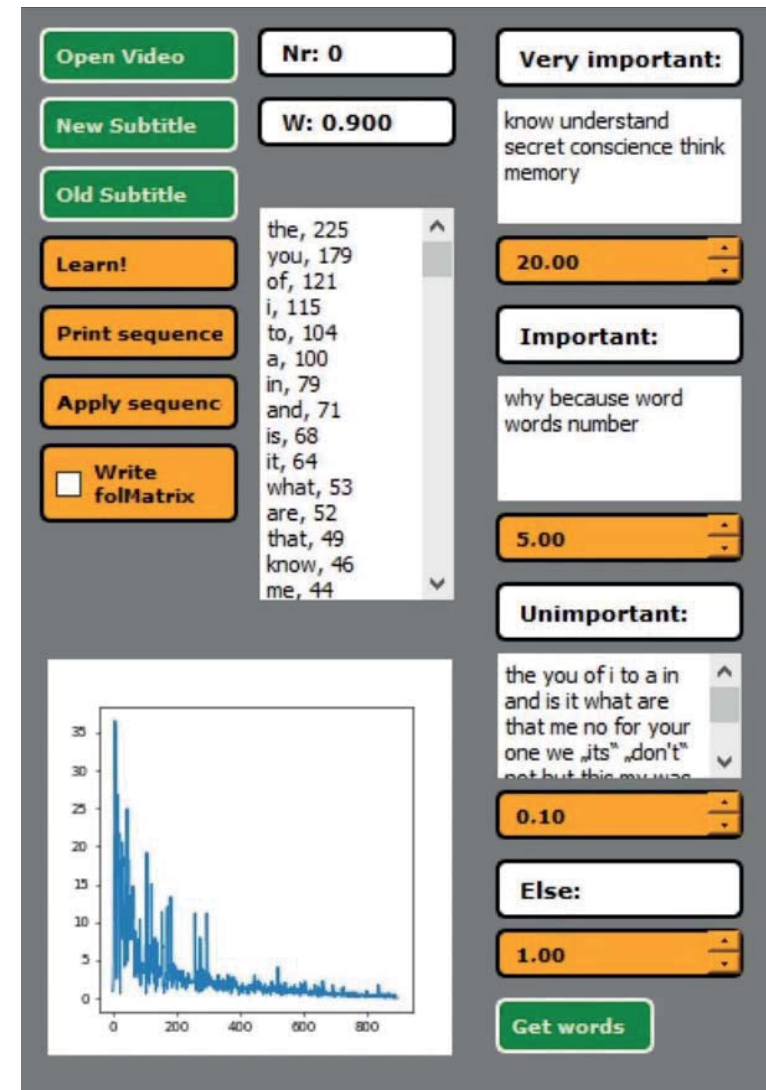
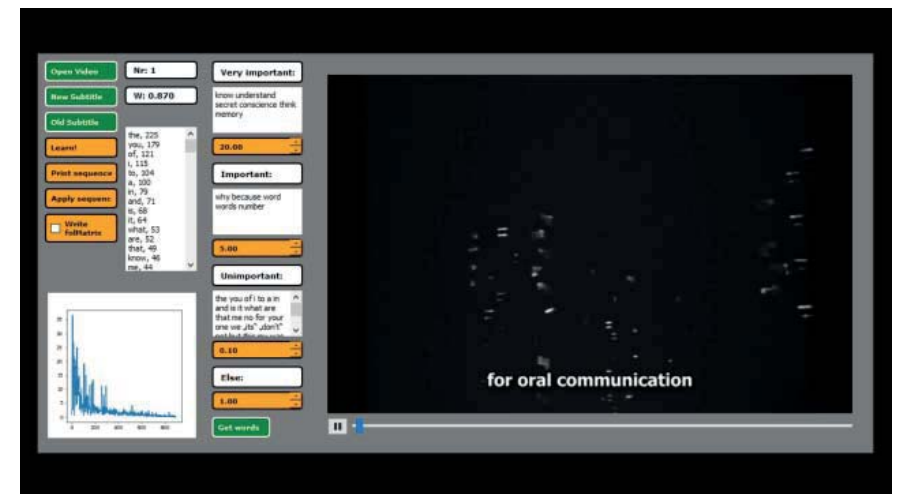
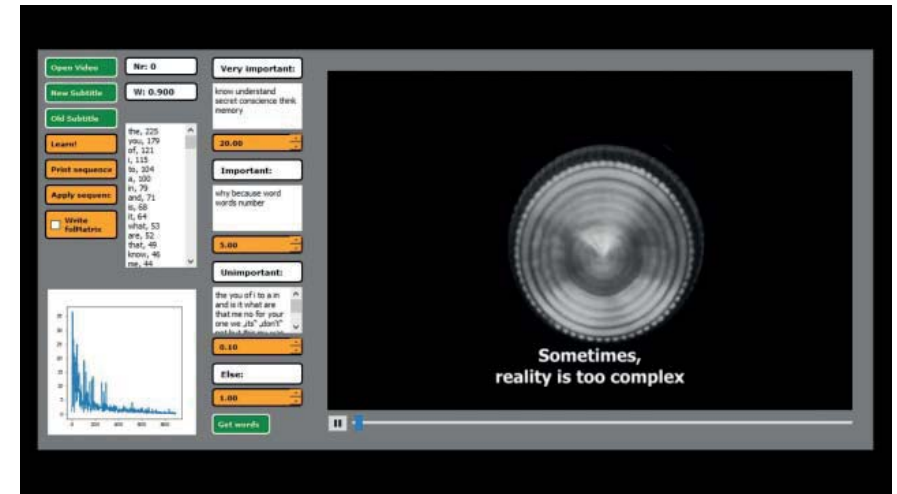


Abb. 26: Bedienoberfläche des Montage-Automaten mit den Ein-
 stellungen der vorgestellten »Alphaville« Sequenz. Das Dia-
 gramm unten links zeigt auf der x-Achse die Untertitelnummer
 der generierten Sequenz und auf der y-Achse den in der Fol-
 ge-Matrix ermittelten Wert des Untertitels.

»Das effektive Ziel des Systems generativer Ästhetik besteht darin, die Charakteristiken ästhetischer Strukturen, die in einer Menge materialer Elemente realisierbar sind, numerisch und operationell so zu beschreiben, dass sie als abstrakte Schemata eines ›Prinzips Gestaltung‹, eines ›Prinzips Verteilung‹ und eines ›Prinzips Menge‹ gelten können und manipulierbar einer materialen, ungegliederten (›verdampften‹) Menge von Elementen aufgedrückt werden können, um gemäss diesen ›Prinzipien‹ das hervorzurufen, was wir als ›Ordnungen‹ und ›Komplexität‹ makroästhetisch und als ›Redundanzen‹ und ›Information‹ mikroästhetisch am Kunstwerk wahrnehmen. Das aufdrücken ist indessen nicht als Anwendung einer Schablone zu verstehen, sondern als ein Erzeugungsprinzip. Auch ›Programme‹ in bestimmten ›Programmiersprachen‹ zur ›maschinellen‹ Realisation ›freier‹ (stochastischer, intuitiver) oder ›gebundener‹ (im voraus festgelegter, deduzierter) ästhetischer Strukturen gehören zum System generativer Ästhetik und ihren Projekten, sofern sie metrische (Abstände, Wortlängen), statistische (Wortfolgen, Positionierungen) und topologische (Verbindungen, Deformationen) Bestimmungen einkalkulieren, um ›ästhetische Information‹ zu erzeugen. Da nun ästhetische Strukturen nur insofern ›ästhetische Information‹ enthalten, als sie Innovationen aufweisen und diese natürlich stets nur eine wahrscheinliche, keine definitive Wirklichkeit darstellen, kann man sagen, dass die künstliche erzeugung von einer Norm abweichender Wahrscheinlichkeiten durch Theoreme und Programme das zentrale Motiv der generativen Ästhetik und ihrer Projekte ist.«⁴²

42 Max Bense: Projekte generativer Ästhetik, 1965. S. 2. http://dada.compart-bremen.de/docUploads/Bense_Manifest.pdf [22.5.2020].



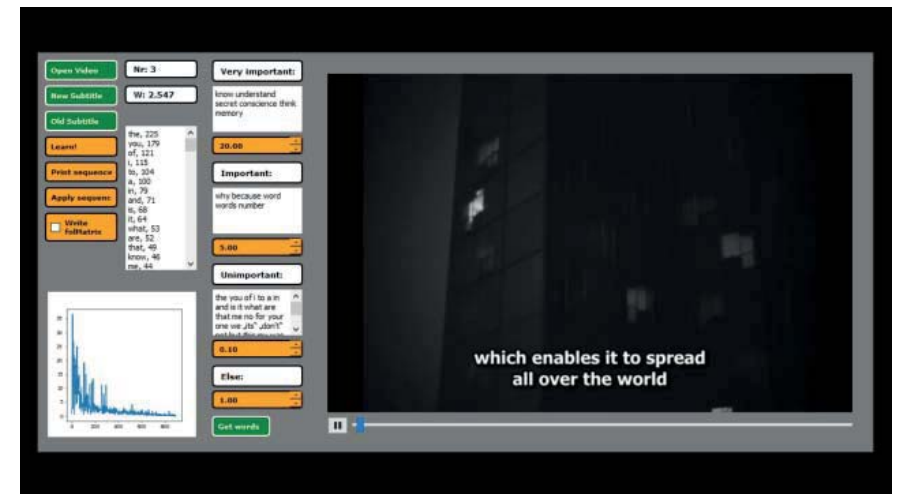
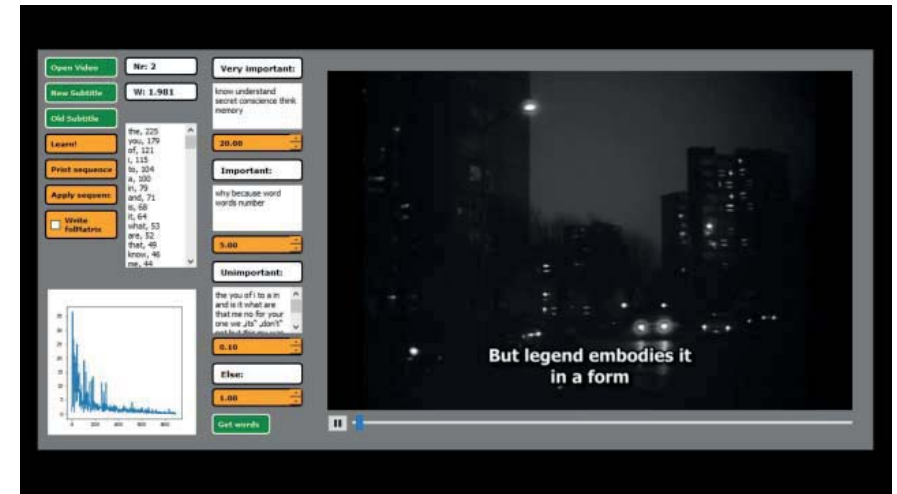
»Aber ist denn Selbstprogrammierung überhaupt noch Programmierung, wenn dieser Begriff doch normalerweise das Konditionieren von etwas anderem meint? Und was wäre dann die Identität dieses »Selbst«, das das, was es programmiert, selber ist? Und weiter: wovon wird das sich selbst programmierende Kunstwerk unterschieden, wenn nicht mehr von dem Unzugänglichen, das es symbolisiert, oder von dem Gegenstand, den es bezeichnet, indem es ihn imitiert?«⁴³

Mit der ersten Frage spricht Niklas Luhmann eine Vorstellung vom Programmieren an, die auch ich zunächst hatte. Demnach ist es das Konditionieren von etwas anderem, in meinem Fall einem Computerprogramm. So wie der Montage-Automat, der die ihm einprogrammierte Konditionierung ausführt.

Die Selbstprogrammierung dagegen findet man eher in der Kunst als in der Informatik. Allerdings ist ein Computerprogramm auch nur dann nicht selbst programmierend, wenn das Konzept schon vorher besteht, und nur noch bestmöglich umgesetzt werden muss. Gerade bei dem experimentellen Programmieren ist es aber oft so, dass auch die Resultate des Programms einen Einfluss auf das weitere Programmieren haben, und somit gewissermaßen auch ein Computerprogramm (bis zu seiner Fertigstellung) selbst programmierend sein kann. Man könnte vielleicht sagen, das alles, was kreativ geschaffen wird, und nicht einfach einem Plan folgt, sich gleichzeitig auch selbst programmiert.

Was ist aber dieses »Selbst«? In wie weit hat es mit den Menschen zu tun, die dieses »Selbst« erschaffen? Auch wenn das »Selbst« gewissermaßen selbstständig ist, ist es gleichzeitig abhängig davon, wie dessen Möglichkeiten genutzt werden. Also davon, welche Operationen in der Sache (zum Beispiel dem Computerprogramm / dem Text / der Montagesequenz / dem Kunstwerk) gesehen werden.

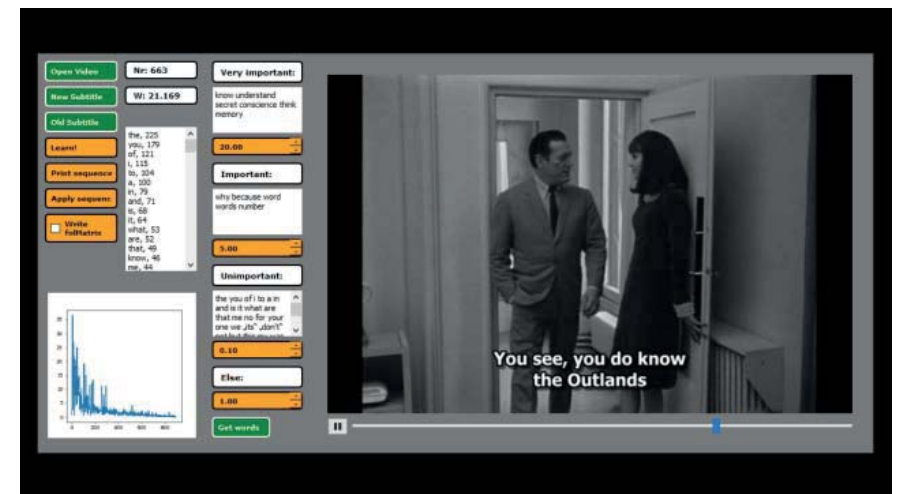
⁴³ Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 332.



»Der Begriff der Selbstprogrammierung löst die Probleme des traditionellen Freiheitsverständnisses auf, indem er Freiheit auf selbsterzeugte kognitive Vorgaben bezieht. Selbstprogrammierung soll nicht heißen, das einzelne Kunstwerk sei ein autopoietisches, sich selbst erzeugendes System. Man kann jedoch sagen: es konstituiere die Bedingungen seiner eigenen Entscheidungsmöglichkeiten. Oder: es beobachte sich selbst. Oder vielleicht genauer: es sei nur als Selbstbeobachter beobachtbar.«⁴⁴

Die Auflösung des traditionellen Freiheitsverständnisses in der Kunst löst sich nach Luhmann durch die Idee der Selbstprogrammierung auf. Das finde ich sehr spannend, da auch meine Überlegung zur Improvisation in der Filmmontage die traditionelle Vorstellung von Improvisation aufgelöst hat, und als das Reagieren auf die von dem Kunstwerk ermöglichten Operationen verstanden werden kann.

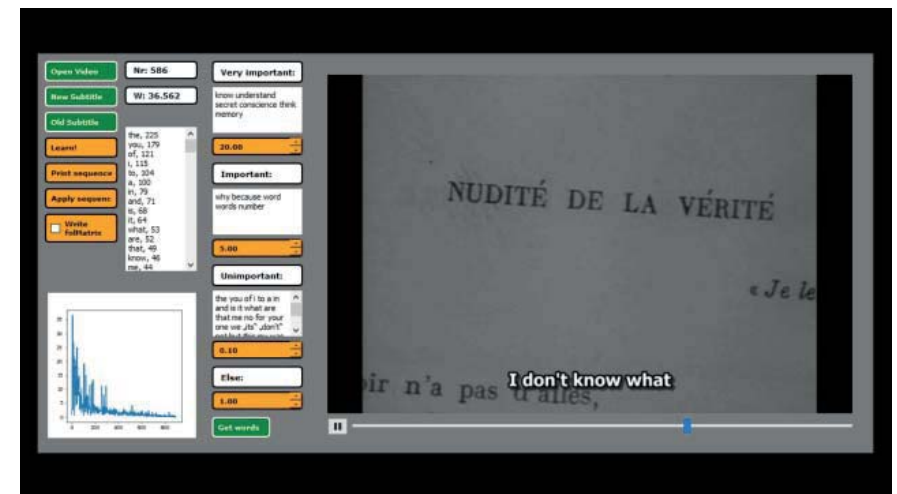
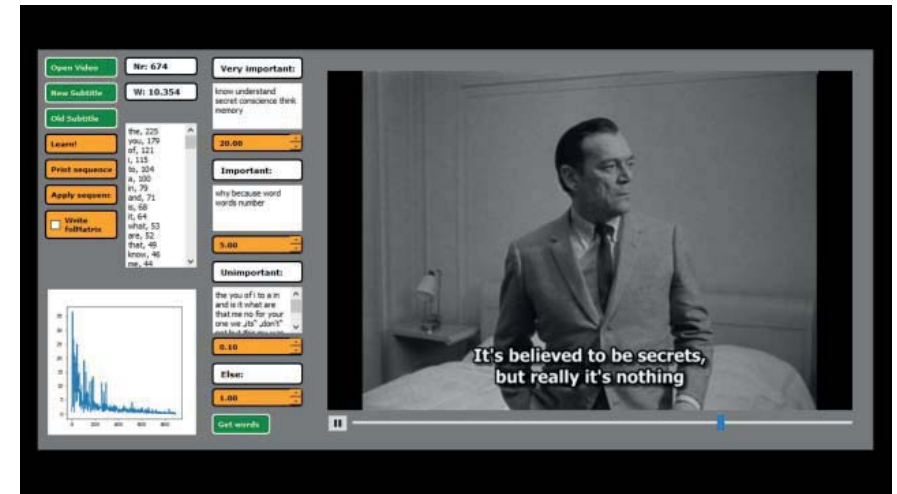
⁴⁴ Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 331.



»Ein Beobachter des Kunstgeschehens kann, während gleichzeitig geschieht, was geschieht, sehr verschiedene Unterscheidungen verwenden, um zu bezeichnen, was er beobachtet. Es liegt an ihm. Natürlich ist er gebunden, dem Objekt und den Unterscheidungen, die in es eingelassen sind, gerecht zu werden. Es wäre falsch, zu sagen, es sei aus Granit, wenn es aus Marmor ist. Aber warum die Unterscheidung Granit /Marmor? Warum nicht alt/neu oder billig/teuer oder: wohin damit? Haus oder Garten? Erst recht ist die Theorie in der Wahl ihrer Unterscheidungen frei - und deshalb begründungsbedürftig!«⁴⁵

Beobachter des Kunstgeschehens können auch die Montieren sein. Wie beurteile ich in der Montage mein Material? Wie treffe ich Entscheidungen? Die Unterscheidung ist für Luhmann ein wesentlicher Begriff, der letztlich auch zu Entscheidungen führt. Es gibt viele Möglichkeiten das Material zu lesen, dementsprechend bietet auch das sich selbst programmierende Kunstwerk (zum Beispiel die Montagesequenz) eine Vielfalt möglicher Operationen. Damit etwas trotzdem nicht beliebig wird, muss ich immer wieder reflektieren, weshalb ich mich für eine der möglichen Operationen entscheide. Der Montage-Automat dagegen, der generiert, arbeitet die vorher programmierten Begründungen einfach ab. Die Unterscheidungen sind nicht frei und daher auch (im Gegensatz zur Funktionsweise des Programms und den Überlegungen dahinter) nicht begründungsbedürftig.

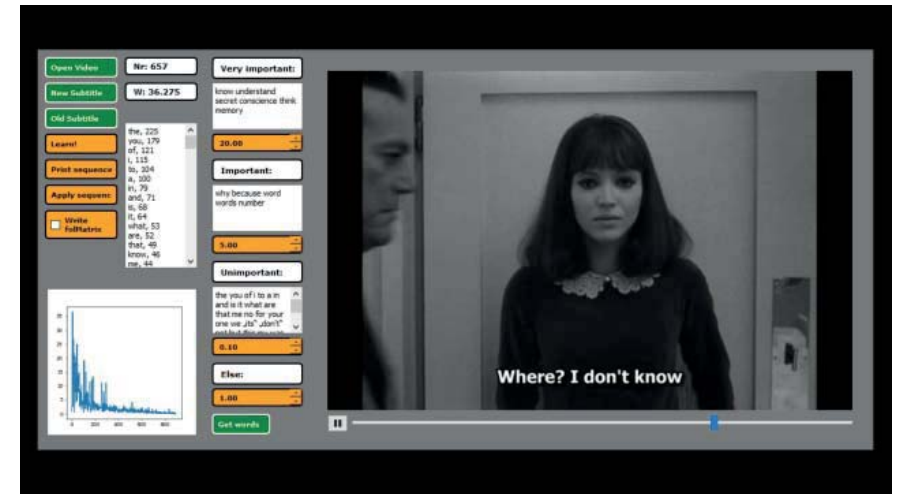
⁴⁵ Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 165.



»In mindestens einer Hinsicht vermag die Auffassung, das einzelne Kunstwerk programmiere sich selbst, nicht zu befriedigen. Es hinterläßt die Frage, ob Kunstwerke völlig zusammenhanglos zu denken seien oder ob es eine Programmierung der Programmierung geben müsse, die doch, wenn auch in veränderter Form, auf so etwas wie eine Regel-Kunst zurückführe. Vielleicht war es denn auch diese offene Frage, die es nicht zuließ, das Einzelwerk ganz in die Autonomie zu entlassen. Müßte das dann nicht heißen: Zufallsentstehung oder mindestens: Neubeginn in jedem Einzelfall?«⁴⁶

Die Programmierung der Programmierung ist, im Sinne von Niklas Luhmanns Idee des sich selbst programmierenden Kunstwerks, die Kunst- oder bezogen auf die Montage die Filmgeschichte. Man kann diesen Gedanken noch weiter führen, da sich nicht nur die Geschichte der Kunst auf das Werk auswirkt, sondern ebenso die Geschichte der Schaffenden, die wiederum von allem geprägt sind, was sie erfahren haben. In der Filmmontage könnte auch das vorhandene Rohmaterial als eine weitere Programmierung der Programmierung gesehen werden, da es das Vokabular ist, aus dem sich das Kunstwerk und die Montierenden bedienen.

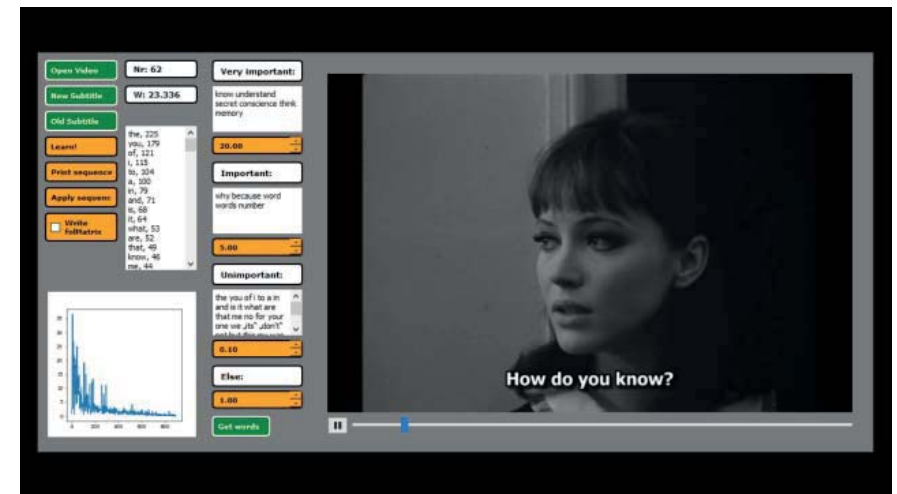
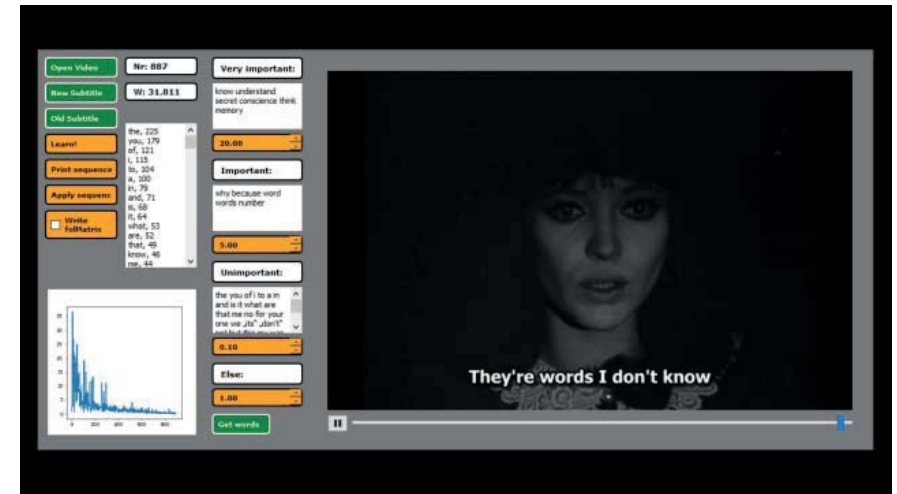
⁴⁶ Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 336.



»Die Antwort könnte deshalb darin liegen, daß jedes Kunstwerk sein eigenes Programm ist und sich, wenn genau das gezeigt werden kann, als gelungen und eben damit als neu erweist. Die Programmatik durchdringt, könnte man sagen, das Einzelwerk, und erlaubt dann kein zweites derselben Ausführung mehr. Was damit begrifflich ausgeschlossen wird, ist der Fall, auf den Arthur Danto seine Kunsttheorie konzentriert: daß völlig gleich aussehende, ästhetisch nicht unterscheidbare Objekte durch Interpretation zu verschiedenen Kunstwerken »transfiguriert« werden. (Nicht ausgeschlossen ist selbstverständlich, daß ein und dasselbe Kunstwerk verschieden interpretiert werden kann.) Man mag Serienmalerei zulassen, in der ein Bildgedanke in verschiedenen Versionen ausprobiert wird. Aber das ist dann nur eine Variante zur Grundidee der Selbstprogrammierung des Werkes – eine Variante, die mehr Komplexität zu zeigen erlaubt, als dies an einer einzigen Raumstelle möglich wäre.«⁴⁷

Mit diesem Zitat beschreibt Niklas Luhmann, wodurch sich für ihn ein gelungenes Kunstwerk auszeichnen könnte. Und zwar dadurch, dass es sein eigenes Programm ist. Sich also durch eine Originalität kennzeichnet, die es von anderen Kunstwerken unterscheidet. Das bedingt, dass sich die Kunstschaffenden Ihrer Methoden und der Kunstgeschichte bewusst sind, um Wiederholung und Redundanz zu vermeiden. Es reicht nicht aus, dass ein Kunstwerk sich isoliert von Kontexten bisheriger Kunstwerke aus seinen eigenen Operationen heraus selbst programmiert, um als gelungen zu gelten. Es kann sich nicht nur auf sich selbst beziehen, sondern steht in Bezug zur Welt.

⁴⁷ Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 228 – 329.

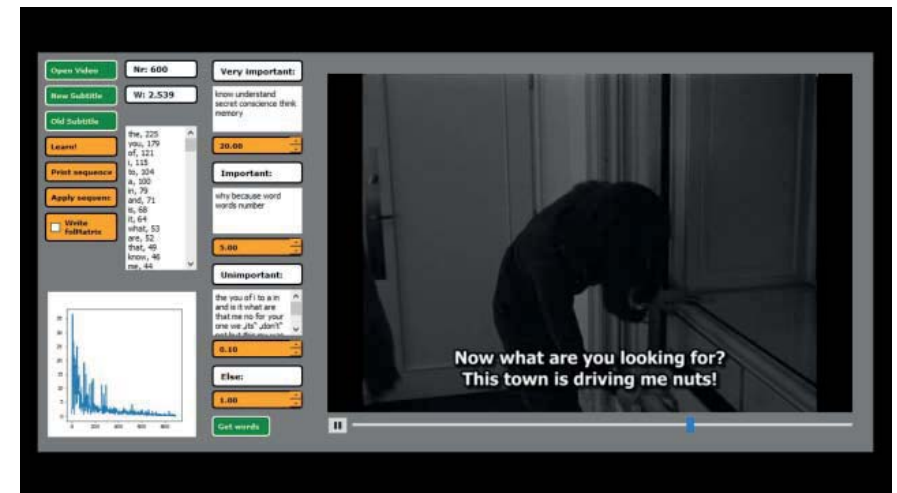
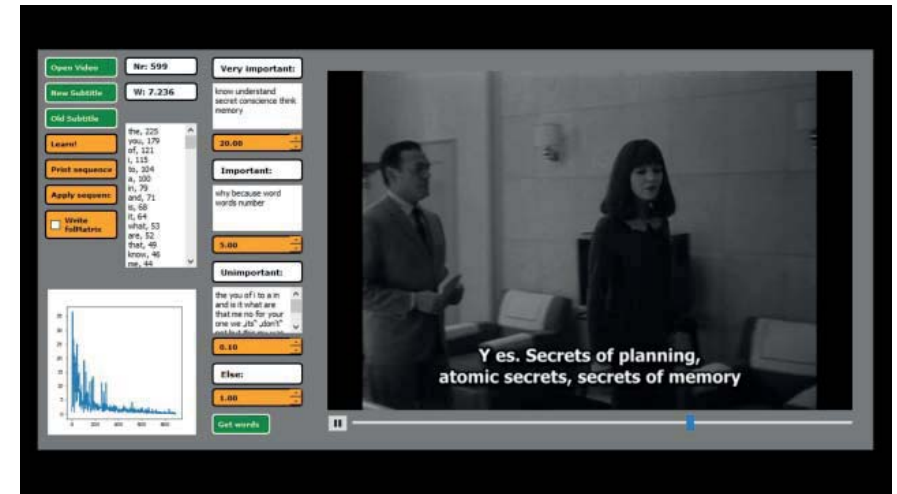


»Der britische Kunsthistoriker Martin Kemp umschreibt den Problemhorizont gemeinsamer, Kunst und Wissenschaft zu Grunde liegender Motive mit dem Begriff der *strukturellen Intuition*. Mit diesem Begriff versucht er »das uralte Bestreben von Wissenschaftlern und Künstlern zu fassen, die Strukturen beobachteter Phänomene zu erkennen und aus dem Chaos der Erscheinungen Ordnungen unterschiedlicher Komplexität zu extrahieren.«⁴⁸⁴⁹

Der Begriff der strukturellen Intuition erinnert mich an den der Improvisation. Bei beiden geht es darum, aus Beobachtungen Strukturen zu schaffen. In der Wissenschaft durch eine Beschreibung und in der Kunst durch das Kunstwerk selbst. Auf beiden Gebieten kann man sich die Frage stellen, ob man die Ordnung und die Struktur aus dem vorhandenen Material nur extrahiert, oder ob man sie schafft. Die Frage nach dem Finden oder dem Erfinden.

⁴⁸ Martin Kemp: Wissen in Bildern – Intuitionen in Kunst und Wissenschaft, in: Christa Maar (Hrsg.): Iconic Turn. Die neue Macht der Bilder. Köln: DuMont Buchverlag, 2004. S. 389.

⁴⁹ Markus Koller: Die Grenzen der Kunst – Luhmanns gelehrte Poesie. Wiesbaden: VS Verlag für Sozialwissenschaften, 2007. S. 13.



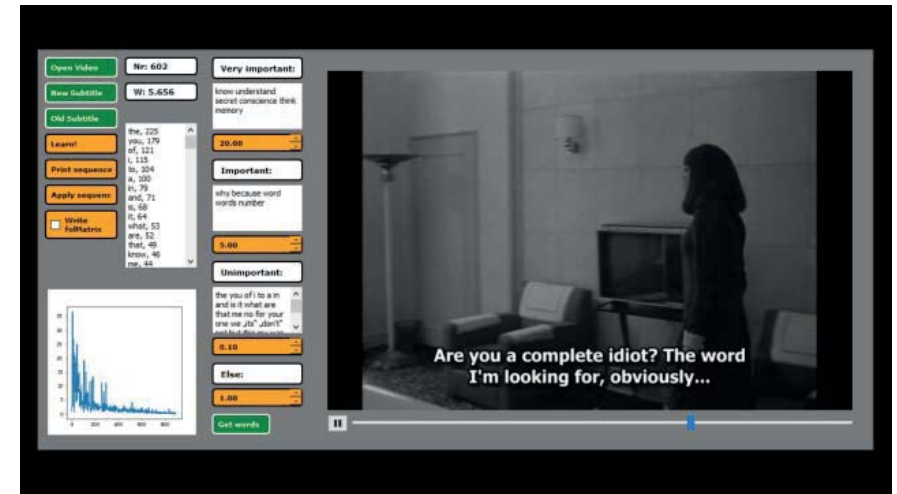
»Also, ich denke die erste Aufgabe, deswegen Experiment, ist eine Struktur vorzustellen, die man dann verbessern oder ablehnen kann, wo man sich überlegen kann wie können wir Alternativen konstruieren, die anderen Interessen besser gerecht wird, und zugleich in der Komplexität, oder was auch immer überlegen ist. Aber wenn man gar nicht erst versucht, solch ein Formenwerk zu produzieren, dann ist – eine Aufgabe, die an sich da ist, wird dann nicht bedient. Dieser Kollektivsingulär ist so verhängnisvoll, weil da einfach eine Nichtthematisierung des zentral-sozialen Problems – viele Menschen erleben und handeln gleichzeitig, und da gibt es keine Ordnung in der Gleichzeitigkeit.«⁵⁰

Niklas Luhmann über den Ansatz seiner soziologischen Systemtheorie. In seiner Arbeit spielt die Fantasie eine große Rolle, auch weil er sich unterschiedlichen Wissensbereichen bedient, und so etwas Neues schafft. Er spricht von der Konstruktion, von einem Formenwerk. Auch vom Experiment und vom Vorstellen von Strukturen.

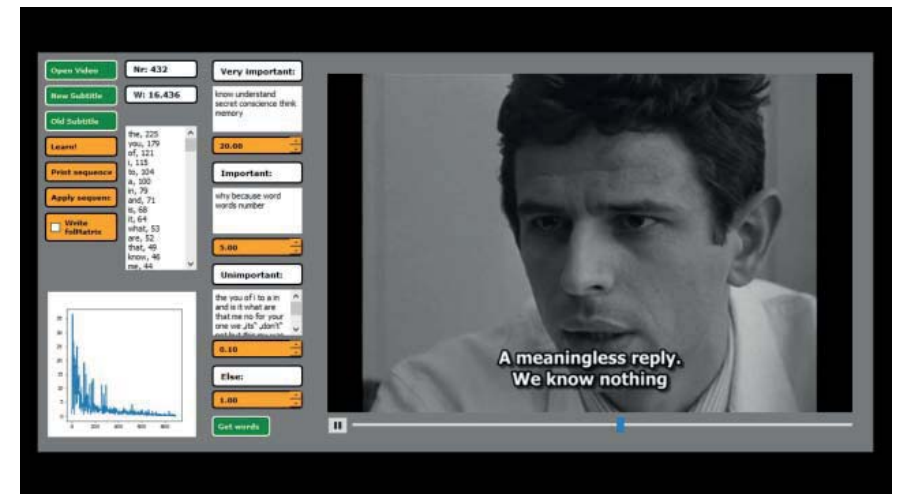
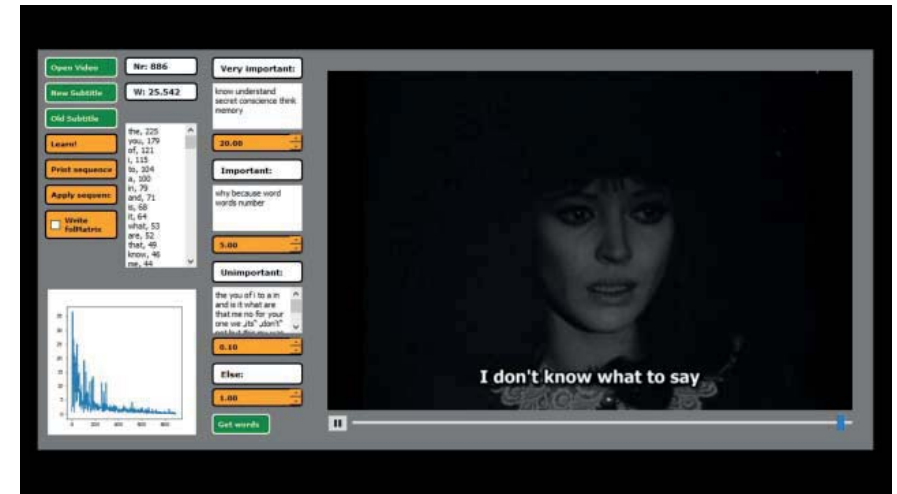
Diese Überlegungen lassen sich auch auf die Kunst und die Filmmontage übertragen. Auch dort führt erst das Suchen nach neuen Formen und Strukturen zu einer Diversität und Multiperspektivität, die den Diskurs in der Kunst und im Film am Leben erhält.

Wenn in der Kunst wie in der Wissenschaft Strukturen nur reproduziert werden, wird es irgendwann an neuen Erkenntnissen fehlen.

⁵⁰ Niklas Luhmann: Niklas Luhmann – 1997 – Es gibt keine Biographie. <https://www.youtube.com/watch?v=nFhQ6SrIKVo> (Position 53:15) [22.5.2020].



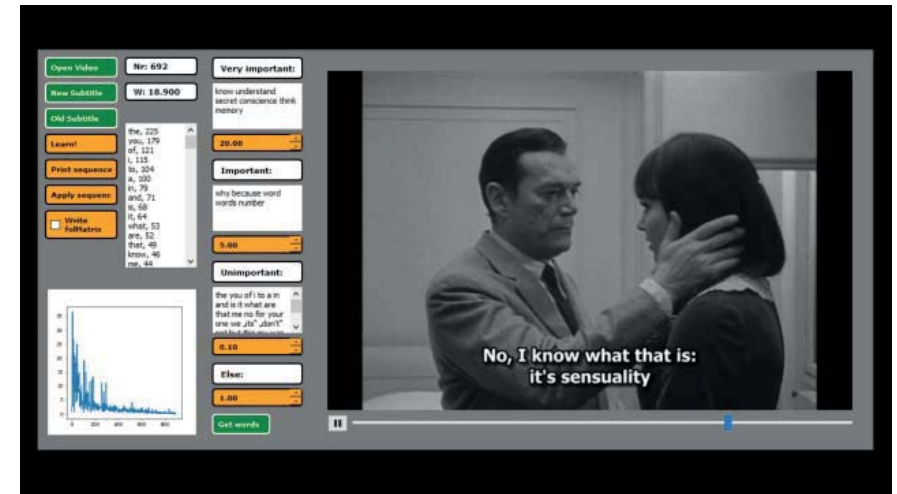
»Kann man denn in einer Galerie herumgehen und sagen: Dieses Bild ist erhaben und dieses ist nicht erhaben? Es lenkt viel zu sehr von der Selbstkompositionsleistung eines Kunstwerks ab, wenn man einen solchen Begriff in die Diskussion einbringt. Das Erhabene ist ein Abwehrbegriff für Beliebigkeit, aber wenn man Kunstwerke anschaut, sind sie alles andere als beliebig.«⁵¹



51 Niklas Luhmann / Dirk Baecker / Frederick Bunsen: Unbeobachtbare Welt – Über Kunst und Architektur. Bielefeld: Haux, 1990. S. 66.

»Nach Gottfried Fischborn lässt sich der Begriff [der Dramaturgie] auf alle prozessualen und strukturierten Tätigkeiten, kommunikativen Akte (die Sprechakte eingeschlossen), Geschehnisfolgen und Vorgänge im gesellschaftlichen wie im individuellen Leben der Menschen beziehen, in der Sphäre der symbolischen Repräsentation wie in der des Alltags, in der Realität wie in den Künsten. Unabdingbar für die Anwendbarkeit des Begriffes seien raum-zeitliche Strukturiertheit (Gestalt) und Kommunikativität – völlig unstrukturierte Abläufe oder solche ohne kommunikative Intention bzw. Wirkung haben auch keine Dramaturgie.«⁵²

Nach dieser Definition folgt das generierte Ergebnis des Montage-Automaten keiner Dramaturgie, da es zwar zeitlich strukturiert ist, ihm aber die kommunikative Intention fehlt.



⁵² <https://de.wikipedia.org/wiki/Dramaturgie> [22.5.2020].

»None of this proves that computer systems can be truly creative, free, or artistic. All it shows is that our initial intuitions to the contrary are not trustworthy, no matter how compelling they seem at first. If you're sitting there muttering: »Yes, yes, but I know they can't; they just couldn't«, then you've missed the point. Nobody knows. Like all fundamental questions in cognitive science, this one awaits the outcome of a great deal more hard research. Remember, the real issue is whether, in the appropriate abstract sense, we are computers ourselves.«⁵³

Fragen künstlicher Intelligenz und Kreativität, die auch ich nicht beantworten kann, aber sehr spannend finde. Was unterscheidet den Menschen von einem Computer? Wozu werden Computer in Zukunft in der Lage sein? Besonders durch die Forschung mit künstlichen neuronalen Netzen wird es noch spannende Erkenntnisse geben. Meine eigenen Computerprogramme sind jedenfalls weder intelligent noch kreativ, sondern führen schlicht vorher erdachte Berechnungen aus. Es sind Werkzeuge, die mir neue Wege der künstlerischen Arbeit ermöglichen, und deren Ergebnisse mich wieder inspirieren können.

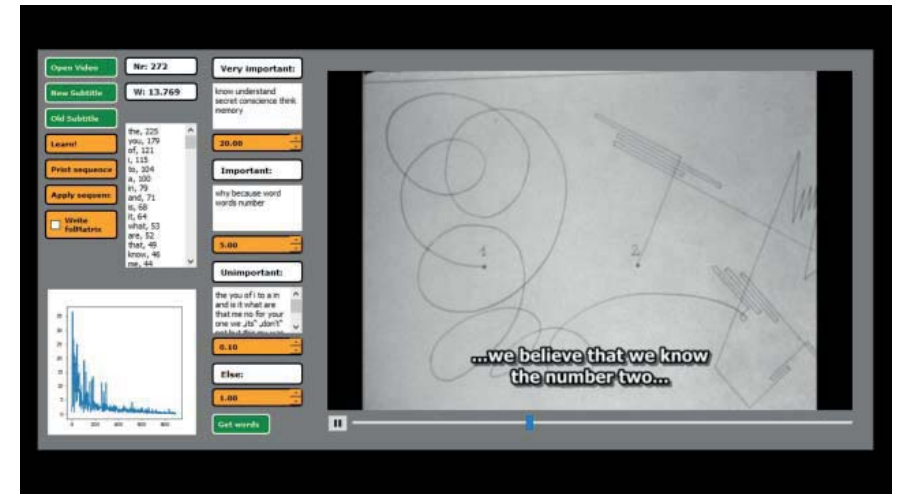
⁵³ John Haugeland: Artificial intelligence – The Very Idea. Cambridge: MIT Press, 1989. S. 12.



»Wir beginnen zunächst mit einer Rekapitulation der Analysen der Form des Kunstwerks. Denn bereits am einzelnen Kunstwerk wird sichtbar, wie Entstehensunwahrscheinlichkeit sich in Erhaltungswahrscheinlichkeit verwandelt. Die erste Unterscheidung, mit der der Künstler die Arbeit aufnimmt, kann durch das Werk noch nicht programmiert sein. Sie kann nur frei getroffen werden – sicher mit einer Typentscheidung (ob es ein Gedicht oder eine Fuge oder ein Glasfenster werden soll) und möglicherweise mit einer Idee im Kopf. Aber jede weitere Entscheidung zurrut das Werk fest, richtet sich nach dem schon Vorhandenen, greift die freie Seite der schon gesetzten Formen auf, um sie zu bestimmen und dadurch die Freiheitsgrade für Weiteres einzuschränken. In dem Maße, als die Unterscheidungen aneinander Halt gewinnen und rekursiv aufeinander Bezug nehmen, tritt also genau das ein, was man von Evolution erwartet: das Kunstwerk gewinnt Halt an sich selbst, kann zum Beispiel wiedererkannt und immer neu beobachtet werden. Destruktion bleibt natürlich möglich, aber Modifikation wird schwieriger und schwieriger. Es mögen zwar ungelöste Probleme oder schwache Stellen drinbleiben, die man dann aber als unverbesserbar in Kauf nehmen muß. Evolution bringt auch hier keine perfekten Zustände hervor.«⁵⁴

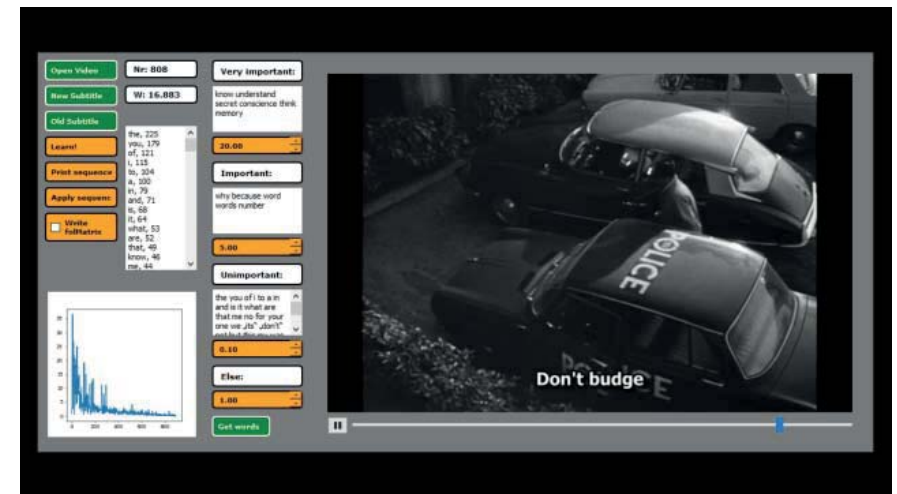
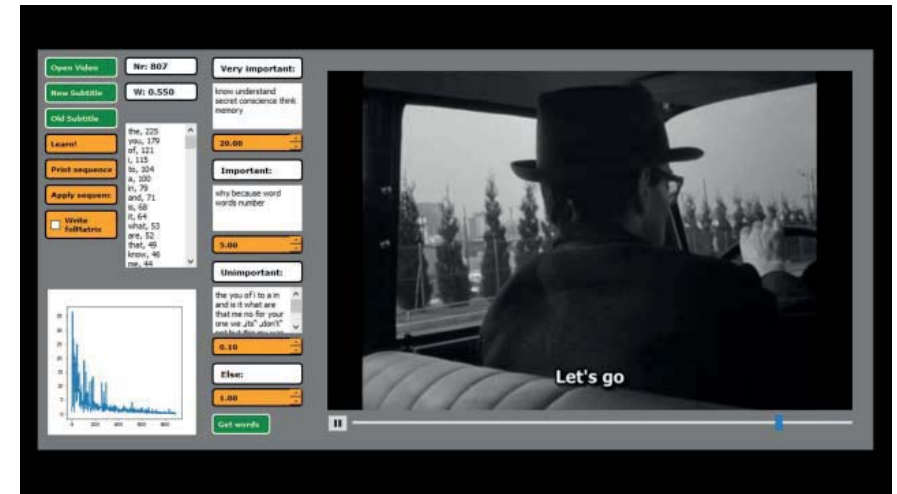
Niklas Luhmann zur Selbstprogrammierung der Kunstwerke. Eine Beschreibung, die ich auch aus der Filmmontage kenne. Wobei die Idee des Films und das Rohmaterial in der Regel schon vor Beginn der Montage vorhanden sind. Während des Montageprozesses grenzen sich die Möglichkeiten immer weiter ein, es bildet sich eine Ordnung aus dem Material, welche die Freiheit der Handlung einschränkt. Kurz vor der Fertigstellung des Films in der Montage gilt es meistens nur noch wenige Entscheidungen zu treffen, um dem Film seine endgültige Form zu geben.

⁵⁴ Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 347.



»Eine solche Produktion kann auch, mehr oder weniger, nach Plan verlaufen. Dann wird, wie auch in der Politik oder der Wirtschaft, der Plan ein Moment in der Evolution. Hält der Künstler starr an einem vorgefaßten Programm fest, wird er entweder Werke produzieren, zwischen denen es keine Qualitätsunterschiede gibt (auch wenn es verschiedene Programme sind) oder er wird zwischen Annahme und Ablehnung des Werkes zu entscheiden haben. Der typische Fall ist dagegen der, in dem der Künstler sich durch das entstehende Werk irritieren und informieren läßt, was auch immer an Planung mitläuft. Der typische Fall ist der der Evolution.«⁵⁵

⁵⁵ Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 348.



»Eine Institutionalisierung von Kunst und die Einrichtung von Informationsbeihilfen (Ausstellungen etc.) erfordern außerdem, daß Kunstwerke untereinander »Diskurse« führen, daß Kunst Kunst zitiert, copiert, ablehnt, innoviert, ironisiert-jedenfalls, wie auch immer, in einem über das Einzelwerk hinausgreifenden Referierzusammenhang reproduziert wird. Man nennt das heute »Intertextualität«. Das heißt in anderen Worten: das Kunstsystem müsse über Gedächtnis verfügen. Das ist auch und in besonderem Maße dann vorausgesetzt, wenn die Evolution der Kunstkommunikation dazu führt, daß das Einzelkunstwerk sich selbst das Gesetz gibt. Wir hatten das Selbstprogrammierung der Kunstwerke genannt. Gerade dann ist eine Spezifikation solcher Verweisungszusammenhänge erforderlich, die die Erkennbarkeit von Kunst als Kunst trotz der mehr und mehr zugelassenen Eigenwilligkeit der Kunstwerke immer noch sicherstellen.«⁵⁶

⁵⁶ Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 395.

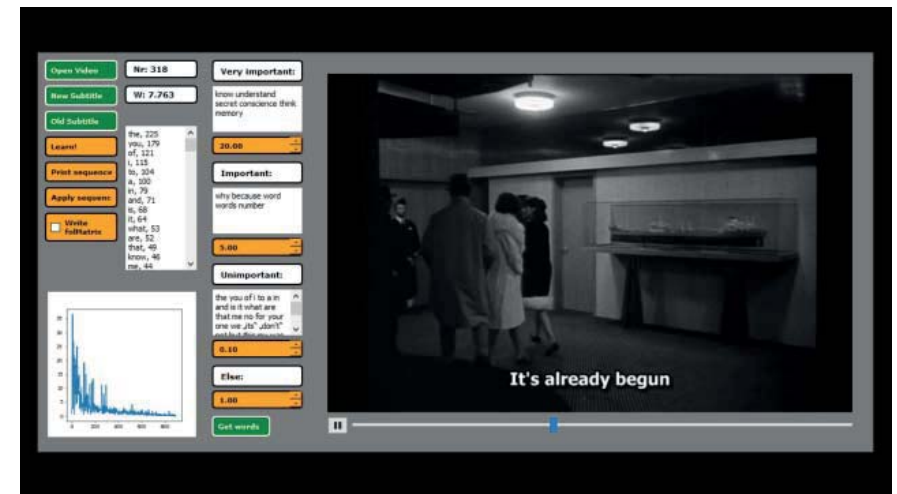
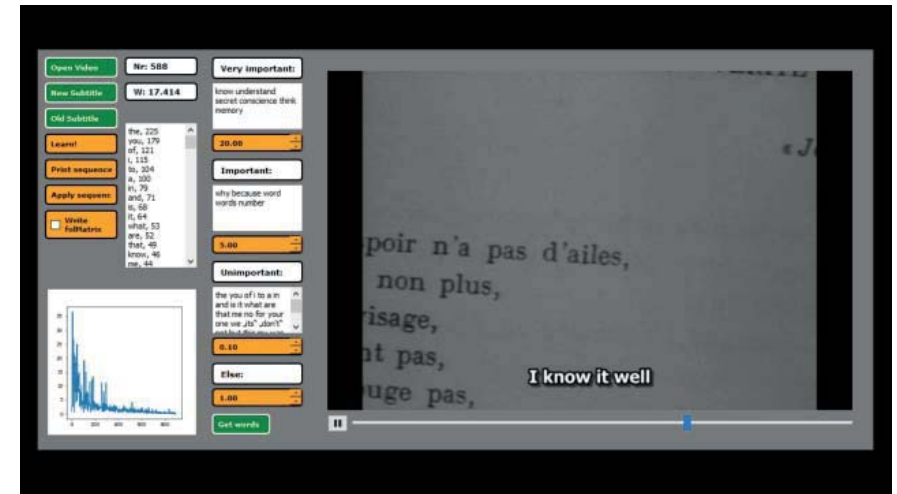


Abb. 27 – 54: Die Bedienoberfläche des Montage-Automaten und die ersten 28 über die Untertitel angeordneten Filmeinheiten des Films »Alphaville« von Jean-Luc Godard.

Der Montage-Automat und meine Erkenntnisse

Lässt sich ein Muster lernen und aus dem gelernten Material etwas Neues generieren? Diese Frage kann ich für mich mit Ja beantworten. Und ich kann die Ergebnisse des Montage-Automaten mit großer Freude beobachten. Ich glaube allerdings nicht und das habe ich auch von Anfang an bezweifelt, dass sich so Geschichten generieren lassen, beziehungsweise die Charakteristik des Lernmaterials wiederzuerkennen ist (auch wenn sie mathematisch korrekt abgebildet wird). Es lassen sich aber, durch die Gewichtung der Wörter, interessante assoziative Verkettungen generieren. Und ich kann mir vorstellen, mit den erzeugten Ergebnissen weiterzuarbeiten. Sie als Material zu sehen, in das ich künstlerisch eingreifen kann.

Dass keine Geschichte und kein Muster erkennbar ist, liegt natürlich auch an der nicht berechneten Filminformation. Alles, was nicht im Untertitel abgebildet wird, wird bei der Mustererkennung des Montage-Automaten weggelassen. Hier könnten, zusammen mit den Untertiteln, weitere Parameter einfließen. Aber ob sich daraus dann eine Geschichte ergibt? Man könnte jedenfalls schon in die aktuelle Version des Montage-Automaten weitere Daten zur Berechnung der Folgen mit einbeziehen.

Visuelle Daten, die sich beispielsweise für die Berechnung der Folgen des Montage-Automaten nutzen lassen: Anzahl der Protagonisten, Geschlechter der Protagonisten, Alter der Protagonisten, Namen der Protagonisten, Orte, Wetter, Bildästhetik, Farbigkeit, Helligkeit, Kontrast, Objekte im Bild, Bildkomposition, Handlung.

Auditive Daten, die sich beispielsweise für die Berechnung der Folgen des Montage-Automaten nutzen lassen: Musik, Klang der Sprache, Lautstärke, Tonart des Gesprochenen, Geräusche, Stille.

Allerdings müssten alle diese Daten, im Gegensatz zu den Untertiteln und mit den mir zur Verfügung stehenden Möglichkeiten, händisch erzeugt werden.

Vielleicht wäre eine subjektive Beschreibung von kurzen Montageabfolgen so, wie es auch oft in der klassischen Montage gemacht wird, die beste Methode, um die Struktur eines Films zu erfassen? Dadurch würde sich allerdings für jede Person durch die Beschreibung eine andere Struktur ergeben, weil es keine allgemeingültigen Regeln der Kategorisierung mehr gibt. Aber selbst dann frage ich mich, wie und ob ein Computerprogramm den Kontext der Beschreibung erfassen und eine eigenständige Geschichte generieren könnte. Und ob es nicht doch wieder nur mit Häufigkeiten und Ähnlichkeiten arbeiten würde. Und damit zwar ein mathematisch korrektes, aber gleichzeitig sinnloses Ergebnis generieren würde.

Auch die Methode mit den Untertiteln lässt sich sicherlich optimieren. Man könnte zum Beispiel zunächst mit Sätzen eines Romans arbeiten anstatt mit Untertiteln. Das hätte den Vorteil das ein Text alle Information der Geschichte enthält und nicht nur den Bruchteil des Gesprochenen, wie es bei den Untertiteln der Fall ist. Allerdings wäre das dann keine algorithmische Filmmontage mehr, sondern algorithmische Textkomposition. Weiterhin könnte man taxonomische und grammatikalische Klassen bilden, so das zum Beispiel den Wörtern Baum, Pflanze und Ahorn eine gemeinsame Ähnlichkeit zugeordnet wird.

Natürlich sind meine Fähigkeiten in der Computerprogrammierung begrenzt und deswegen nicht alle meiner Ideen (bisher) für mich umsetzbar. Aber auch hoch entwickelte Forschung aus dem Bereich der künstlichen Intelligenz konnte bisher keine mich überzeugenden Geschichten generieren. Die Ergebnisse wirken oft beliebig oder unfreiwillig komisch, auch wenn immer wieder interessante Zufälle entstehen. Was in der Zukunft noch möglich ist, kann ich nicht sagen. Besonders auch mit künstlichen neuronalen Netzen wird man sich sicherlich weiterhin daran versuchen. Die Idee, Strukturen von Geschichten zu lernen, um daraus neue zu generieren, fasziniert mich nach wie vor. Vielleicht lernen auch wir Menschen Strukturen von Geschichten aus denen wir mit neuem Material oder neuen Ereignissen wieder neue Geschichten generieren?

Der Montage-Automat als Quelltext

Warum gebe ich im folgenden Abschnitt den Quelltext des Montage-Automaten wieder? Einerseits finde ich den Gedanken der Reproduzierbarkeit spannend. Im Gegensatz zu Film, Bild oder Ton, die erst aus Ihrem Medium übersetzt, also codiert werden müssen, lässt sich der Code des Programms, wie jeder andere Text auch, in Textform exakt reproduzieren.

Andererseits ist es der quelloffene (Open Source) Gedanke, Software, also Programme, frei zugänglich zu machen, der mich dazu verleitet hat den Quelltext des Programms abzu drucken. Vielleicht hat jemand eine Idee dazu, vielleicht kann es jemand nutzen?

Der Quelltext des Montage-Automaten besteht aus vier Dateien, dem Hauptprogramm und drei Klassen, eine davon zum Abspielen von Videos, die von diesem benötigt werden. Natürlich hat die Veröffentlichung eines Computerprogramms auf Papier einen eher performativen Charakter, da es in dieser Form schwer nutzbar ist. Deshalb ist es auf GitHub auch in digitaler Form zu finden: <https://github.com/Jonathhhan/Montageautomat>

»Ein Code ist eine Abbildungsvorschrift, die jedem Zeichen eines Zeichenvorrats (Urbildmenge) eindeutig ein Zeichen oder eine Zeichenfolge aus einem möglicherweise anderen Zeichenvorrat (Bildmenge) zuordnet. Beispielsweise stellt der Morsecode eine Beziehung zwischen Buchstaben und einer Abfolge kurzer und langer Tonsignale (und umgekehrt) her. In der Kommunikationswissenschaft bezeichnet ein Code im weitesten Sinne eine Sprache. Jegliche Kommunikation beruht auf dem Austausch von Informationen, die vom Sender nach einem bestimmten Code erzeugt werden und die der Empfänger gemäß demselben Code interpretiert. In der Kodierungstheorie nennt man die Elemente, aus denen ein Code besteht, »Codewörter«, die Symbole, aus denen die Codewörter bestehen, bilden ein »Alphabet«.⁵⁷

⁵⁷ <https://de.wikipedia.org/wiki/Code> [22.5.2020].

Montageautomat009.py

```
import sys, os
from threading import Timer
import random
from PyQt5 import QtGui, QtCore
from PyQt5.QtWidgets import QWidget, QPushButton, QSpinBox, QApplication, QFileDialog, QLabel, QCheckBox, QVBoxLayout, QPlainTextEdit, QDoubleSpinBox, QTreeWidget
from PyQt5.QtCore import QDir, QUrl
from PyQt5.QtGui import QIcon, QPixmap
from Subtitle_class import Subtitles, findfollowers, make_seq
from PyQt5.QtMultimedia import QMediaContent
from VideoPlayer_class import VideoPlayer
from MplCanvas_class import MyMplCanvas
from colorama import Fore, Style, init
from collections import defaultdict, Counter
init()

#if getattr(sys, 'frozen', False):
#    # we are running in a bundle
#    bundle_dir = sys._MEIPASS
#else:
#    # we are running in a normal Python environment
#    bundle_dir = os.path.dirname(os.path.abspath(__file__))

#def resource_path(relative):
#    return os.path.join(
#        os.environ.get(
#            "_MEIPASS2",
#            os.path.abspath(".")
#        ),
#        relative
#    )
#pic_path=os.path.join(bundle_dir, "test.png")
pic_path = "test.png"

style = "text-align: left;"\
        "padding: 6px;"\
```



```

        „background-color: orange;“\
        „font: bold 10px;“\
        „font-family: Verdana;“\
        „color: black;“\
        „border-style: outset; border-width: 2px; border-color: black;“\
        „border-radius: 5px;“

class Interface(QWidget):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        print(„Montageautomat 2018“)

        self.very_important_value = 0
        self.important_value = 0
        self.unimportant_value = 0
        self.else_value = 0

        # Fenster Eigenschaften
        self.setWindowTitle(„Montageautomat 2018“)
        self.setFixedSize(1150, 550)
        self.setWindowIcon(QIcon(pic_path))
        self.bgpic = QLabel(self)
        self.bgpic.setFixedSize(1150, 550)
        self.bgpic.setStyleSheet(„background-color:
grey“)
        self.bgpic2 = QLabel(self)
        self.bgpic2.setFixedSize(720, 435)
        self.bgpic2.move(400, 33)
        self.bgpic2.setStyleSheet(„background-color:
black“)
        #self.bgpic.setPixmap(QPixmap(pic_path))

        # Video Player
        self.player = VideoPlayer(self)
        self.player.resize(740, 533)
        self.player.move(390, 20)

```

```

        self.player.mediaPlayer.setMedia(QMediaContent(-
        QUrl.fromLocalFile(„Alphaville.avi“)))
        self.player.playButton.setEnabled(True)

        # Matplotlib Canvas
        self.plot = QWidget(self)
        l = QVBoxLayout(self.plot)
        sc = MyMplCanvas(self.plot, width=4.1, height=4,
        dpi=50)
        l.addWidget(sc)
        self.plot.move(5, 320)

        # Button: Open Video
        self.openVideo = QPushButton(„Open Video“, self)
        self.openVideo.setStyleSheet(style+„backg-
round-color: green;color: white; border-color: white;“)
        self.openVideo.setFixedWidth(100)
        self.openVideo.clicked.connect(self.player.open-
        File)
        self.openVideo.move(10, 10)

        # Label to show current subtitle
        self.label1 = QLabel(self)
        self.label1.setStyleSheet(style+„background-co-
lor: white;font-size: 8pt;“)
        self.label1.setText(„Nr: 0“)
        self.label1.setFixedWidth(100)
        self.label1.setFixedHeight(25)
        self.label1.move(120, 10)

        # Label to show current follow significance
        self.label2 = QLabel(self)
        self.label2.setStyleSheet(style+„background-co-
lor: white;font-size: 8pt;“)
        self.label2.setText(„W: 0“)
        self.label2.setFixedWidth(100)
        self.label2.setFixedHeight(25)
        self.label2.move(120, 45)

        #TextEdit1 Label
        self.textEditLabel1 = QLabel(self)

```

```

        self.textEditLabel1.setStyleSheet(style+"backg-
round-color: white;font-size: 8pt;")
        self.textEditLabel1.setText(„Very important:“)
        self.textEditLabel1.setFixedWidth(125)
        self.textEditLabel1.setFixedHeight(30)
        self.textEditLabel1.move(240,10)
        # TextEdit1
        self.textEdit1 = QPlainTextEdit(self)
        self.textEdit1.setFixedWidth(125)
        self.textEdit1.setFixedHeight(65)
        self.textEdit1.keyPressEvent
        self.textEdit1.appendPlainText(„know understand
secret conscience think memory“)
        self.textEdit1.move(240,45)
        self.very_important_words = self.textEdit1.toP-
lainText().split(„ “)
        print(„Very important words: „, self.very_important_
words)
        def getsettext1():
            self.very_important_words = self.textEdit1.
toPlainText().split(„ “)
            print(„Very important words: „, self.very_im-
portant_words)

        # Number Box1
        self.enterNumber1 = QDoubleSpinBox(self)
        self.enterNumber1.setStyleSheet(style)
        self.enterNumber1.setRange(0,1000000)
        self.enterNumber1.setFixedWidth(125)
        self.enterNumber1.setFixedHeight(25)
        self.enterNumber1.move(240,115)
        self.enterNumber1.setValue(20)
        self.very_important_value = self.enterNumber1.
value()
        print(„Very important value: „, self.very_important_
value)
        def getsetvalue1():
            self.very_important_value = self.enterNum-
ber1.value()
            print(„Very important value: „, self.very_im-
portant_value)

```

```

        self.enterNumber1.valueChanged.connect(getset-
value1)

        #TextEdit2 Label
        self.textEditLabel2 = QLabel(self)
        self.textEditLabel2.setStyleSheet(style+"backg-
round-color: white;font-size: 8pt;")
        self.textEditLabel2.setText(„Important:“)
        self.textEditLabel2.setFixedWidth(125)
        self.textEditLabel2.setFixedHeight(30)
        self.textEditLabel2.move(240,150)
        # TextEdit2
        self.textEdit2 = QPlainTextEdit(self)
        self.textEdit2.setFixedWidth(125)
        self.textEdit2.setFixedHeight(65)
        self.textEdit2.appendPlainText(„why because word
words number“)
        self.textEdit2.move(240,185)
        self.important_words = self.textEdit2.toPlain-
Text().split(„ “)
        print(„Important words: „, self.important_words)
        # Number Box2
        self.enterNumber2 = QDoubleSpinBox(self)
        self.enterNumber2.setStyleSheet(style)
        self.enterNumber2.setRange(0,1000000)
        self.enterNumber2.setFixedWidth(125)
        self.enterNumber2.setFixedHeight(25)
        self.enterNumber2.move(240,255)
        self.enterNumber2.setValue(5)
        self.important_value = self.enterNumber2.value()
        print(„Important value: „, self.important_value)
        def getsetvalue2():
            self.important_value = self.enterNumber2.
value()
            print(„Important value: „, self.important_
value)
        self.enterNumber2.valueChanged.connect(getset-
value2)

        #TextEdit3 Label
        self.textEditLabel3 = QLabel(self)

```

```

        self.textEditLabel3.setStyleSheet(style+"backg-
round-color: white;font-size: 8pt;")
        self.textEditLabel3.setText(„Unimportant:“)
        self.textEditLabel3.setFixedWidth(125)
        self.textEditLabel3.setFixedHeight(30)
        self.textEditLabel3.move(240,290)
        # TextEdit3
        self.textEdit3 = QPlainTextEdit(self)
        self.textEdit3.setFixedWidth(125)
        self.textEdit3.setFixedHeight(65)
        self.textEdit3.appendPlainText(„the you of i to
a in and is it what are that me no for your one we „its“
„don't“ not but this my was with be or sir here do like
they mr outlands have so from where very must all al-
phaville johnson if „ill“ will he alpha when see as by
60 „youre“ yes who an them which can never now well go
vonbraun him too at es light come say those us afraid
„whats“ about man were get thank going miss there night
professor control said „ive“ only our his their men day
on her natasha death „thats“ „theres““)
        self.textEdit3.move(240,325)
        self.unimportant_words = self.textEdit3.toPlain-
Text().split(„ „)
        print(„Unimportant words: „, self.unimportant_
words)
        # Number Box3
        self.enterNumber3 = QDoubleSpinBox(self)
        self.enterNumber3.setStyleSheet(style)
        self.enterNumber3.setRange(0,1000000)
        self.enterNumber3.setFixedWidth(125)
        self.enterNumber3.setFixedHeight(25)
        self.enterNumber3.move(240,395)
        self.enterNumber3.setValue(0.1)
        self.unimportant_value = self.enterNumber3.
value()
        print(„Unimportant value: „,self.unimportant_
value)
        def getsetvalue3():
            self.unimportant_value = self.enterNumber3.
value()
            print(„Unimportant value: „,self.unimportant_
value)

```

```

        self.enterNumber3.valueChanged.connect(getset-
value3)

        #TextEdit4 Label
        self.textEditLabel4 = QLabel(self)
        self.textEditLabel4.setStyleSheet(style+"backg-
round-color: white;font-size: 8pt;")
        self.textEditLabel4.setText(„Else:“)
        self.textEditLabel4.setFixedWidth(125)
        self.textEditLabel4.setFixedHeight(30)
        self.textEditLabel4.move(240,430)
        #Number Box4
        self.enterNumber4 = QDoubleSpinBox(self)
        self.enterNumber4.setStyleSheet(style)
        self.enterNumber4.setRange(0,1000000)
        self.enterNumber4.setFixedWidth(125)
        self.enterNumber4.setFixedHeight(25)
        self.enterNumber4.move(240,465)
        self.enterNumber4.setValue(1)
        self.else_value = self.enterNumber4.value()
        print(„Else value: „, self.else_value)
        def getsetvalue4():
            self.else_value = self.enterNumber4.value()
            print(„Else value: „, self.else_value)
            self.enterNumber4.valueChanged.connect(getset-
value4)

        #self.textEdit1.changeEvent(getsettext)
        self.words_button = QPushButton(„Get words“,self)
        self.words_button.setStyleSheet(str(style+"backg-
round-color: green;color: white; border-color: white;"))
        def getsettext1():
            self.very_important_words = self.textEdit1.
toPlainText().split(„ „)
            print(„Very important words: „, self.very_im-
portant_words)
            self.words_button.clicked.connect(getsettext1)
        def getsettext2():
            self.important_words = self.textEdit2.toP-
lainText().split(„ „)
            print(„Important words: „, self.important_
words)

```

```

        self.words_button.clicked.connect(getsettext2)
    def getsettext3():
        self.unimportant_words = self.textEdit3.toPlainText().split(" ")
        print("Unimportant words: ", self.unimportant_words)
        self.words_button.clicked.connect(getsettext3)
        self.words_button.setFixedWidth(80)
        self.words_button.move(240,500)

    # Button: Open New Subtitles
    self.new = "Alphaville.ENG.srt"
    def opennew():
        self.new = QFileDialog.getOpenFileName(self,
        "Select new Subtitle", "", "Srt (*.srt)")[0]
        self.openNewST = QPushButton("New Subtitle", self)
        self.openNewST.setStyleSheet(str(style+"background-color: green;color: white;border-color: white;"))
        self.openNewST.clicked.connect(opennew)
        self.openNewST.setFixedWidth(100)
        self.openNewST.move(10,45)

    # Button: Open Learned Subtitles
    self.learned = "Alphaville.ENG.srt"
    def openlearned():
        self.learned = QFileDialog.getOpenFileName(self, "Select old Subtitle", "", "Srt (*.srt)")[0]
        learned = Subtitles()
        learned.readUT(self.learned)
        print(Counter(learned.countlist).most_common(10))
        tups = Counter(learned.countlist).most_common(200)
        tupslst = , , .join(map(str, tups))
        tupslst = tupslst.replace(", ", "\n")
        tupslst = tupslst.replace(", ", "")
        tupslst = tupslst.replace("'", "")
        self.textEdit4.insertPlainText(tupslst)
        self.openLearnedST = QPushButton("Old Subtitle", self)
        self.openLearnedST.setStyleSheet(str(style+"background-color: green;color: white;border-color: white;"))

```

```

        self.openLearnedST.setFixedWidth(100)
        self.openLearnedST.clicked.connect(openlearned)
        self.openLearnedST.move(10,80)
        learned = Subtitles()
        learned.readUT(self.learned)
        print(Counter(learned.countlist).most_common(10))
        tups = Counter(learned.countlist).most_common(200)
        tupslst = , , .join(map(str, tups))
        tupslst = tupslst.replace(", ", "\n")
        tupslst = tupslst.replace(", ", "")
        tupslst = tupslst.replace("'", "")

    # TextEdit4
    self.textEdit4 = QPlainTextEdit(self)
    self.textEdit4.setFixedWidth(100)
    self.textEdit4.setFixedHeight(200)
    self.textEdit4.appendPlainText(tupslst)
    self.textEdit4.move(120,100)

    #self.wordList.addTopLevelItems(Counter(learned.countlist).most_common(10))
    #self.wordList.move(240,325)

    # Button: Make Sequence # self_important words
    see textedit above TypeError("index 0 has type 'tuple' but 'str' is expected",)

    def seq():
        try:
            self.sequence, self.sig = make_seq(self.new, self.learned, self.write, self.else_value, self.very_important_value, self.important_value, self.unimportant_value, self.very_important_words, self.important_words, self.unimportant_words)
            sc.refresh_figure(self.sig)
            self.plot = QWidget(self)
            l = QVBoxLayout(self.plot)
            l.addWidget(sc)

```

```

        self.plot.move(5,320)
        self.plot.show()
    except:
        if self.write == 0: print("\nFollow Mat-
rix not available - click ,write fol_Matrix'")
        else: print("New and learned Subtitles
need to be chosen!")
        self.write = 0 # can be changed with checkbox
        self.start = QPushButton("Learn!", self)
        self.start.setStyleSheet(style)
        self.start.setFixedWidth(100)
        self.start.clicked.connect(seq)
        self.start.move(10,115)

# Button: Print Sequence, Plot follow Significan-
ce, set Video positions
def test():
    try:
        print("\nFilmfolge: ", self.sequence,
"\n")
        new = Subtitles()
        new.readUT(self.new)
        for i in range(20):
            nr = self.sequence[i]
            print(Fore.GREEN, new.subtitle[nr].
start, " - ", new.subtitle[nr].end, ": ",
Style.RESET_ALL, " ".join(new.
subtitle[nr].words))
            print("...")
            learned.countlist.extend(new.countlist)
        except: print("No sequence yet")
        self.test = QPushButton("Print sequence", self)
        self.test.setStyleSheet(style)
        self.test.setFixedWidth(100)
        self.test.clicked.connect(test)
        self.test.move(10,150)

# Button: Apply sequence
def apply():
    try:
        new = Subtitles()
        new.readUT(self.new)

```

```

# hier wird die Position anhand der
"self.sequence" fortlaufend neu bestimmt
def setposition(t):
    if t >= len(new.subtitle): re-
turn("end")
    nr = self.sequence[t]
    self.player.mediaPlayer.setPositi-
on(1000*new.subtitle[nr].start -0.4)
    self.label1.setText("Nr: "+str(nr))
    self.label2.setText("W: "+str("%.3f"
% self.sig[t]))
    print(nr)
    t += 1
    # stoppe wenn der letzte UT erreicht,
oder die Signifikanz 0 ist
    if nr < len(self.sequence)-1 and 0 <
self.sig[t]:
        #if t < 5:
            duration = new.subtitle[nr].dura-
tion + (new.subtitle[nr+1].start - new.subtitle[nr].end)
            - 0.4
            Timer(duration, lambda: setposi-
tion(t)).start()
        elif nr < len(self.sequence)-1:
            print("Letzte Szene!")
            duration = new.subtitle[nr].
duration + (new.subtitle[nr+1].start - new.subtitle[nr].
end)
            Timer(duration, self.player.
play).start()
        else: print("Letzte Szene!")
            self.player.play()
            setposition(0)
        except: print("No sequence yet")
        self.test = QPushButton("Apply sequence", self)
        self.test.setStyleSheet(style)
        self.test.setFixedWidth(100)
        self.test.clicked.connect(apply)
        self.test.move(10,185)

# Checkbox: Write folMatrix
self.check = QCheckBox("Write \nfolMatrix", self)

```



```

        self.check.setStyleSheet(style)
        self.check.setFixedWidth(100)
        self.check.stateChanged.connect(self.checked)
        self.check.move(10,220)

    self.show()

def checked(self, state):
    if state == QtCore.Qt.Checked: self.write = 1
    else: self.write = 0

if __name__ == '__main__':

    app = QApplication(sys.argv)
    GUI = Interface()
    sys.exit(app.exec_())

```

Subtitle_class.py

```

from collections import defaultdict, Counter
import random
import json
from numpy import array, argsort, unique
# This class stores a single subtitle with start and end
time.
# Methods can change time format.
class sub():
    def __init__(self, words = [], start = 0, end = 0):
        self.words = words
        self.length = len(words)
        self.start = start
        self.end = end
        self.duration = end - start

    def add(self, word):
        self.words.append(word)
        self.length = len(self.words)

    def frameToSec(self, fps = 25):
        self.start = self.start/fps
        self.end = self.end/fps
        self.duration = self.end - self.start

    def secToFrame(self, fps = 25):
        self.start = self.start*fps
        self.end = self.end*fps
        self.duration = self.end - self.start

# This class stores a set of subtitles. Methods:
# readUT reads in a subtitle from an .srt file.
# make_sigMatrix makes the sigMatrix (from learned subtitles)
# make_folMatrix makes the folMatrix (from new subtitles
and the sigMatrix)
class Subtitles:
    def __init__(self, name = ""):
        self.name = name
        self.subtitle = []
        self.length = len(self.subtitle)

```

```

def readUT(self, path): # subtitle aus .srt Datei
einlesen
    with open(path) as file:
        UT_raw = file.read()
        UT = [] # Liste fuer formatierten Untertiteln
        lines = []
        countwords = []
        for i in UT_raw.split("\n"):
            if i != ',': lines.append(i) # Untertitel sammeln
            elif lines == []: continue
            else: # Untertitel formatieren und in UT einfügen
                t_start_ = lines[1].replace(",", ".").split(" --> ")[0].split(":")
                t_start = 60*60*int(t_start_[0])+60*int(t_start_[1])+float(t_start_[2])
                t_end = lines[1].replace(",", ".").split(" --> ")[1].split(":")
                t_end = 60*60*int(t_end[0])+60*int(t_end[1])+float(t_end[2])
                words = , ,.join(lines[2:]).lower()
                for s in ,;":.!?-&':
                    words = words.replace(s, "")
                words = words.lower().split(" ")
                countwords.extend(words)
                UT.append(sub(words, t_start, t_end))
                lines = []
        self.countlist = countwords
        #countlist = Counter(countwords).most_common(10)
        #print(countlist)

        # Einsetzen
        self.name = path.split("/")[-1].replace(".srt", "")
        self.subtitle = UT
        self.length = len(UT)

    def word_follow_significance(self, word1, word2, else_value = 0, very_important_value = 0, important_value = 0,

```

```

unimportant_value = 0, very_important_words = "", important_words = "", unimportant_words = ""):
    y = else_value
    very_important = very_important_words
    important = important_words
    unimportant = unimportant_words
    if word1 in very_important or word2 in very_important: y = very_important_value
    if word1 in important or word2 in important: y = important_value
    if word1 in unimportant or word2 in unimportant: y = unimportant_value
    return(y)

    def make_sigMatrix(self, else_value = 0, very_important_value = 0, important_value = 0, unimportant_value = 0,
    very_important_words = [""], important_words = [""], unimportant_words = [""]):
        sigMatrix = defaultdict(Counter)
        last = sub()
        for current in self.subtitle:
            for word1 in last.words:
                for word2 in current.words:
                    sigMatrix[word1][word2] += self.word_follow_significance(word1, word2, else_value, very_important_value, important_value, unimportant_value,
                    very_important_words, important_words, unimportant_words)
                last = current
        return [sigMatrix, self.name]

    def make_folMatrix(self, sigMatrix, write = 1):
        filename = "folMatrix_{}-{}.txt".format(self.name, sigMatrix[1])
        #try:
        #    with open(filename, 'r') as in_file:
        #        folMatrix = json.load(in_file)
        #    print("\nUsing saved follow Matrix", filename)
        #except:
        #    def follow_sign(ut1, ut2):
        #        return(sum([sigMatrix[0][word1][word2] for word1 in ut1.words for word2 in ut2.words]))

```

```

        # print("\nstart making folmatrix")
        # folMatrix = [[follow_sign(ut1, ut2)/(ut1.
length+ut2.length)
        # for ut2 in self.subtitle] for
ut1 in self.subtitle]
        # print("\nend making folmatrix")
        # for i,j in enumerate(folMatrix): folMat-
rix[i][i] = -1
        # if write == 1:
        #     with open(filename, 'w') as out_file:
        #         json.dump(folMatrix, out_file)
        if write == 0:
            with open(filename, 'r') as in_file:
                folMatrix = json.load(in_file)
                print("\nUsing saved follow Matrix", filename)
        if write == 1:
            def follow_sign(ut1, ut2):
                return(sum([sigMatrix[0][word1][word2]
for word1 in ut1.words for word2 in ut2.words]))
                print("\nstart making folmatrix")
                folMatrix = [[follow_sign(ut1, ut2)/(ut1.
length+ut2.length)
                                for ut2 in self.subtitle] for
ut1 in self.subtitle]
                print("\nend making folmatrix")
                for i,j in enumerate(folMatrix): folMatrix[i]
[i] = -1
                with open(filename, 'w') as out_file:
                    json.dump(folMatrix, out_file)
                return folMatrix

# Nachfolger-Funktion
def findfollowers(x, folMatrix):
    sig = max(folMatrix[x])
    best_uts = [i for i,j in enumerate(folMatrix[x]) if j
== sig]
    new_x = random.choice(best_uts)
    folMatrix[x] = [-1 for i in range(len(folMatrix))]
    for r,s in enumerate(folMatrix[x]):
        folMatrix[r][x] = -1
    return([sig, new_x])

```

```

# Funktion zum Erstellen der neuen Filmabfolge
def make_seq(newname, learnname, write = 1, else_value =
0, very_important_value = 0, important_value = 0, unim-
portant_value = 0, very_important_words = [""], import-
ant_words = [""], unimportant_words = [ "")]:
    learn = Subtitles()
    learn.readUT(learnname)
    sigMatrix = learn.make_sigMatrix(else_value, very_im-
portant_value, important_value, unimportant_value, very_
important_words, important_words, unimportant_words)
    new = Subtitles()
    new.readUT(newname)
    folMatrix = new.make_folMatrix(sigMatrix, write)
    # Filmabfolge
    x = 0 # start
    sequenz = [x]
    sig = []
    # for i in range(10):
    while True:
        ff = findfollowers(x, folMatrix)
        if ff[0]<0: break
        sig.append(ff[0])
        sequenz.append(ff[1])
        x = ff[1]
    # Output
    print("Neues Material: {}".format(new.name) +
        "\nLernmaterial: {}".format(learn.name) +
        "\nDie Filmabfolge besteht aus {} von {} Szenen.".
format(str(len(sequenz)),new.length))
    return(sequenz, sig)

```

VideoPlayer_class.py

```
from PyQt5.QtCore import QDir, Qt, QUrl
from PyQt5.QtMultimedia import QMediaContent, QMediaPlayer
from PyQt5.QtMultimediaWidgets import QVideoWidget
from PyQt5.QtWidgets import (QApplication, QFileDialog,
                              QHBoxLayout, QLabel,
                              QPushButton, QSizePolicy, QSlider, QStyle, QV-
                             BoxLayout, QWidget)

class VideoPlayer(QWidget):
    def __init__(self, parent=None):
        super(VideoPlayer, self).__init__(parent)

        self.mediaPlayer = QMediaPlayer(None, QMediaPlay-
        er.VideoSurface)

        videoWidget = QVideoWidget()

        self.playButton = QPushButton()
        self.playButton.setEnabled(False)
        self.playButton.setIcon(self.style().standardI-
        con(QStyle.SP_MediaPlay))
        self.playButton.clicked.connect(self.play)

        self.positionSlider = QSlider(Qt.Horizontal)
        self.positionSlider.setRange(0, 0)
        self.positionSlider.sliderMoved.connect(self.set-
        Position)

        self.errorLabel = QLabel()
        self.errorLabel.setSizePolicy(QSizePolicy.Prefer-
        red,
                                   QSizePolicy.Maximum)

        controlLayout = QHBoxLayout()
        controlLayout.setContentsMargins(0, 0, 0, 0)
        controlLayout.addWidget(self.playButton)
        controlLayout.addWidget(self.positionSlider)
```

```
        layout = QVBoxLayout()
        layout.addWidget(videoWidget)
        layout.addLayout(controlLayout)
        layout.addWidget(self.errorLabel)

        self.setLayout(layout)
        self.mediaPlayer.setVideoOutput(videoWidget)
        self.mediaPlayer.stateChanged.connect(self.medi-
        aStateChanged)
        self.mediaPlayer.positionChanged.connect(self.po-
        sitionChanged)
        self.mediaPlayer.durationChanged.connect(self.
        durationChanged)
        self.mediaPlayer.error.connect(self.handleError)

        def openFile(self):
            fileName, _ = QFileDialog.getOpenFileName(self,
            „Open Movie“)

            if fileName != ,':
                self.mediaPlayer.setMedia(
                    QMediaContent(QUrl.fromLocalFile(file-
                    Name)))
                self.playButton.setEnabled(True)

        def play(self):
            if self.mediaPlayer.state() == QMediaPlayer.Play-
            ingState:
                self.mediaPlayer.pause()
            else:
                self.mediaPlayer.play()

        def mediaStateChanged(self, state):
            if self.mediaPlayer.state() == QMediaPlayer.Play-
            ingState:
                self.playButton.setIcon(
                    self.style().standardIcon(QStyle.SP_
                    MediaPause))
            else:
                self.playButton.setIcon(
                    self.style().standardIcon(QStyle.SP_
                    MediaPlay))
```

```

def positionChanged(self, position):
    self.positionSlider.setValue(position)

def durationChanged(self, duration):
    self.positionSlider.setRange(0, duration)

def setPosition(self, position):
    self.mediaPlayer.setPosition(position)
    self.mediaStateChanged(position)

def handleError(self):
    self.playButton.setEnabled(False)
    self.errorLabel.setText("Error: " + self.media-
Player.errorString())

```

MplCanvas.py

```

import matplotlib
from PyQt5.QtWidgets import QSizePolicy
matplotlib.use("Qt5Agg")
from numpy import arange, sin, pi
from matplotlib.backends.backend_qt5agg import FigureCan-
vasQTAgg as FigureCanvas
from matplotlib.figure import Figure

class MyMplCanvas(FigureCanvas):
    """Ultimately, this is a QWidget (as well as a Figu-
reCanvasAgg, etc.)."""
    def __init__(self, parent=None, width=5, height=4,
dpi=50, data = [0]):
        fig = Figure(figsize=(width, height), dpi=dpi)
        self.axes = fig.add_subplot(111)
        self.compute_initial_figure()

        FigureCanvas.__init__(self, fig)
        self.setParent(parent)

        FigureCanvas.setSizePolicy(self,
            QSizePolicy.Expanding,
            QSizePolicy.Expanding)
        FigureCanvas.updateGeometry(self)

    def compute_initial_figure(self):
        pass

    def refresh_figure(self, data):
        t = range(len(data))
        s = data
        self.axes.plot(t, s)

```


Zellulärer Automat

Eine weitere Möglichkeit, generative Strukturen zu erzeugen, mit der ich mich auseinandergesetzt habe, ist eine Form des zellulären Automaten, John Conways »Game of Life«. Das ist einer der Algorithmen, mit denen auch künstlerisch schon viel gearbeitet wurde. Dabei war meine Beschäftigung damit nicht von vorneherein geplant, sondern hat sich über Umwege ergeben. Angefangen hat es damit, dass ich zusammen mit Ingo Stock mit »Pure Data« ein zweidimensionales Raster als Interface programmiert habe (Abb. 55). Damit lassen sich Töne oder Bilder sowohl durch Einzeichnen mit der Maus, als auch algorithmisch generativ, in einer visuellen Matrix, also sozusagen einer Timeline mit mehreren Spuren, arrangieren.

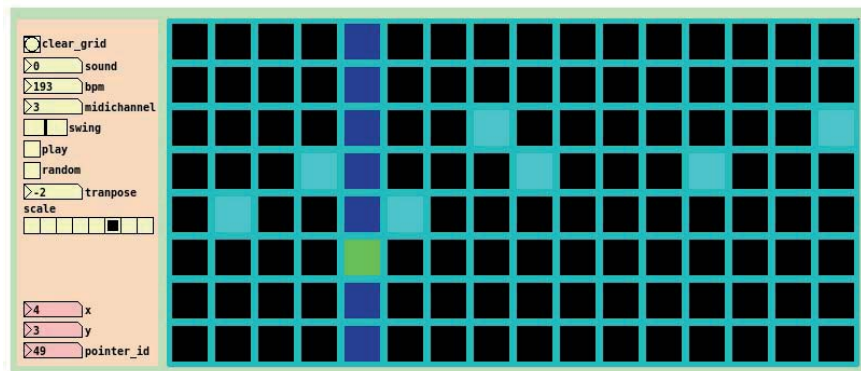


Abb. 55: Bedienoberfläche des Struct-Sequencers, der Ausgangspunkt für die Experimente mit den zellulären Automaten war.

Aufgrund dieser Vorlage haben wir unterschiedliche Algorithmen zum Erzeugen von (in erster Linie musikalischen) Strukturen ausprobiert. Wichtig war besonders bei diesem Projekt auch der Austausch der Programme und Ideen über das Internet-Forum »Pure Data Patch Repository«⁵⁸. So ist dann auch von dem Nutzer »Weightless« der »Game of Life«

⁵⁸ <https://forum.pdpatchrepo.info> [22.5.2020].

Algorithmus von John Conway implementiert worden. Damit habe ich wiederum versucht, durch Modifikationen und Zuordnung von Tönen, musikalische Strukturen zu erzeugen. Die auf den folgenden Seiten abgedruckte »Game of Life« Sequenz (Abb. 56 - 151) habe ich wiederum mit einem Programm erzeugt, das ich einige Zeit später mit der »Pure Data« Bibliothek »Ofelia« geschrieben habe. Auch dieses Programm ist in erster Linie als Sequenzer gedacht, in diesem Fall habe ich das Raster aber zur visuellen Darstellung der generierten Struktur genutzt.

Reizvoll finde ich an dem zellulären Automaten nicht nur die generierten Strukturen, sondern auch die Tatsache, dass es sich dabei um einen Automaten im mathematischen Sinn handelt, der unter anderem dazu in der Lage ist, die Populationsentwicklung von Lebewesen zu simulieren.

Zur Funktionsweise und Geschichte des zellulären Automaten:

»Zelluläre Automaten sind Algorithmen, die mit relativ einfachen Regeln Prozesse in zwei- oder mehrdimensionalen Räumen beschreiben können. Sie wurden ab den 1940er-Jahren von verschiedenen MathematikerInnen und WissenschaftlerInnen entwickelt. Der Mathematiker John Horton Conway hat 1970 mit »Game of Life« einen zellulären Automaten definiert, mit dem es möglich ist Prozesse der Populationsentwicklung von Lebewesen zu simulieren. Conways zweidimensionaler Automat besteht quasi aus einem karierten Blatt auf dem einzelne Zellen platziert sind. Diese können in der Grundform zwei Existenzformen aufweisen: lebendig oder tot. Der Zustand der jeweiligen Zellen wird durch den Zustand der Nachbarzellen definiert. Dazu kommt der Ausgangszustand, bei dem beliebige Zellen als lebendig festgelegt werden. Nun wird jede Zelle mit folgenden Regeln konfrontiert:

Eine tote Zelle mit genau drei lebenden Nachbarzellen wird in der Folgegeneration neu geboren.

Lebende Zellen mit weniger als zwei lebenden Nachbarzellen sterben in der Folgegeneration an Einsamkeit.

Eine lebende Zelle mit zwei oder drei lebenden Nachbarzellen bleibt in der Folgegeneration am Leben.

Lebende Zellen mit mehr als drei lebenden Nachbarzellen sterben in der Folgegeneration an Überbevölkerung.«⁵⁹

Von lebenden und toten Zellen wird gesprochen, weil der Algorithmus unter anderem die Populationsentwicklung von Lebewesen simuliert. Man kann aber auch ganz allgemein von zwei Zuständen sprechen.

Für mein Beispiel habe ich die Regeln des »Game of Life« modifiziert. Eine Zelle stirbt nicht sofort, sondern verändert sechsmal ihre Farbe, und zeichnet sich solange weiterhin durch die Eigenschaften einer lebenden Zelle aus. Durch diese Modifikation wird die Struktur durch eine zeitliche Farbigkeit als weitere Dimension ergänzt. Das Resultat stellt keine Notation für etwas dar, sondern steht als sich in der Zeit verändernde Form für sich.

»Der Grund, warum sich zahlreiche freie künstlerische Arbeiten mit zellulären Automaten befassen, ist vermutlich weniger deren praktische Anwendbarkeit an sich als vielmehr die Frage, warum diese überhaupt erst besteht. Einige zelluläre Automaten wie das »Game of Life« bilden natürliche Vorgänge mit erstaunlichem Realismus ab und sind gleichzeitig zu universeller Berechnung fähig. Die Frage nach einem tieferen Zusammenhang dieser beiden Eigenschaften ist dabei älter als die Begriffe der Universalität und zellulärer Automaten selbst. Der Gedanke, dass »Mathematik die Sprache der Natur«⁶⁰ sei, zieht sich durch die Geschichte der Wissenschaft. Häufig kommt dieser in besonderen Zahlenverhältnissen, die in der Natur zu finden sind, zum Ausdruck. Diese finden sich auch in der Kunst wieder. Auch wenn zelluläre Automaten ein neuerer Gegenstand in Mathematik und Informatik sind, fin-

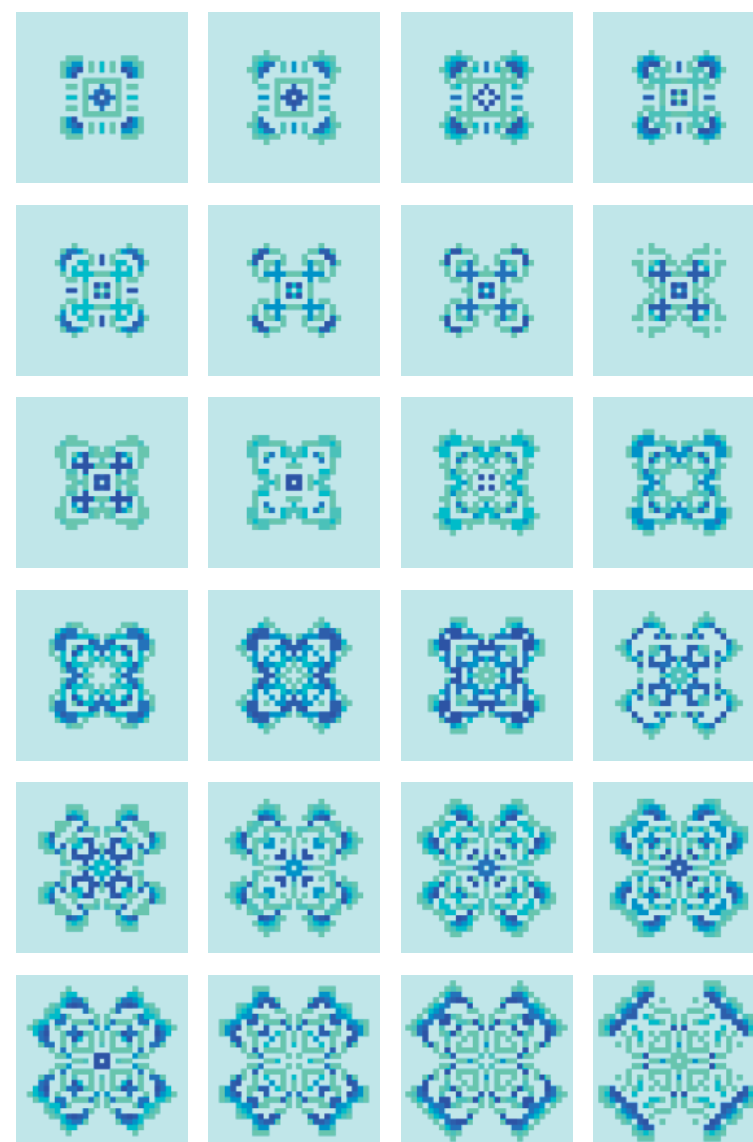
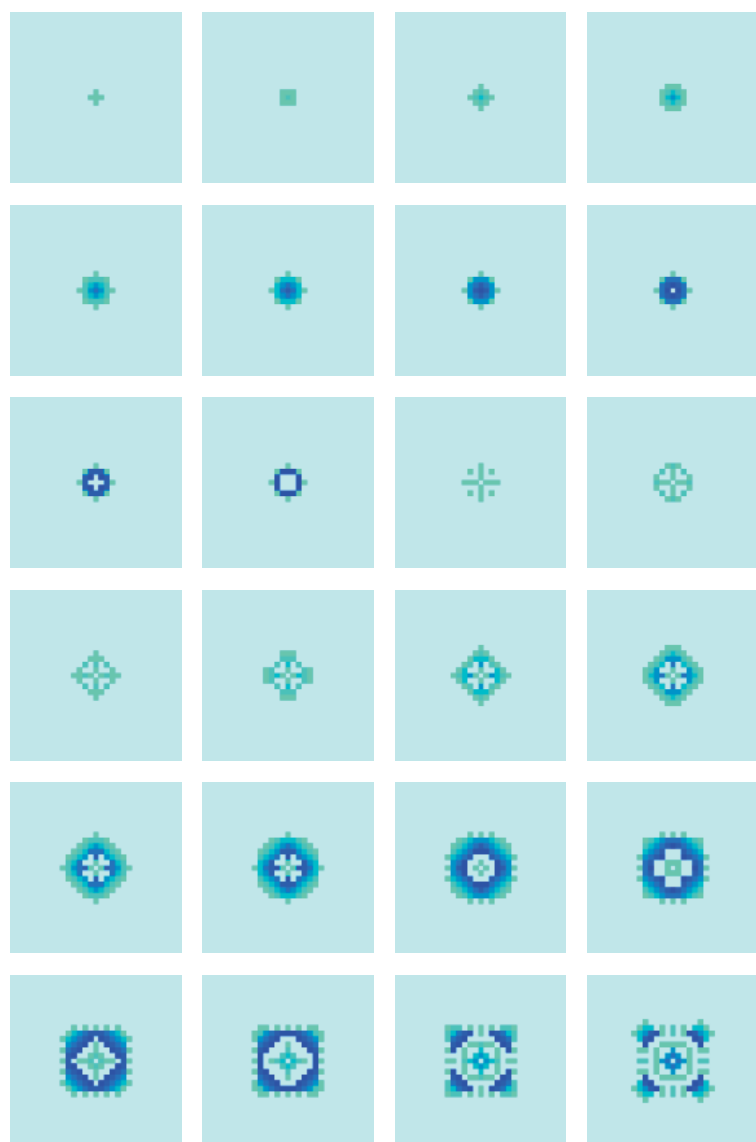
59 Ludger Brümmer / Benjamin Miller: CellularAutomataExplorer, 2017. <https://zkm.de/de/cellularautomataexplorer> [22.5.2020].

60 Erhard Behrends: Ist Mathematik die Sprache der Natur? In: Spektrum der Wissenschaft Highlights. Heidelberg: Spektrum Verlag, 2014. S. 82.

det sich bereits in den Schriften von Leibniz die Idee, dass die Welt diskreten Regeln folge und die Organismen in ihr agieren wie die Zellen in einem Automaten. Diese Idee wird im 20. Jahrhundert auch von John von Neumann in »Theory of Self Replicating Automata« (1966) aufgegriffen. Kurz darauf spitzt Konrad Zuse in seiner Schrift »Rechnender Raum«⁶¹ den Gedanken auf die These zu, dass das gesamte Universum der innere Zustand eines nach simplen Regeln arbeitenden zellulären Automaten sei. Diese Theorie bildet damit einen Gegensatz zu den immer komplexer werdenden Erklärungsversuchen der Physik. Eine mögliche Antwort auf die Frage, warum zelluläre Automaten in der freien Kunst thematisiert werden, könnte folglich so formuliert werden: Zelluläre Automaten verkörpern die uralte Frage nach dem Zusammenhang von Natur und Mathematik auf eine Weise, die die komplexen Erklärungsmodelle des 20. und 21. Jahrhunderts berücksichtigt und gleichzeitig kontrastiert. Da diese Frage bis heute nicht beantwortbar ist, ist sie bis heute ein Anreiz für Auseinandersetzungen in den bildenden und angewandten Künsten.«⁶²

61 Konrad Zuse: Rechnender Raum, 1967. In: Spektrum der Wissenschaft – Spezial: Ist das Universum ein Computer? Heidelberg: Spektrum Verlag, 2007. S. 7 - 15.

62 Franziska Schneider: Explorative Visualisierung zellulärer Automaten. Augsburg: Bachelorarbeit im Studiengang Kommunikationsdesign der Hochschule Augsburg, 2018. S. 46. <http://zellmemore.de/zellmemore.pdf> [22.5.2020].



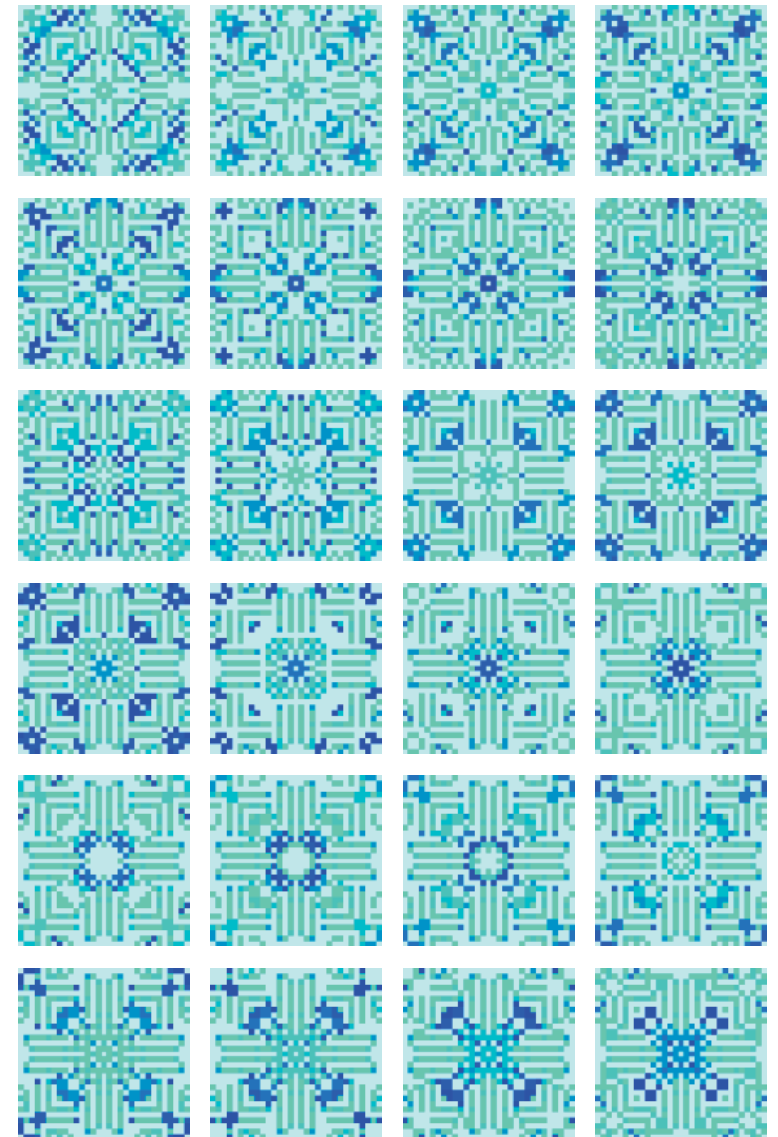
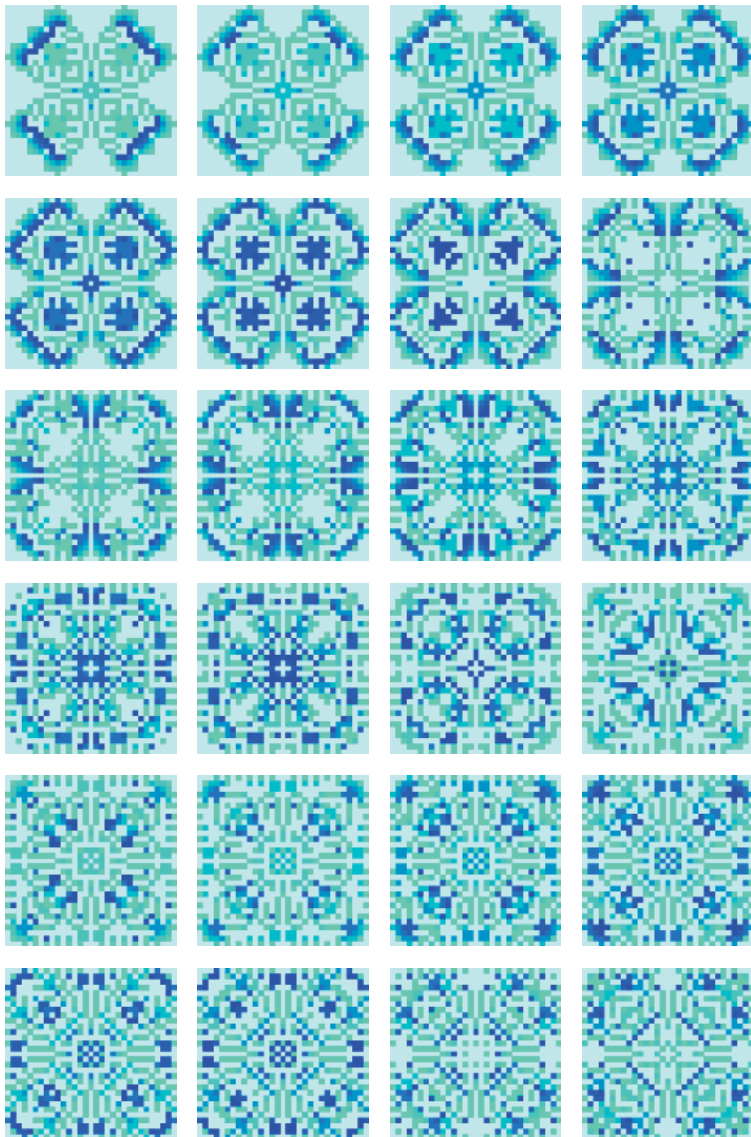


Abb. 56 - 151: Eine Sequenz aus 96 Generationen von John Conways »Game of Life«. Generiert von einem Programm, das ich mit »Pure Data« und dessen Bibliothek »Ofelia« geschrieben habe.

»Pure Data« Experimente - ein Katalog

Im Rahmen meiner Masterarbeit sind neben dem Montage-Automaten viele weitere Programmierexperimente mit »Pure Data« entstanden. Einige davon möchte ich in Form eines Katalogs exemplarisch vorstellen, um einen Überblick über die Vielfalt meiner Arbeit und die Möglichkeiten von »Pure Data« zu verschaffen.

Wie in einem freien Forschungslabor bin ich mit Johannes Frank und Ingo Stock, aber auch viel alleine, unterschiedlichsten Überlegungen zur algorithmischen Komposition nachgegangen. Dabei sind unter anderem Musik-Generatoren, wiederverwendbare Module und Bedienoberflächen entstanden.

Den besten Eindruck von den Programmen bekommt man jedoch nicht durch Screenshots und eine Beschreibung, sondern durch das Ausprobieren. Deshalb sende ich allen Interessierten gerne meine Programme zu, und erkläre bei Bedarf deren Benutzung. In Zukunft will ich eine Auswahl auch auf Github⁶³ zur Verfügung stellen.

Zusätzlich zu dem Katalog gibt es Videoskizzen auf meinem Youtube-Kanal »AudioVideo«⁶⁴ und einzelne ausführbare Programme auf meiner Internetseite »Handmadeproductions«⁶⁵.

63 <https://github.com/Jonathhhan> [22.5.2020].

64 <https://www.youtube.com/channel/UCVI9jk5rnDcCHARIwAn5yYA> [22.5.2020].

65 <http://index.handmadeproductions.de> [22.5.2020].

»GLSL« Video-Effekte

»GLSL« ist die Abkürzung für die Programmiersprache »Graphics Library Shading Language«. Mit ihr lassen sich Programme, sogenannte Shader, programmieren, die von Grafikkartenprozessoren ausgeführt werden. Grafikkarten eignen sich besonders für rechenintensive Operationen, wie dem Verändern von Videoinformation in Echtzeit, da sie zwar nicht so genau wie ein Computerprozessor sind, aber dafür mit vielen Prozessoren parallel berechnen können. Diese Eigenschaft der Grafikkarte wird auch im Machine-Learning zum Trainieren von künstlichen neuronalen Netzen genutzt. Ich habe mit der Programmiersprache »GLSL«, sowie »Ofe-lia« und »Pure Data« unter anderem ein Video-Effekt Programm geschrieben, mit dem man unterschiedliche Effekte erzeugen und miteinander kombinieren kann.

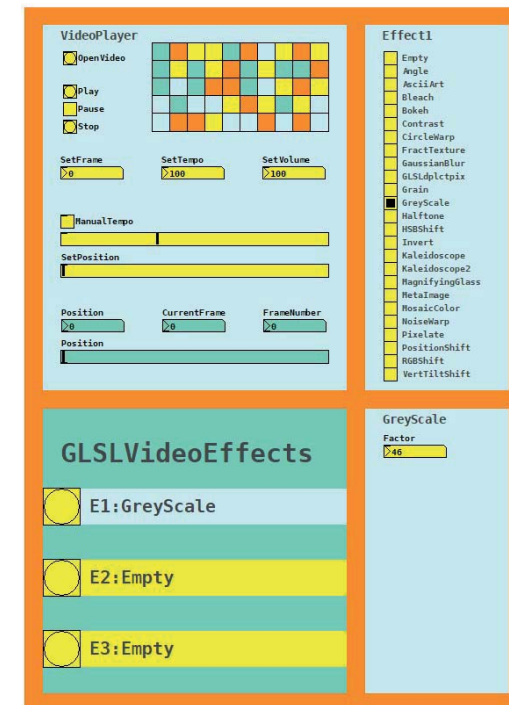


Abb. 152: Bedienoberfläche des Programms GLSLVideoEffects.

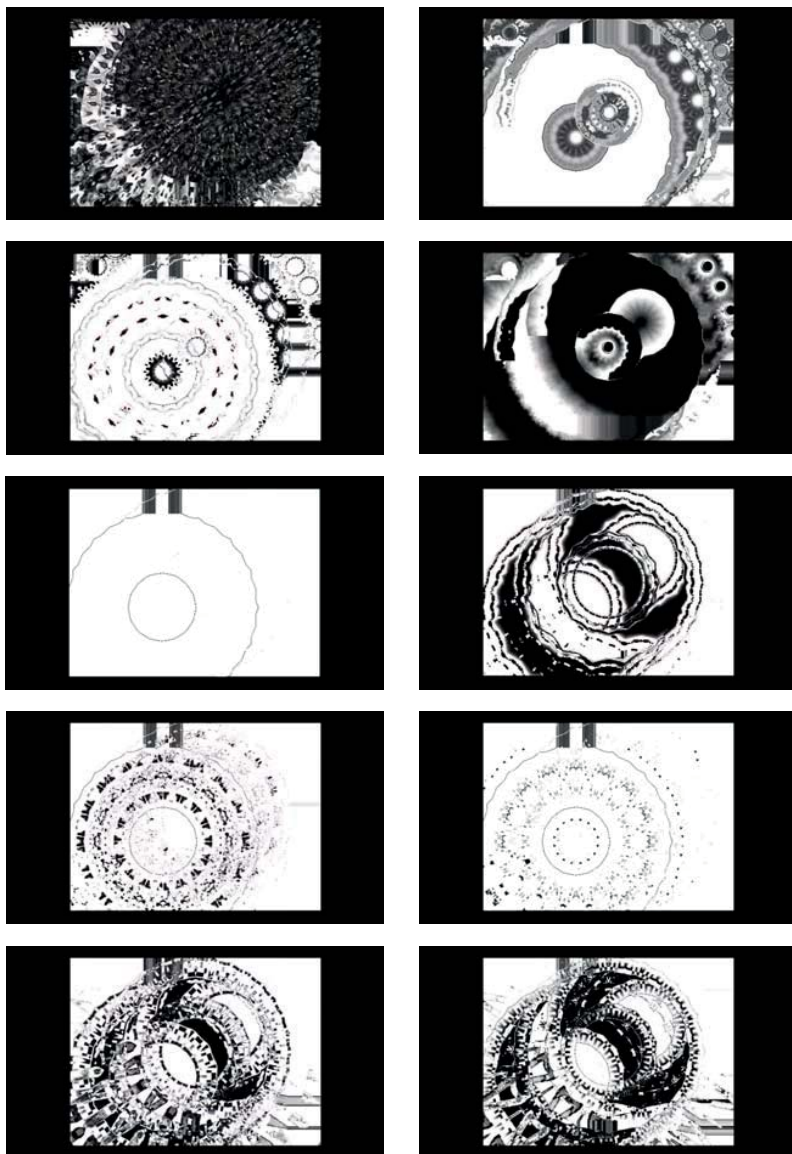


Abb. 153 - 162: Eine Serie von 10 Screenshots einer eigenen Videoaufnahme, manipuliert mit den Effekten des Programms GLSLVideoEffects.

Audiovisueller Automat

Der audiovisuelle Automat ist eines meiner ersten Experimente mit »Pure Data«. Er besteht aus zwei programmierbaren Tonsequenzen mit einer Länge von acht Tönen. Die Töne der einen Sequenz sind mit monochromen Fotos (Abb. 1) verknüpft und die Töne der anderen Sequenz mit Farben. Die Farben und Fotos werden den Tönen entsprechend arrangiert und überblendet.

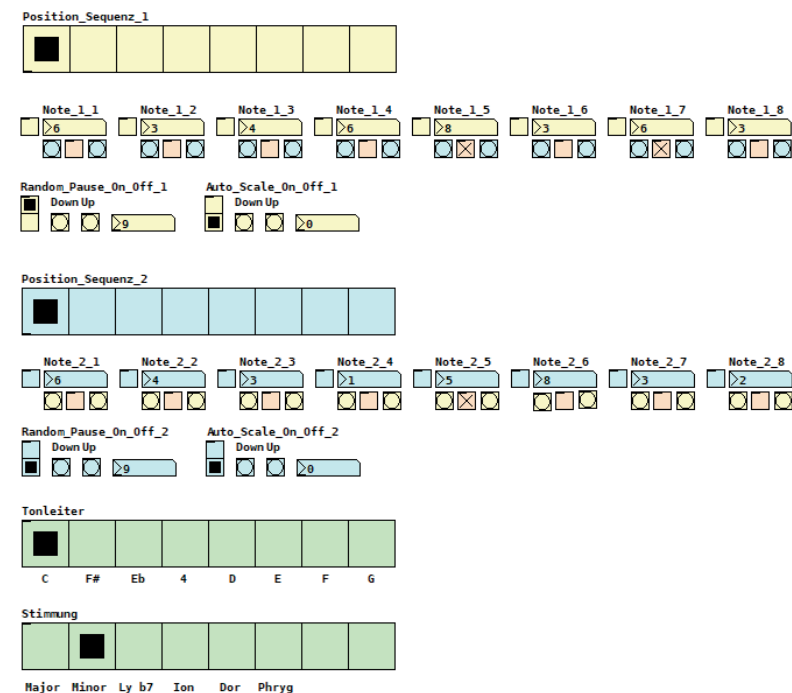


Abb. 163: Bedienoberfläche des audiovisuellen Automaten.

Melodie-Generator

Der Melodie-Generator generiert aus einer Sinuswelle mit Obertönen einen Melodieverlauf mit den Parametern Tonhöhe, Lautstärke und Tonlänge. Wie auch bei anderen meiner Experimente werden die erzeugten Tonwerte nach frei wählbaren Tonarten skaliert. Die Sinuswellen lassen sich außerdem automatisiert in der Zeit bewegen, sodass auch komplexere Melodieverläufe generiert werden können. Ein großer Teil der Arbeit ist in die Programmierung der grafischen Bedienoberfläche geflossen, da wir einen Multi-Slider haben wollten, mit dem man Parameter darstellen, aber auch mit der Maus einzeichnen kann.

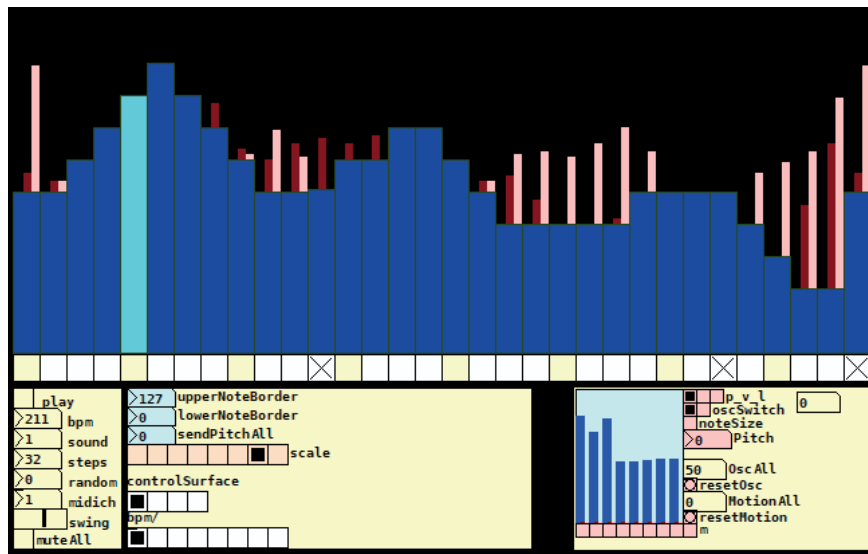


Abb. 164: Bedienoberfläche des Melodie-Generators.

Lissajous-Sequencer

Der Lissajous-Sequencer generiert duofone Melodieverläufe anhand von Lissajous-Figuren. Dabei repräsentiert die Position auf der horizontalen Achse der Darstellung die eine Tonfolge und die Position auf der vertikalen Achse die andere Tonfolge. Wie bei dem Melodie-Generator lässt sich auch hier der Verlauf der Form in der Zeit ändern.

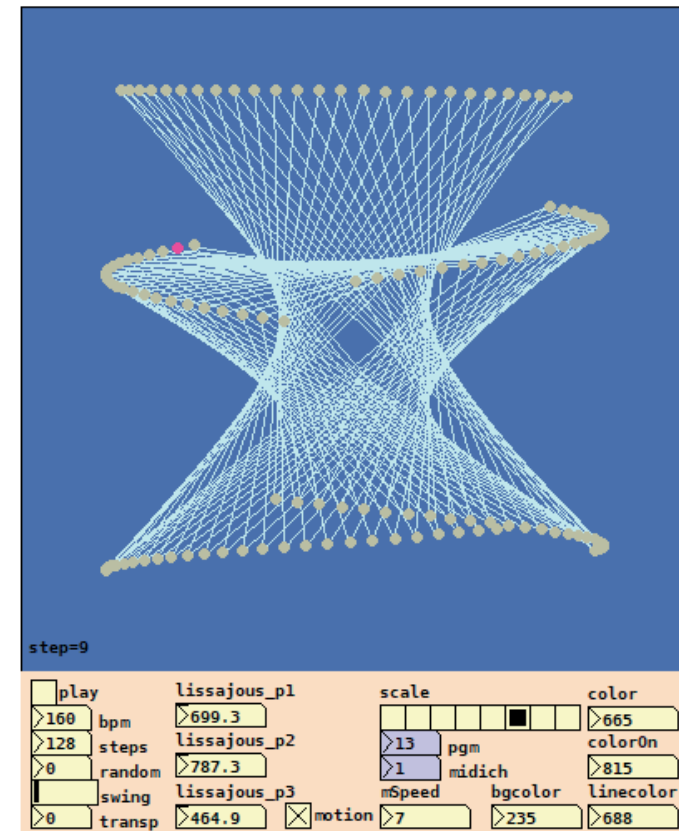


Abb. 165: Bedienoberfläche des Lissajous-Sequencers.

Multi-Slider

Der Multi-Slider ist eine Variation des Melodie-Generators. Anstatt von einer Sinuswelle wird die Melodie durch bedingte Anweisungen generiert: Wenn → dann → sonst. Zum Beispiel: Wenn ein Ton höher X und gerade ist, dann spiele den nächsten Ton um Y tiefer, sonst spiele X + 1. Dabei muss darauf geachtet werden das sich die generierten Töne im hörbaren Bereich befinden, da der Wertebereich nicht automatisch wie bei einer Sinuswelle, deren Werte sich immer zwischen -1 und 1 bewegen, eingegrenzt ist. Inspiriert wurde ich zu dem Experiment durch den Artikel »Algorithmic symphonies from one line of code -- how and why?«⁶⁶ von Viznut.

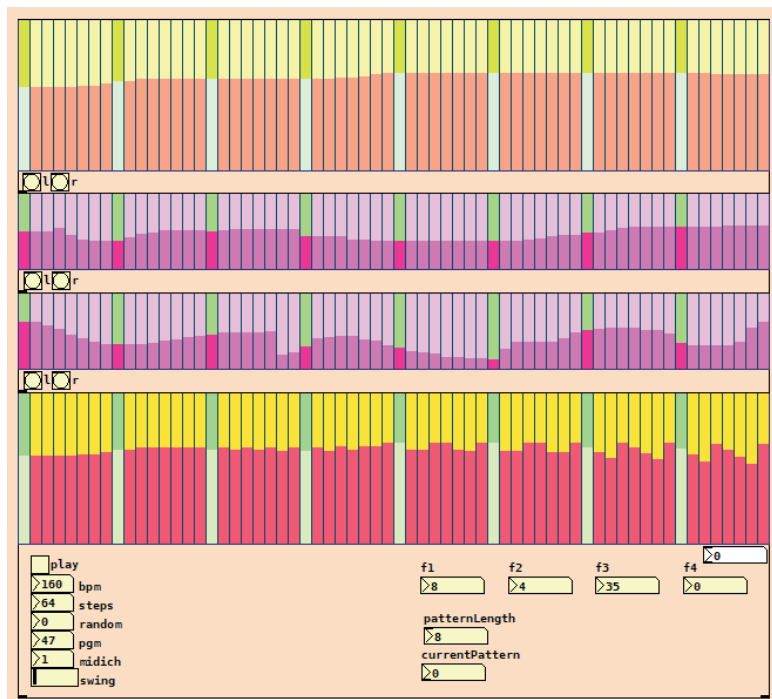


Abb. 166: Bedienoberfläche des Multi-Sliders.

⁶⁶ <http://countercomplex.blogspot.com/2011/10/algorithmic-symphonies-from-one-line-of.html> [22.5.2020].

Slit-Scan

Der Slit-Scan Generator ist ein Versuch, Videoeffekte in der Zeit zu generieren. Dazu wird eine Anzahl bereits abgespielter Videobilder von dem Programm zwischengespeichert und kann dann mit dem aktuellen Bild zusammen verarbeitet werden.

»Slitscan ist eine Aufnahmetechnik in Film und Fotografie. Der Film wird an einem schmalen Streifen oder Schlitz vorbeibewegt und durch diesen belichtet. In der digitalen Bildtechnik kann dies durch entsprechende Bildbearbeitungen ausgeführt werden. Die wohl bekannteste Filmszene mit dieser Technik ist in dem Film 2001: Odyssee im Weltraum zu finden.«⁶⁷

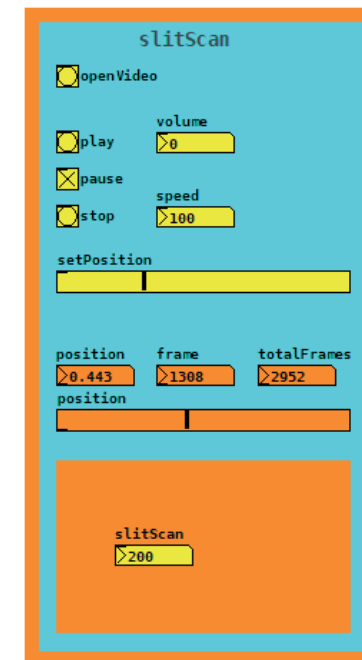


Abb. 167: Bedienoberfläche des Slit-Scan Generators.

⁶⁷ <https://de.wikipedia.org/wiki/Slitscan> [22.5.2020].

Video-Delay

Das Video-Delay baut auf der gleichen Zwischenspeicherung der Videobilder wie der Slit-Scan Effekt auf, nur dass die gespeicherten Bilder anders verarbeitet werden. Inspiriert wurde ich einerseits durch den Kurzfilm »Pas de Deux« aus dem Jahr 1968 von Norman McLaren, der dort mit einer ähnlichen Technik arbeitet, andererseits durch die Idee des Echos aus dem akustischen Bereich.



Abb. 168: Screenshot des Video-Delay Effekts. Eigene Mondaufnahme.

Grid-Sequencer

Der Grid-Sequencer ist ein Versuch, Töne und Bilder in einem Raster anzuordnen. Die Sequenzen werden mit der Maus eingezeichnet. Dabei können Sequenzen manipuliert, gespeichert und miteinander verschaltet werden.

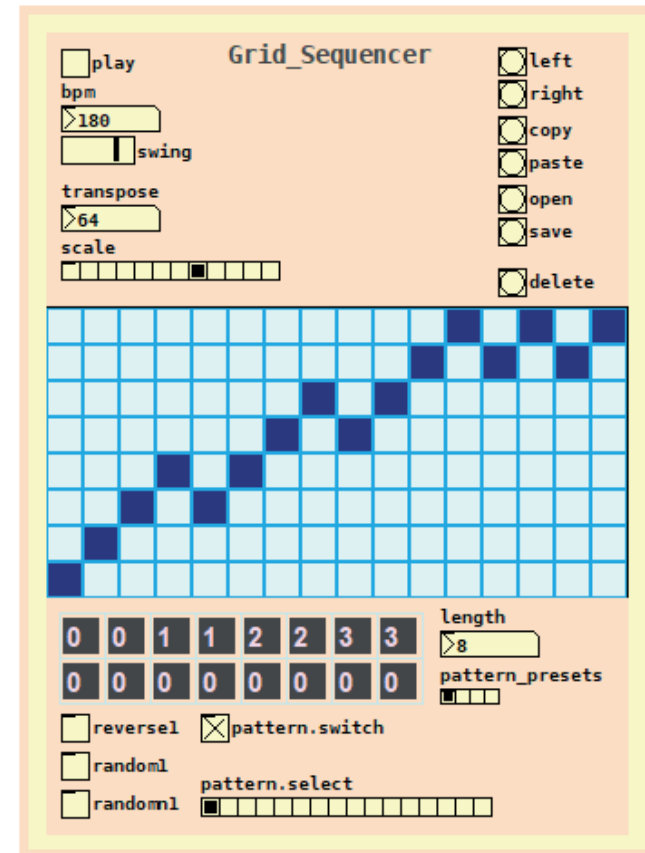


Abb. 169: Bedienoberfläche des Grid-Sequencers.

»AudioVideo«

Als eine Art audiovisuelles Skizzenbuch habe ich seit einiger Zeit einen YouTube-Kanal mit dem Namen »AudioVideo«⁶⁸. Dort sammle ich von Programmen generierte Ergebnisse und Tests. Auf den folgenden Seiten ist, in chronologischer Reihenfolge, eine exemplarische Auswahl zu sehen.

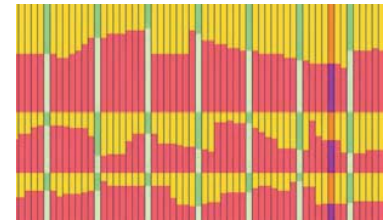


Abb. 170: pure data one line
<https://youtu.be/edjIA40Ux48>



Abb. 171: markov 1
<https://youtu.be/qDohu4cU6dk>



Abb. 172: audiovisueller automat 1
<https://youtu.be/gQhU-Gxyric>

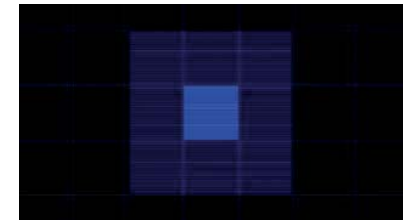


Abb. 173: GemSquare 3
<https://youtu.be/ssAbzF602-o>



Abb. 174: mond
<https://youtu.be/5TG2jKGmDss>

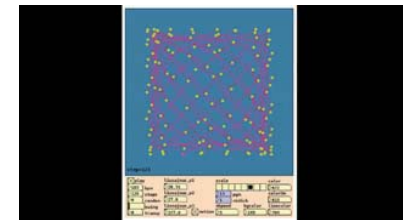


Abb. 175: lissajous sequencer 6
<https://youtu.be/u2RJWqli8mQ>

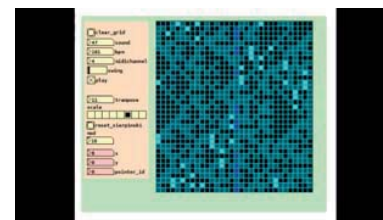


Abb. 176: 2 lsys
<https://youtu.be/2DFx098Seuo>

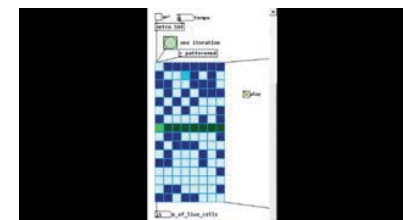


Abb. 177: gol seq 9
<https://youtu.be/UT6zBUJZiKg>

⁶⁸ <https://www.youtube.com/channel/UCVI9jk5rnDcCHARIwAn5yYA/>
 [22.5.2020].

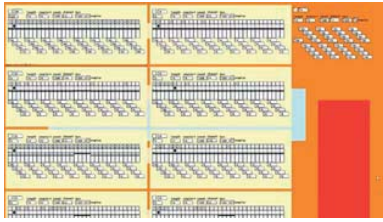


Abb. 178: polyrhythm
<https://youtu.be/3T1KGFAQLic>

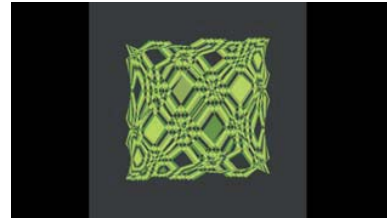


Abb. 179: ofelia test 1
<https://youtu.be/H-Mu5eBlpvA>

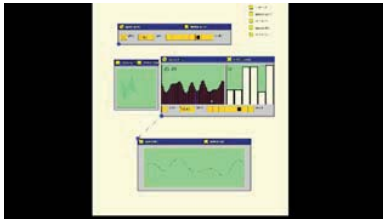


Abb. 180: drag 2
<https://youtu.be/CVCAZ74wjRA>



Abb. 181: ofelia sample player
<https://youtu.be/-3pDFzU04AU>



Abb. 182: AudioMarkov
<https://youtu.be/al8VtaG9zas>



Abb. 183: AudioToMidiMarkov 6
<https://youtu.be/TYP0-ieBHfs>

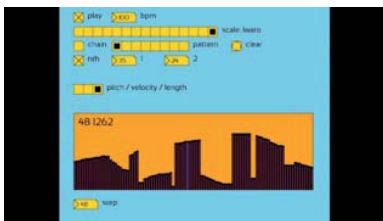


Abb. 184: ofelia multitoggle 2
<https://youtu.be/15lfN3aIRCA>

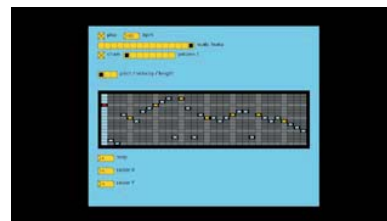


Abb. 185: ofelia multiradio 2
<https://youtu.be/tfLI7l-PpCo>

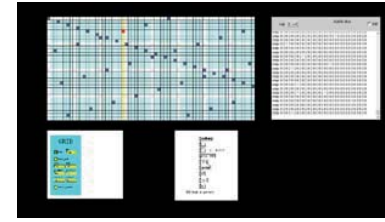


Abb. 186: ofelia 2 gridX 2
https://youtu.be/3_BHHLdkAsA

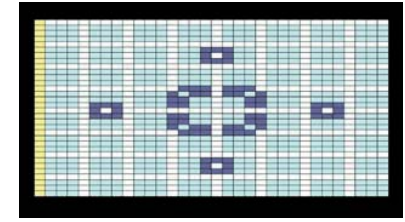


Abb. 187: ofelia 2 gridXconway 123 3
<https://youtu.be/el6gTCZFwKk>

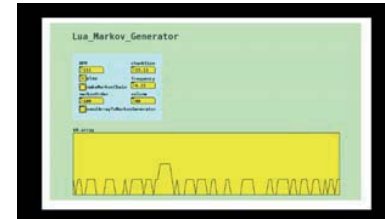


Abb. 188: lua markov generator
<https://youtu.be/6WN1oggnlKQ>



Abb. 189: LuaMIDICVSMarkov 5
<https://youtu.be/GPa-gG-mRSU>

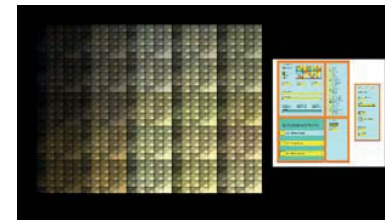


Abb. 190: MusicPlayer 1
<https://youtu.be/u47H4wpqovs>



Abb. 191: videoDelayTest
<https://youtu.be/TV8W8Fy1w60>



Abb. 192: WebAudioMIDIBut-terchurn
<https://youtu.be/kJWH3THSgn8>

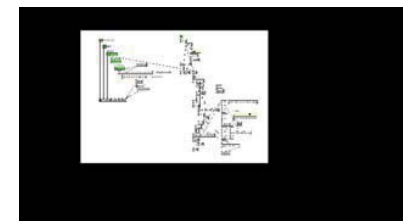


Abb. 193: map
https://youtu.be/Va_K-6Mc_9g

»Handmadeproductions«

Eine weitere Form, meine audiovisuellen Experimente zugänglich zu machen, neben den Aufzeichnungen des YouTube-Kanals und dem Veröffentlichen der Programme, ist meine Internetseite »Handmadeproductions«⁶⁹, auf der ich einzelne meiner Experimente implementiere. Es ist möglich, Parameter der Algorithmen zu ändern und eigenes Material durch sie bearbeiten zu lassen. Davon erhoffe ich mir, anderen den Zugang zu meinen Experimenten zu erleichtern. Viele der Internet-Experimente sind vorerst Skizzen, die ich noch ausbauen möchte. Auch den Montage-Automaten könnte ich eines Tages möglicherweise auf der Internetseite als interaktive Anwendung implementieren. Eine Herausforderung dabei ist, die Bedienoberflächen der Programme dafür neu zu programmieren, da man über die Internetseite nicht auf die »Pure Data« eigenen Bedienelemente zurückgreifen kann. Eine andere Herausforderung ist der Austausch von Steuerdaten (MIDI) und Material (Bild und Ton), mit der ich viel Zeit verbracht habe, und noch nicht am Ende der Entwicklung angekommen bin. So lassen sich zum Beispiel die Parameter einzelner Experimente mit der Technologie Web-MIDI durch MIDI-Controller verändern, ebenso kann man MIDI-Daten ausgeben um damit wieder andere Geräte, zum Beispiel Drumcomputer oder Synthesizer, anzusteuern. Außerdem lassen sich externe Bild- und Tondateien als Material nutzen. Das Einbinden von externem Videomaterial ist (aufgrund der Dateigröße) noch nicht möglich, sollte aber auch umsetzbar sein.

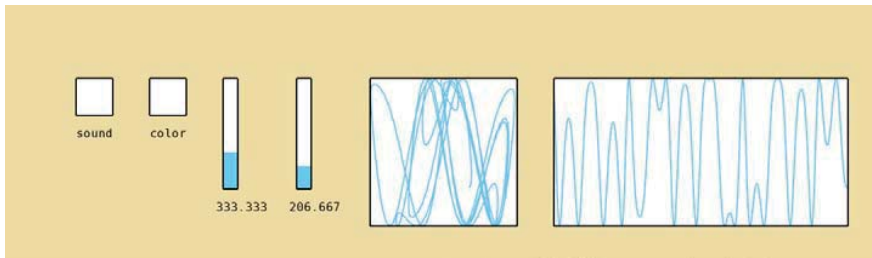


Abb. 194: Mein erster Versuch »Pure Data« Programme mit »Ofelia« und »Emscripten« in eine Internetseite zu integrieren.
<https://puredata.handmadeproductions.de>

⁶⁹ <https://index.handmadeproductions.de> [22.5.2020].

Eine Auswahl meiner Internet-Experimente:

gles: Eine reduzierte Version des GLESThreeEffects Programms. Während ein Video in einer Schleife läuft, können mit den Tasten x und y die Videoeffekte gewechselt werden.
<https://gles.handmadeproductions.de>

gles2: Auch hier ein Video in einer Schleife. Anstatt die Effekte zu wechseln, kann mit den Parametern eines Kaleidoskop Effekts gespielt werden. Gleichzeitig ist es ein Versuch, Klänge auf einer Internetseite zu synthetisieren und als Sequenzen anzuordnen.
<https://gles2.handmadeproductions.de>

pdmidi: Ein Experiment mit Web-MIDI. Mit einem »Nano-Kontrol« MIDI-Controller lassen sich die Farben eines Quadrats verändern, während von der Internetseite eine veränderbare MIDI-Sequenz erzeugt wird, mit der sich externe MIDI-Instrumente ansteuern lassen.
<https://pdmidi.handmadeproductions.de>

slitscan: Das Slit-Scan Programm. Eine Frage für mich war, ob sich die für den Effekt nötige Zwischenspeicherung der Bilder auch für das Internet umsetzen lässt.
<https://slitscan.handmadeproductions.de>

webmidi: Ein Versuch, Visualisierungen im Internet mit Klängen durch Frequenz und Lautstärke zu verändern. Töne werden durch externe MIDI-Instrumente angesteuert und von der Seite erzeugt. Die Visualisierungen werden von »Butterchurn«⁷⁰ erzeugt, einer WebGL Implementation des »Milkdrop« Plug-Ins für den »Winamp« Media-Player.
<https://webmidi.handmadeproductions.de>

⁷⁰ <https://github.com/jberg/butterchurn> [22.5.2020].

Das Experiment pdspectrum (Abb. 195) ist keine Adaption eines bestehenden »Pure Data« Programms, sondern ich habe es speziell für die Anwendung im Internet programmiert, um mich mit den notwendigen Technologien auseinanderzusetzen. Dabei ging es mir um das Einbinden und Manipulieren externer Bild- und Tondateien. Das Bild wird von einem Kaleidoskop Effekt verändert und es lassen sich Parameter wie die Anzahl der Seiten oder der Winkel einstellen. Gleichzeitig wird das Spektrum des Klangs analysiert und in die visuelle Darstellung eingebunden. Der Klang wiederum lässt sich filtern (die hohen Klanganteile werden reduziert), mit einem Echo belegen, und in der Geschwindigkeit verändern. Den Einsatz der Effekte würde ich als technologische Forschung bezeichnen, sie haben im Rahmen dieses Experiments keinen unmittelbaren künstlerischen Wert. Vielmehr dienen sie mir als Grundlage für zukünftige künstlerische Arbeiten.

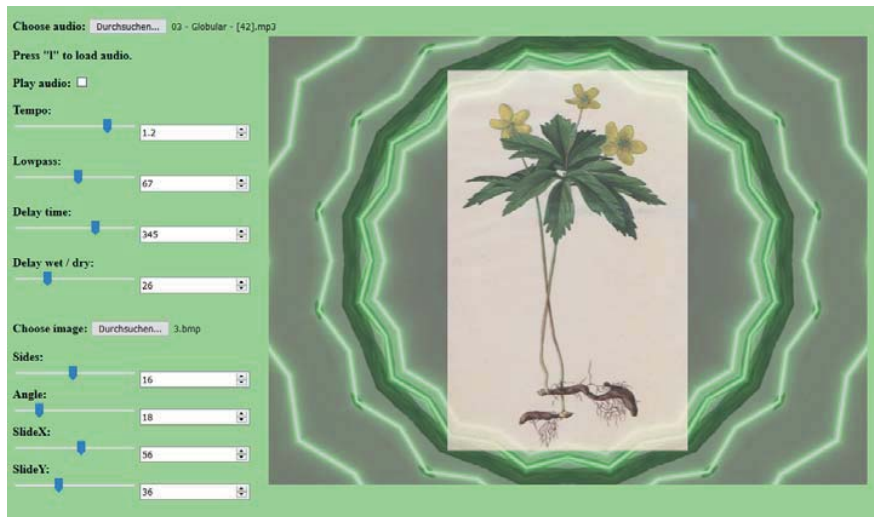


Abb. 195: Ein Screenshot der Internetseite <https://pdspectrum.handmade-productions.de>. Erste Versuche von mir Bild- und Tondateien in Echtzeit im Internet zu manipulieren. Programmiert ist die Seite mit »Pure Data«, »Ofelia« und »Emscripten«. Die Bedienoberfläche ist mit »Java Script« erstellt.

Über künstlerische Forschung

»Aber mein Argument ist, das man nur durch präzise methodische Analyse genau dieser Disziplinierung [der künstlerischen Forschung] möglicherweise auch entgehen kann.«⁷¹

Anke Haarmann spricht damit einen interessanten Ansatz an. Vielleicht ist es gerade die Vielfältigkeit, die entsteht, wenn man sich mit den Begriffen beschäftigt. Um die künstlerische Forschung, einen Begriff, den es seit Anfang der 1990er Jahre gibt, besser zu verstehen, habe ich angefangen mich mit den Begriffen Forschung, Kunst, Wissenschaft und Erkenntnis zu beschäftigen. Begriffe auf die sich die künstlerische Forschung bezieht und an denen sie sich reibt. Ich habe dabei viel gelernt und einige interessante Theorien gelesen, kann aber vielleicht gerade deshalb nun die künstlerische Forschung weniger als eine Disziplin, sondern mehr als einen (wichtigen) Diskurs sehen.

Weiterhin ein Zitat aus dem Jahr 1973 über den Begriff der Ästhetik, das sich sehr treffend auch auf die künstlerische Forschung übertragen lässt:

»Statt voreiliger Einigkeit und Übereinstimmung provoziert die Frage [, was ist Ästhetik] teils den Widerspruch, teils die Skepsis, teils das Bedürfnis nach schärferen Abgrenzungskriterien und Definitionen, um schließlich in einer *concordia discors* einzumünden.«⁷²

Aus demselben Jahr ein Zitat von Ernst Bruche, der auch in der wissenschaftlichen Forschung ein Problem der Disziplinierung, und gleichzeitig eine Nähe zu freiem künstlerischem Schaffen sieht:

⁷¹ Anke Haarmann: 10. TDK des IKF: »Künstlerische Forschung: Kunst als Wissensproduktion«. Potsdam-Babelsberg: Filmuniversität Babelsberg KONRAD WOLF, 2014. <https://www.youtube.com/watch?v=KjwLfMeSjdy> (Position 0:40) [22.5.2020].

⁷² Anastasios Giannaras: Ästhetik heute. München: Francke Verlag, 1973. S. 7.

»Heute, wo wir Institute, Vereinigungen, Gesellschaften, Ministerien der Forschung besitzen, ist Forschung oft nicht mehr Berufung, sondern Beruf geworden, und bei den in dieser Sparte Tätigen besteht die Gefahr, zu vergessen, daß Forschung mit dem freien künstlerischen Schaffen verwandt ist. Forschung wird heute leicht als Untersuchung nach gelernten Regeln aufgefaßt oder als Spielerei fortgeschoben, wenn die eingefahrenen Geleise nicht benutzt werden. War Eugen Goldstein in dieser Sicht ein Forscher, als er sich immer wieder andere Glaskolben für seine Entladungen herstellte, mit denen er eines Tages die Kanalstrahlen fand? Hat man nicht fast Hemmungen, Röntgen bei seinem Spiel mit den Kathodenstrahlen als Forscher anzusehen? Kein Wunder, wenn da ein Ausschuß unserer Zeit etwa erklärt, das Studium der Wechselwirkung zwischen Strahlen und schmelzbarer Materie und die fotografische Fixierung dieser Beobachtungen sei keine Forschung, sondern die optische Beschreibung rein phänomenologischer Faktoren und daher nicht förderungswürdig. Wir meinen: Forschung ist nicht nur Untersuchung nach konventionellen Regeln, Forschung ist mehr.«⁷³

Als ich gegen Anfang meines Studiums die ersten Male von dem Begriff der künstlerischen Forschung gehört habe, war ich von ihm, und was ich mir darunter vorgestellt habe, sehr fasziniert. Ich muss das so naiv sagen. Ich habe ihn nicht genauer hinterfragt, hatte aber das Gefühl, dass einige der Künstler, mit denen ich mich beschäftigt habe, wie Christopher Dell oder DJ Spooky, durch Ihre selbstreflexive Arbeit und die Verbindung von Theorie und Praxis, genau das tun. Christopher Dell, indem er als Musiker eine Theorie der Improvisation entwickelt und DJ Spooky, indem er als DJ eine Theorie des Remix entwickelt. Jeweils theoretische Überlegungen, die sich aus den praktischen Arbeiten ergeben und in Wechselwirkung mit ihnen stehen. Auch glaubte ich, dass ein Teil meiner Arbeiten, wie zum Beispiel der Montage-Automat, dem Bereich

73 Ernst Bruche: Was ist eigentlich Forschung? In: Physikalische Blätter, Volume 29, Issue 3. Mosbach: Physik Verlag, 1973. S. 141. <https://onlinelibrary.wiley.com/doi/pdf/10.1002/phbl.19730290307> [22.5.2020].

der künstlerischen Forschung zuzuordnen ist. Im Fall des Montage-Automaten ist für mich der Prozess der Entstehung von großer Bedeutung. Das liegt auch daran, dass die Fragen, die Überlegungen und die Gespräche, die sich aus dem Projekt ergeben haben, aber auch die Umwege, die es genommen hat, mich mehr bereichert haben, als die generierten Untertitelfolgen des Automaten. Wobei auch diese wieder Ausgangspunkt weiterer Überlegungen und künstlerischer Arbeit sein können. Je mehr ich mich mit dem Begriff beschäftige, umso eher komme ich zu dem Schluss: Ich weiß nicht, was künstlerische Forschung ist, und kann demzufolge auch nicht beantworten, ob meine Arbeiten dem Bereich zuzuordnen sind. Dass was Christopher Dell und DJ Spooky machen, finde ich sehr spannend, weil sie das Methodische in Ihrer Arbeit untersuchen. Aber ist das reflektierte Sichtbarmachen von künstlerischen Prozessen wirklich das, was die künstlerische Forschung ausmacht? Auch die These, dass sich in der künstlerischen Forschung die Methoden aus der Arbeit ergeben, würde für mich nicht ausreichen, da das auch auf die Kunst selbst zutrifft. Auch meine Überlegungen bezüglich Improvisation und Algorithmus sind nicht spezifisch für die künstlerische Forschung, sondern lassen sich auf jede kreative Arbeit beziehen.

Anke Haarmann fasst einige Fragestellungen und Probleme in Bezug auf die künstlerische Forschung zusammen:

»Nimmt man den Begriff allerdings philosophisch ernst, so ist man sehr schnell mit einer Reihe schwieriger Probleme konfrontiert. Das Problem etwa, was vom Anspruch der Wissenschaftlichkeit bleibt, wenn die erfahrungshaltige Materialität der Kunst zur symbolischen Kommunikation und Forschung avanciert. Umgekehrt wiederum – führen die neuen proklamierten Ansprüche der Wissenschaftlichkeit an die Kunst mitunter zu bemerkenswert unreflektierten methodischen Korsetts an den neu geschaffenen ›artistic research Instituten‹ der Kunsthochschulen. Methodische Korsette, derer sich jede andere Forschungsdisziplin aus guten Gründen erwehren würde. Da wird in Regelwerken festgeschrieben, was geleistet werden müsse, um als künstlerische Forschung anerkannt zu werden, Regelwerke die im wahrsten Sinne des Wortes einer Disziplinierung

gleichkommt, ohne überhaupt geklärt zu haben, ob es sich beim geforderten ›artistic research‹ um Forschung mit der Kunst, in der Kunst oder mit den originären Mitteln künstlerischer Praxis handeln solle und um welche Praxis es dabei geht.«⁷⁴

Zwischenzeitlich habe ich mich von dem Begriff der künstlerischen Forschung auf eine merkwürdige Weise eingeengt gefühlt, und musste mich erst wieder davon befreien. Diese Einengung ist Teil des Diskurses um die künstlerische Forschung und bedingt sich gleichzeitig aus dem nachvollziehbaren Versuch diese definierbar zu machen. Zum jetzigen Zeitpunkt sehe ich das alles wieder sehr entspannt, auch weil ich nicht mehr das Gefühl habe, mit dem, was ich mache, der künstlerischen Forschung entsprechen zu wollen. Und ich hoffe, ich mache es mir nicht zu einfach damit zu behaupten: Ob ich künstlerische forsche, das müssen andere beurteilen. Oder anders gesagt, wenn künstlerische Forschung einer bestimmten Kategorie entspricht, dann möchte ich mich nicht in dem, was ich künstlerisch tue, danach richten müssen. Gleichzeitig finde ich den Diskurs um die künstlerische Forschung sehr wertvoll, auch weil er (alte) Fragen nach Erkenntnisgewinn neu aufwirft.

Vielleicht mache ich auch forschende Kunst, das hört sich irgendwie selbstverständlicher an. Möglicherweise, weil der Begriff weniger gebraucht wird, oder weil die Betonung bei dem Begriff der Kunst liegt. Nehmen wir an, ich montiere einen Dokumentarfilm. Auch da gibt es in der Regel eine Fragestellung, der man nachgeht und aus der sich im Zusammenspiel mit den Beteiligten und dem Material eine Erkenntnis ergibt. Nur das die Erkenntnis nicht der Antwort auf die ursprüngliche Fragestellung entsprechen muss. Das ist vielleicht etwas, das mir, abhängig von dessen Definition, in der künstlerischen Forschung fehlt, die Erkenntnis muss nicht der Antwort auf die ursprüngliche Fragestellung entsprechen. Denn durch den Prozess

⁷⁴ Anke Haarmann: Künstlerische Praxis als methodische Forschung. Düsseldorf: 2011. S. 7. <http://www.dgae.de/wp-content/uploads/2011/09/Haarmann.pdf> [22.5.2020].

selber können sich auch die gestellten Fragen verschieben, weil in dem Material vielleicht noch etwas anderes steckt und sich daraus etwas Unvermutetes ergeben kann. Diese Offenheit ist auch wieder das Improvisatorische und das Spielerische in der Montage oder in der Kunst, das die Methode, und damit das Kunstwerk aus dem Zusammenwirken von Material, Beteiligten und Arbeit (dem entstehendem Kunstwerk) entstehen lässt. Und dadurch, dass sich die Methode jeweils aus dem Prozess und nicht schon aus der Fragestellung ergibt, entsteht auch das Forschende in der Kunst.

»Der Begriff der künstlerischen Forschung, wie er im Kontext der Künste seit einigen Jahren verwendet wird, verweist auf eine Tendenz in der zeitgenössischen Kunst, in der die künstlerische Praxis nicht nur vom abgeschlossenen Werk her begriffen wird – werkästhetisch, sondern von den Praktiken und Strategien der künstlerischen Produktion her – produktionsästhetisch. Der Prozess der Genese einer künstlerischen Arbeit rückt in das Zentrum der Aufmerksamkeit und Künstler nehmen diesen Prozess als fundamentale Phase der Entwicklung einer Arbeit wahr, die von Erkenntnisinteresse geleitet ist und einer systematischen Form-Inhalt-Korrelation folgt.«⁷⁵

Die von Anke Haarmann beschriebene Verschiebung von der Werkästhetik zur Produktionsästhetik bezieht sich auf die bildenden Künste, und ist für mich neu, aber gleichzeitig selbstverständlich. Vielleicht, da man in der Filmmontage das Resultat eher als Film denn als Kunstwerk, also Werk, bezeichnet, und es im Rahmen des Studiums, aber auch in der Filmmontage im Allgemeinen, selbstverständlich ist, über den Prozess und die Zwischenergebnisse zu sprechen? Da ich jetzt aber angefangen habe, den Begriff des Kunstwerks in Bezug auf meine Programmierarbeit, aber auch in Bezug auf die Filmmontage zu nutzen, sollte auch ich mir die Frage nach dem Werkbegriff stellen. Ich habe den Begriff des Kunstwerks nicht wegen des Werkbegriffs gewählt,

⁷⁵ Anke Haarmann: Künstlerische Praxis als methodische Forschung. Düsseldorf: 2011. S. 1. <http://www.dgae.de/wp-content/uploads/2011/09/Haarmann.pdf> [22.5.2020].

sondern eher weil ich glaube, künstlerisch zu arbeiten. Ich mag die Begriffe Position und Arbeit, die laut Anke Haarmann in der bildenden Kunst inzwischen den Werkbegriff ersetzen und manchmal möchte ich den Prozess auch transparent und selbstreflexiv gestalten. Aber ich finde, der Prozess darf auch für die Rezipienten im Verborgenen bleiben. Wenn ich mich nur noch mit Strukturen und Prozessen beschäftige, kann ich mich vielleicht irgendwann nicht mehr auf den Inhalt einlassen (es sei denn, der Inhalt ist die Struktur). Ich musste mich sowohl als Rezipient wie auch als Künstler schon davon lösen, Kunst zu analytisch zu betrachten, um sie wieder auf mich wirken zu lassen. Die Struktur ist ja trotzdem gemacht und vorhanden. Sowohl der Prozess als auch das Ergebnis sind essenzieller Teil der künstlerischen Arbeit und es ist ebenso eine künstlerische Entscheidung, inwieweit man den Prozess in dem Ergebnis sichtbar machen möchte. Eine andere Frage ist, ob die Reflexionsebene des Herstellungsprozesses in dem künstlerischen Ergebnis nicht irgendwann auch erschöpft ist. Und zwar dann, wenn der Herstellungsprozess, das Gemachte, auch vom Rezipienten als selbstverständlicher Teil der Arbeit gesehen wird. Solange sich aber die künstlerische Methode von Kunstwerk zu Kunstwerk ändert, sich immer wieder erneuert, solange wird es auch neue Strukturen in der Kunst geben, die es zu Untersuchen lohnt.

Ein Definitionsvorschlag der künstlerischen Forschung von Anke Haarmann:

»Ich möchte nun vorschlagen, dass die methodischen Ansprüche an die Kunst etwa mit den methodischen Ansprüchen an die Philosophie vergleichbar sein könnten, dass nämlich beide keinem vorgesetzten Regelkanon und Methoden-katalog folgen, sondern die jeweilige Methodik aus der forschenden Fragestellung und Praxis selber entwickeln, dabei aber mit dem Anspruch der höchsten Konsequenz. Philosophische Schriften entwickeln ihre argumentativen Standards und die Systematik ihrer Schlussfolgerungen an den philosophischen Fragen, die sie zu beantworten beanspruchen. Da hilft es wenig, nach der allgemeinen Methode der Philosophie als Wissenschaft zu fragen und doch

ist alles Philosophische hoch methodisch. [...] Ich habe also versucht zu skizzieren, dass erstens die kunsthistorische Entwicklung mit dem Aufkommen der konzeptuellen Kunst eine prozessuale und kontextuelle Ebene in die Kunst eingetragen hat, die als Weg der künstlerischen Praxis in den Arbeiten selber sichtbar wird. Dies wiederum legt zweitens nahe, unter produktionsästhetischen Gesichtspunkten über die Kunst als Praxis nachzudenken bzw. die Kunst als Praxis zu bestimmen. Der Praxischarakter des Künstlerischen wiederum erlaubt es drittens überhaupt über die Kunst als Forschung insofern nachzudenken, als das Forschen eine Praxis ist. Diese Praxis der Forschung als methodos – als Weg des Wissens – in der künstlerischen Praxis auszuweisen kann aber nur bedeuten, so die These, sie induktiv aus konkreten künstlerischen Praktiken herausarbeiten, denn die Künste sind wie die Philosophie einer konsequenten Form-Inhalt-Relation verpflichtet, das heißt sie setzen sich die Systematik und Form ihrer Methode entsprechend den Inhalten, um die es ihnen geht.«⁷⁶

Den Vergleich von Philosophie und Kunst finde ich sehr spannend, da sich in beiden Disziplinen die jeweiligen Methoden aus der Fragestellung und dem Prozess ergeben. Vielleicht ist die Philosophie in Ihrer Logik doch etwas exakter, während im Künstlerischen das Spielerische von größerer Bedeutung ist? Jetzt kann man natürlich fragen: Kann Philosophie nicht auch spielerisch und Kunst philosophisch sein? Wenn ich Anke Haarmann richtig verstanden habe, dann versteht sie die Kunst an sich als forschende Disziplin, also als künstlerische Forschung. Nur das der erkenntnistheoretische Diskurs in der Kunst und in der Wissenschaft das Forschende in der Kunst erst wieder sichtbar gemacht hat.

Zur Gegenüberstellung ein anderer Definitionsvorschlag der künstlerischen Forschung von Corinna Carduff:

⁷⁶ Anke Haarmann: Künstlerische Praxis als methodische Forschung. Düsseldorf: 2011. S. 7. <http://www.dgae.de/wp-content/uploads/2011/09/Haarmann.pdf> [22.5.2020].

»Künstlerische Forschung studiert auch nicht künstlerisch vermittelte Wissensbereiche und macht deren Befragung deutlich erkennbar. Nicht künstlerisch heißt dabei zunächst, dass jeder nur denkbare Wissensbereich möglich ist (Physik, Philosophie, Gentechnologie, Rhetorik der Gesetzgebung etc.). »Nicht künstlerisch« schließt aber nicht aus, dass der befragte Wissensbereich mit der jeweiligen Kunst verwandt sein kann: Ein Fotograf etwa kann theoretische Texte zur Fotografie studieren oder ein Komponist musikwissenschaftliche Abhandlungen über den Rhythmus, wobei aber diese Auseinandersetzung im künstlerischen Werk selbst explizit kenntlich gemacht wird.

Künstlerische Forschung setzt für die relevant gehaltenen Wissensdaten, die dieser Befragung entstammen, künstlerisch in Szene.

Künstlerische Forschung folgt einer klaren Fragestellung, die zum einen im vorliegenden Kunstwerk selbst deutlich erkennbar ist und zum anderen zusätzlich in Paratexten explizit angeführt wird. Eine sprachliche Präsentation der Fragestellung ist im Minimalfall durch einen Paratext gewährleistet.

Paratexte sind etwa Informationsblättern, die in Museen aufliegen; Programmhefte von Konzerten und Theateraufführungen; Klappentexte von Buchen etc. Paratexte geben beispielsweise Auskunft über die Herleitung der Fragestellung oder über deren Kontextualisierung im Umfeld des vorliegenden Kunstwerks.

Mit diesen Parametern ist ein Verständnis von Künstlerischer Forschung gewährleistet, wonach diese sich sowohl von allgemeiner Kunst als auch von wissenschaftlicher Forschung unterscheidet: Es handelt sich um einen dezierten Umgang mit einem klar benennbaren und bestimmten Wissensbestandteil, der künstlerisch reflektiert wird.

Zwar kann eine solche Bestimmung nicht darüber hinwegtäuschen, dass es trotzdem in manchen konkreten Fällen unklar bleiben dürfte, ob nun Künstlerische Forschung vorliegt oder nicht. Das Merkmalset dient jedoch in erster

Linie dazu, Künstlerische Forschung zu charakterisieren und dadurch die Felder sichtbar zu machen, die sie herstellt und in welchen sie ihre (Er-)Kenntnisse und Wissensproduktion vollzieht.«⁷⁷

Corinna Carduff schlägt vor, dass sich künstlerische Forschung durch die künstlerische Bearbeitung auch nicht künstlerischer Themen auszeichnet. Wichtig ist dabei ein Text, der die Fragestellung formuliert. Aber wird damit nicht jede künstlerische Arbeit durch einen Text, der die Fragestellung formuliert, zur künstlerischen Forschung? Auch sie selbst merkt an, dass nicht jede Arbeit, auf die Ihre Definition zutrifft, auch als künstlerische Forschung zu bezeichnen ist.

Die beiden teilweise konträren Anschauungen oder Positionen von Anke Haarmann und Corinna Carduff zeigen exemplarisch das breite Spektrum von dem, was als künstlerische Forschung verstanden werden kann. Und vielleicht ist genau diese Diversität an Vorstellungen auch das, was die künstlerische Forschung ausmacht. Eben weil die Diversität in den Positionen eine Offenheit und einen Austausch erlaubt, der die Reflexion über die Kunst, aber auch über die Wissenschaft, also über die Forschung im Allgemeinen, bereichert. Solange diese Offenheit bewahrt wird, dienen die Institute der künstlerischen Forschung auch dazu an den Kunsthochschulen, trotz einer tendenziellen Versuchung im Zuge der Bologna-Reform, einen wichtigen künstlerische Freiraum zu erhalten.

Auch ich als Künstler werde mir die Frage nach der künstlerischen Forschung weiterhin stellen und den Diskurs verfolgen. Aber gleichzeitig meine Arbeit nicht darüber definieren, was sie künstlerisch forscht. Nicht weil ich ausschließe, dass es sich bei dem, was ich mache um künstlerische Forschung handeln könnte, sondern weil der Begriff mir Grenzen setzt, die ich so in der Kunst nicht habe.

⁷⁷ Corinna Carduff: Literatur und künstlerische Forschung. In: Zürcher Jahrbuch der Künste 2009, Band 6 - Kunst und künstlerische Forschung. Zürich: Verlag Scheidegger & Spiess AG, 2010. S. 115 - 116.

Abschließende Gedanken

Gegen Ende meines Studiums und meiner Masterarbeit möchte ich versuchen, ein paar abschließende Gedanken zu finden. Was habe ich durch meine Programmierexperimente herausfinden können? Was habe ich künstlerisch gelernt? Und was habe ich speziell über die Filmmontage gelernt?

Improvisation und Algorithmus bedingen sich gegenseitig. Dabei hängt es von dem Verhältnis der beiden Begriffe ab, wie frei das Kunstwerk entstehen kann. Da das Programm, die Notation und die Struktur Formen des Algorithmus sind, gilt das ebenso für sie. Das Programm kann notiert sein, kann aber auch in Form eines Computerprogramms oder menschlicher Erfahrung vorliegen. Oder eben als sich selbst programmierendes Kunstwerk.

Ich möchte das anhand von zwei Musikern der freien, improvisierten Musik veranschaulichen: Die entstehende Musik bezeichne ich als Kunstwerk. Angenommen beide haben viel, aber unterschiedliche musikalische Erfahrung, und vorher noch nicht zusammengespield. Gleichzeitig sind sie an einem Ort zu einer Zeit, aber haben unterschiedlichen Geschichten, und damit Strukturen auf die sie zurückgreifen können. Aus dem ersten Ton entsteht, ohne zu determinieren, dass Grundprogramm des Kunstwerks, auf dem alles weitere aufbaut. Durch den andauernden Dialog zwischen Musikern und Kunstwerk verdichtet sich das Programm, die Möglichkeiten werden immer weiter eingeschränkt, bis es abgeschlossen ist (auch ein offenes Ende ist möglich). Das Beispiel der freien Musik lässt sich auch auf die Filmmontage übertragen, mit dem Unterschied, dass sich in der Regel aus dem Rohmaterial und der Idee zu dem Film schon eine Grundstruktur, ein Ensemble an Möglichkeiten, ergibt. In der Filmmontage bewegt man sich in der Freiheit der Improvisation vielleicht zwischen einem Musiker der freien Musik, der etwas aus dem *Nichts* schafft, und einem Orchestermusiker der einer vorgegebenen Notation folgt.

Mir haben die Überlegungen und praktischen Experimente im Rahmen meiner Masterarbeit geholfen, ein besseres Verständnis von dem Verhältnis zwischen Freiheit und Struk-

tur oder Improvisation und Algorithmus in der Kunst zu bekommen.

Bei meinen theoretischen Überlegungen hat mich Niklas Luhmanns Idee des sich selbst programmierenden Kunstwerks inspiriert. Ich habe versucht, das sich selbst programmierende Kunstwerk, die Improvisation und das Algorithmische zusammen zu denken, um dadurch den Vorgang meiner Arbeitsweise zu reflektieren und besser zu verstehen.

So wie ich erst durch meine Überlegungen zum Programmieren gekommen bin, bin ich erst durch meine Programmierarbeit auf Niklas Luhmanns Idee des sich selbst programmierenden Kunstwerks gestoßen. So zeigt sich in meiner Arbeit eine Verflechtung und Wechselwirkung von Theorie und Praxis.

Der Montage-Automat ist das einzige meiner Programmierexperimente, das der algorithmischen Komposition in der Filmmontage explizit zuzuordnen ist. Ich bin froh, dass er nicht wie ursprünglich geplant, eine theoretische Überlegung geblieben ist, sondern von mir auch umgesetzt wurde. Ich finde es spannend, spielerisch Regeln zu formulieren, und mich von dem generierten Ergebnis überraschen zu lassen. Lösungen zu finden, um Strukturen algorithmisch abzubilden. Ich habe vor, den Montage-Automaten wieder aufzugreifen. Vielleicht die aktuelle Version weiterzuentwickeln oder es mit einem neuen Ansatz zu versuchen. Außerdem will ich generierte Folgen künstlerisch nutzen, sie als Ausgangsmaterial für weitere Experimente sehen.

Es fällt mir schwer, zu sagen, dass ich durch den Montage-Automaten etwas über die klassische Filmmontage gelernt habe, da es doch ein ganz anderer Ansatz ist. Aber sicher habe ich etwas über die algorithmische Filmmontage und die algorithmische Komposition gelernt und mir somit ein neues künstlerisches Feld erschlossen, mit dem ich mich auch weiterhin beschäftigen will.

Die meisten anderen Programmierexperimente sind eher dem musikalischen Bereich zuzuordnen. Ich sehe da durchaus eine nahe Verwandtschaft zur algorithmischen Filmmontage,

die grundsätzliche Methode (das Anordnen nach mathematischen Formeln) ähnelt sich sehr. Der wesentliche Unterschied zum Montage-Automaten liegt im nicht-begrifflichen, abstrakten Charakter der Musik.

Über die praktischen Experimente habe ich mir gleichzeitig Programmiergrundlagen geschaffen, die für meine künstlerische Arbeit nützlich sind, und auf denen ich in Zukunft aufbauen werde. Ich will aber auch weiterhin, unter anderem, Dokumentarfilme ohne Computeralgorithmus montieren. Eine Geschichte mit der Regie aus dem Material herausarbeiten und so etwas über dessen Thematik lernen. Das machen, was die klassische Filmmontage genannt wird. Weil es eine andere Qualität hat, die ich nicht missen möchte, und die für mich nicht durch das Schreiben von Computerprogrammen zu ersetzen ist.

Ich habe im Rahmen meines Studiums jede Menge über die Montage, die Kunst, und den Film gelernt. Und bin sehr dankbar, dass ich das wahrnehmen durfte. In Bezug auf meine Masterarbeit möchte ich mich vor allem bei meinem Bruder Johannes Frank und Ingo Stock bedanken, die mich mit Ihrem mathematischen Wissen beim Programmieren unterstützt haben. Einige der vorgestellten Experimente sind mit ihnen gemeinsam entstanden und wir haben viele Gespräche über die algorithmische Komposition, die künstliche Intelligenz und das Programmieren geführt. Ohne sie wäre meine Arbeit so nicht möglich gewesen. Dann möchte ich mich bei Marlis Roth für die Betreuung dieser und der anderen künstlerischen Arbeiten bedanken. Ich bin mir sicher, ich kann viel davon mitnehmen und weiß es sehr zu schätzen. Ebenso möchte ich meinen Kommilitonen und den anderen Lehrenden für die bereichernde Zeit und Freunden und Familie für die Unterstützung danken.

Abbildungsverzeichnis

Abb. 1: Ein Baum auf dem Tempelhofer Feld. Diese Aufnahme habe ich für eines meiner ersten algorithmischen Experimente, den audiovisuellen Automaten, genutzt. Eigene Aufnahme.

Abb. 2: Eine mit »Pure Data« erstellte diagrammatische Partitur (doc/4. data.structures/07.sequencer.pd). »This patch shows an example of how to use data collections as musical sequences (with apologies to Yuasa and Stockhausen). Here the black traces show dynamics and the colored ones show pitch. The fatness of the pitch traces give bandwidth. Any of the three can change over the life of the event.« Eigener Screenshot.

Abb. 3: Werner Graeff mit Siebdrucken seiner Filmpartituren. <http://www.wernergraeff.de/film/film.html> [22.5.2020].

Abb. 4: Montagediagramm für die Friseursequenz in »Man with a Movie Camera« von Sziga Vertov. Der Text in den nummerierten Kästen beschreibt den Bildinhalt, während die Linie dazwischen den Rhythmus der Montage beschreibt. <https://medium.com/janbot/a-brief-history-of-algorithmic-editing-732c3e19884b> [22.5.2020].

Abb. 5: Filmstreifen aus Oskar Fischingers »Tönende Ornamente«. https://www.researchgate.net/figure/Oskar-Fischinger-display-card-from-Ornament-Sound-experiments-C-Center-for-Visual_fig5_326414520 [22.5.2020].

Abb. 6: Montagediagramm (Kaderplan) für »6 / 64 Mama und Papa« von Kurt Kren. <https://medium.com/janbot/a-brief-history-of-algorithmic-editing-732c3e19884b> [22.5.2020].

Abb. 7: Plan für die Abfolge der schwarzen und weißen Bilder in Tony Conrads Film »The Flicker«. https://en.wikipedia.org/wiki/The_Flicker [22.5.2020].

Abb. 8: Screenshot aus »Synchrony« von Norman McLaren. Eigener Screenshot.

Abb. 9: Ein Programm als Programmablaufplan, datensstromorientierter Programmierung und textbasierter Programmierung (von links nach rechts). Eigene Darstellung.

Abb. 10: Bei einer Markov-Kette erster Ordnung ergibt sich die Folgewahrscheinlichkeit aus dem letzten Zustand. Eigene Darstellung.

Abb. 11: Eine Übergangsmatrix zur Berechnung von Markov-Ketten erster Ordnung. Eigene Darstellung.

Abb. 12: Ein Prozessdiagramm zur Veranschaulichung der Folgewahrscheinlichkeiten einer Markov-Kette. Eigene Darstellung.

Abb. 13: Bei einer Markov-Kette zweiter Ordnung ergibt sich die Folgewahrscheinlichkeit aus den letzten beiden Zuständen. Eigene Darstellung.

Abb. 14: Folgewahrscheinlichkeiten für eine Markov-Kette zweiter Ordnung. Eigene Darstellung.

Abb. 15: Es können auch mehrere Variablen für die Berechnung von Folgewahrscheinlichkeiten genutzt werden. Eigene Darstellung.

Abb. 16: Folgewahrscheinlichkeiten für Markov-Ketten erster Ordnung mit zwei Variablen am Beispiel einer duofonen Tonfolge (Abb. 15). Eigene Darstellung.

Abb. 17: Ausschnitt der Bedienoberfläche des Markov-Generator 1. Modul für einen MIDI-Kanal. Eigener Screenshot.

Abb. 18: Bedienoberfläche des Markov-Generator 2. Eigener Screenshot.

Abb. 19: Bedienoberfläche des Markov-Generator 3. Eigener Screenshot.

Abb. 20: Bedienoberfläche des Markov-Generator 4. Eigener Screenshot.

Abb. 21: Ein Ausschnitt eines Versuchs von meinem Vater Rolfdieter, mir das Prinzip der Markov-Kette zu erklären. Dass er und mein Bruder Johannes Mathematiker sind inspiriert mich auch im künstlerischen Schaffen, auch wenn mein mathematisches Verständnis im Vergleich sehr begrenzt ist. Ohne die mathematische Hilfe meines Bruders wäre der Montage-Automat in der jetzigen Form nicht entstanden, ebenso wenig aber ohne meine künstlerischen Ideen. Darstellung von Rolfdieter Frank.

Abb. 22: Strukturen einer Eisfläche als Metapher für Störungen und Verbindungen. Eigene Aufnahme.

Abb. 23: Schematische Darstellung der Funktionsweise der ersten Version des Montage-Automaten (Algorithmus 1). Eigene Darstellung.

Abb. 24: Schematische Darstellung der Funktionsweise der zweiten Version des Montage-Automaten (Algorithmus 2). Eigene Darstellung.

Abb. 25: Berechnung der Signifikanz-Matrix und der Folge-Matrix. Eigene Darstellung.

Abb. 26: Bedienoberfläche des Montage-Automaten mit den Einstellungen der vorgestellten »Alphaville« Sequenz. Das Diagramm unten links zeigt auf der x-Achse die Untertitelnummer der generierten Sequenz und auf der y-Achse den in der Folge-Matrix ermittelten Wert des Untertitels. Eigener Screenshot.

Abb. 27 - 54: Die Bedienoberfläche des Montage-Automaten und die ersten 28 über die Untertitel angeordneten Filmeinheiten des Films »Alphaville« von Jean-Luc Godard. Eigene Screenshots.

Abb. 55: Bedienoberfläche des Struct-Sequencers, der Ausgangspunkt für die Experimente mit den zellulären Automaten war. Eigener Screenshot.

Abb. 56 - 151: Eine Sequenz aus 96 Generationen von John Conways »Game of Life«. Generiert von einem Programm, das ich mit »Pure Data« und dessen Bibliothek »Ofelia« geschrieben habe.

Abb. 152: Bedienoberfläche des Programms GLSLVideoEffects. Eigener Screenshot.

Abb. 153 - 162: Eine Serie von 10 Screenshots einer eigenen Videoaufnahme, manipuliert mit den Effekten des Programms GLSLVideoEffects. Eigene Screenshots.

Abb. 163: Bedienoberfläche des audiovisuellen Automaten. Eigener Screenshot.

Abb. 164: Bedienoberfläche des Melodie-Generators. Eigener Screenshot.

Abb. 165: Bedienoberfläche des Lissajous-Sequencers. Eigener Screenshot.

Abb. 166: Bedienoberfläche des Multi-Sliders. Eigener Screenshot.

Abb. 167: Bedienoberfläche des Slit-Scan Generators. Eigener Screenshot.

Abb. 168: Screenshot des Video-Delay Effekts. Eigene Mondaufnahme. Eigener Screenshot.

Abb. 169: Bedienoberfläche des Grid-Sequencers. Eigener Screenshot.

Abb. 170: pure data one line. Eigener Screenshot. <https://youtu.be/ed-jlA40Ux48>

Abb. 171: markov 1. Eigener Screenshot. <https://youtu.be/qDohu4cU6dk>

Abb. 172: audiovisueller automat 1. Eigener Screenshot. <https://youtu.be/gQhU-Gxyric>

Abb. 173: GemSquare 3. Eigener Screenshot. <https://youtu.be/ssAbzF602-o>

Abb. 174: mond. Eigener Screenshot. <https://youtu.be/5TG2jKGmDss>

Abb. 175: lissajous. Eigener Screenshot. sequencer 6 <https://youtu.be/u2RJWqli8mQ>

Abb. 176: 2 lsys. Eigener Screenshot. <https://youtu.be/2DFx098Seuo>

Abb. 177: gol seq 9. Eigener Screenshot. <https://youtu.be/UT6zBUJZiKg>

Abb. 178: polyrhythm. Eigener Screenshot. <https://youtu.be/3T1KGFAQLIc>

Abb. 179: ofelia test 1. Eigener Screenshot. <https://youtu.be/H-Mu5eBl-pvA>

Abb. 180: drag 2. Eigener Screenshot. <https://youtu.be/CVCAZ74wjRA>

Abb. 181: ofelia sample player. Eigener Screenshot. <https://youtu.be/-3pDFzU04AU>

Abb. 182: AudioMarkov. Eigener Screenshot. <https://youtu.be/al8VtaG9zas>

Abb. 183: AudioToMidiMarkov 6. Eigener Screenshot. <https://youtu.be/TYp0-ieBHfs>

Abb. 184: ofelia multitoggle 2. Eigener Screenshot. <https://youtu.be/15lfN3aIRCA>

Abb. 185: ofelia multiradio 2. Eigener Screenshot. <https://youtu.be/tfLI7l-PpCo>

Abb. 186: ofelia 2 gridX 2. Eigener Screenshot. https://youtu.be/3_BH-HLdkAsA

Abb. 187: ofelia 2 gridXconway 123 3. Eigener Screenshot. <https://youtu.be/el6gTCZFWKK>

Abb. 188: lua markov generator. Eigener Screenshot. <https://youtu.be/6WN1oggnlKQ>

Abb. 189: LuaMIDICVSMkov 5. Eigener Screenshot. <https://youtu.be/GPa-gG-mRSU>

Abb. 190: MusicPlayer 1. Eigener Screenshot. <https://youtu.be/u47H4wp-qovs>

Abb. 191: videoDelayTest. Eigener Screenshot. <https://youtu.be/TV-8W8Fy1w60>

Abb. 192: WebAudioMIDIButterchurn. Eigener Screenshot. <https://youtu.be/kJwH3THSgn8>

Abb. 193: map. Eigener Screenshot. https://youtu.be/Va_K-6Mc_9g

Abb. 194: Mein erster Versuch »Pure Data« Programme mit »Ofelia« und »Emscripten« in eine Internetseite zu integrieren. Eigener Screenshot.

Abb. 195: Screenshot der Internetseite <https://pdspectrum.handmadeproductions.de>. Erste Versuche von mir Bild- und Tondateien in Echtzeit im Internet zu manipulieren. Programmiert ist die Seite wieder mit »Pure Data«, »Ofelia« und »Emscripten«. Die Bedienoberfläche ist mit »JavaScript« erstellt. Eigener Screenshot.

Literaturverzeichnis

- 1 Christopher Dell: Prinzip Improvisation. Köln: Verlag der Buchhandlung Walther König, 2002. S. 17.
- 2 Miller Puckette: Using Pd as a score language. San Diego: University of California, 2007. S. 1.
- 3 https://de.wikipedia.org/wiki/Algorithmische_Komposition [22.05.2020].
- 4 Philip Galanter: Zitiert von der Mailingliste eu-gene. http://artwarez.org/projects/nagB00K/texte/sarah_de.html [22.05.2020].
- 5 https://de.wikipedia.org/wiki/Digitale_Kunst [22.05.2020].
- 6 Joseph Campbell: Der Heros in tausend Gestalten. Frankfurt: Insel Taschenbuch, 1999.
- 7 Gilles Deleuze: Das Zeit-Bild - Kino 2. Frankfurt: Suhrkamp, 1997. S. 54.
- 8 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 332.
- 9 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 314.
- 10 Thomas Dreher: Das Kunstwerk, Weltkunst und Intermedia Art - kritische Berichte, Bd. 36, Nr. 4. Weimar: Jonas Verlag, 2008. S. 54.
- 11 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 45.
- 12 Markus Koller: Die Grenzen der Kunst - Luhmanns gelehrte Poesie. Wiesbaden: VS Verlag für Sozialwissenschaften, 2007. S. 50.
- 13 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp 1997. S. 76.
- 14 Franz Schuh: Schöpfung ohne Zentrum, 1996. https://www.zeit.de/1996/10/Schoepfung_ohne_Zentrum/komplettansicht [22.5.2020]
- 15 George Spencer Brown: Laws of Form - Gesetze der Form. Leipzig: Bohmeier Verlag, 1997.
- 16 https://de.wikipedia.org/wiki/Absoluter_Film [22.5.2020].
- 17 Clint Enss: A Brief History of Algorithmic Editing, 2018. <https://medium.com/janbot/a-brief-history-of-algorithmic-editing-732c3e19884b> [22.5.2020].

- 18 Pablo Núñez Palma: An algorithmic mindset for filmmaking, 2018. <https://medium.com/janbot/an-algorithmic-mindset-for-filmmaking-572a04c-4cd04> [22.5.2020].
- 19 Christopher Dell: Raumwandler, 2012. <https://www.youtube.com/watch?v=ExCCmp9NI58> (Position 11:14) [22.5.2020].
- 20 Pablo Núñez Palma: An algorithmic mindset for filmmaking, 2018. <https://medium.com/janbot/an-algorithmic-mindset-for-filmmaking-572a04c-4cd04> [22.5.2020].
- 21 Werner Graeff: Film als Film. 1910 bis Heute, 1977 / 1978. <http://www.wernergraeff.de/film/film.html> [22.5.2020].
- 22 Dziga Vertzov: Film Culture Reader - The Writings of Dziga Vertov. Berkeley: University of California Press, 1984. S. 366.
- 23 Oskar Fischinger: Sounding Ornaments, 1932. <http://www.center-forvisualmusic.org/Fischinger/SoundOrnaments.htm> [22.5.2020].
- 24 Oskar Fischinger: An Optical Poem, 1938. <https://www.youtube.com/watch?v=6Xc4g00FFLk> (Position 00:35) [22.5.2020].
- 25 Peter Tscherkassky: Mama and Papa. <https://kurtkren.info/> [22.5.2020].
- 26 Heike Helfert: Tony Conrad »The Flicker«. <http://www.medien-kunstnetz.de/werke/the-flicker/> [22.5.2020].
- 27 Norman McLaren: Letter to Thea Goldberg, 1973, S. 3. NFB Archives. Terence Dobsen: The Film Work of Norman McLaren. Canterbury: University of Canterbury, 1994. S. 198.
- 28 https://de.wikipedia.org/wiki/Pure_Data [22.5.2020].
- 29 <https://forum.pdpatchrepo.info> [22.5.2020].
- 30 <https://github.com/cuinjune/Ofelia> [22.5.2020].
- 31 <https://emscripten.org> [22.5.2020].
- 32 <https://troikatronix.com> [22.5.2020].
- 33 <https://www.reddit.com/r/SubredditSimulator> [22.5.2020].
- 34 Devin Soni, <https://towardsdatascience.com/introduction-to-markov-chains-50da3645a50d> [22.5.2020].
- 35 <https://forum.pdpatchrepo.info/topic/10791/markovgenerator-a-music-generator-based-on-markov-chains-with-variable-length> [22.5.2020].

- 36 Beschreibung des Programms von Ingo Stock. <https://forum.pdpatchrepo.info/topic/12147/midi-into-seq-and-markov-chains/44> [22.5.2020].
- 37 <https://forum.pdpatchrepo.info/topic/11859/lua-ofelia-markov-generator-patch-abstraction> [22.5.2020].
- 38 <https://forum.pdpatchrepo.info/topic/11864/lua-midi-markov/4> [22.5.2020].
- 39 Mark Johnson: The Body in the Mind - The Bodily Basis of Meaning, Imagination and Reason. Chicago: The University of Chicago Press, 1987. S. 168.
- 40 https://de.wikipedia.org/wiki/Lemmy_Caution_gegen_Alpha_60 [22.5.2020].
- 41 <https://www.seo-united.de/glossar/stopwort> [22.5.2020].
- 42 Max Bense: Projekte generativer Ästhetik, 1965. S. 2. http://dada.compart-bremen.de/docUploads/Bense_Manifest.pdf [22.5.2020].
- 43 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 332.
- 44 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 331.
- 45 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 165.
- 46 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 336.
- 47 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 228 - 329.
- 48 Martin Kemp: Wissen in Bildern - Intuitionen in Kunst und Wissenschaft, in: Christa Maar (Hrsg.): Iconic Turn. Die neue Macht der Bilder. Köln: DuMont Buchverlag, 2004. S. 389.
- 49 Markus Koller: Die Grenzen der Kunst - Luhmanns gelehrte Poesie. Wiesbaden: VS Verlag für Sozialwissenschaften, 2007. S. 13.
- 50 Niklas Luhmann: Niklas Luhmann - 1997 - Es gibt keine Biographie. <https://www.youtube.com/watch?v=nFhQ6SrIKVo> (Position 53:15) [22.5.2020].
- 51 Niklas Luhmann / Dirk Baecker / Frederick Bunsen: Unbeobachtbare Welt - Über Kunst und Architektur. Bielefeld: Haux, 1990. S. 66.
- 52 <https://de.wikipedia.org/wiki/Dramaturgie> [22.5.2020].

- 53 John Haugeland: Artificial intelligence - The Very Idea. Cambridge: MIT Press, 1989. S. 12.
- 54 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 347.
- 55 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 348.
- 56 Niklas Luhmann: Die Kunst der Gesellschaft. Frankfurt: Suhrkamp, 1997. S. 395.
- 57 <https://de.wikipedia.org/wiki/Code> [22.5.2020].
- 58 <https://forum.pdpatchrepo.info> [22.5.2020].
- 59 Ludger Brümmer / Benjamin Miller: CellularAutomataExplorer, 2017. <https://zkm.de/de/cellularautomataexplorer> [22.5.2020].
- 60 Erhard Behrends: Ist Mathematik die Sprache der Natur? In: Spektrum der Wissenschaft Highlights. Heidelberg: Spektrum Verlag, 2014. S. 82.
- 61 Konrad Zuse: Rechnender Raum, 1967. In: Spektrum der Wissenschaft - Spezial: Ist das Universum ein Computer? Heidelberg: Spektrum Verlag, 2007. S. 7 - 15.
- 62 Franziska Schneider: Explorative Visualisierung zellulärer Automaten. Augsburg: Bachelorarbeit im Studiengang Kommunikationsdesign der Hochschule Augsburg 2018, S. 46. <http://zellmemore.de/zellmemore.pdf> [22.5.2020].
- 63 <https://github.com/Jonathhhan> [22.5.2020].
- 64 <https://www.youtube.com/channel/UCVI9jk5rnDcCHARIwAn5yYA> [22.5.2020].
- 65 <http://index.handmadeproductions.de> [22.5.2020].
- 66 <http://countercomplex.blogspot.com/2011/10/algorithmic-symphonies-from-one-line-of.html> [22.5.2020].
- 67 <https://de.wikipedia.org/wiki/Slitscan> [22.5.2020].
- 68 <https://www.youtube.com/channel/UCVI9jk5rnDcCHARIwAn5yYA> [22.5.2020].
- 69 <https://index.handmadeproductions.de> [22.5.2020].
- 70 <https://github.com/jberg/butterchurn> [22.5.2020].

71 Anke Haarmann: 10. TDK des IKF: »Künstlerische Forschung: Kunst als Wissensproduktion«. Potsdam-Babelsberg: Filmuniversität Babelsberg KONRAD WOLF, 2014. <https://www.youtube.com/watch?v=KjwLfMeSjdY> (Position 0:40) [22.5.2020].

72 Anastasios Giannaras: Ästhetik heute. München: Francke Verlag, 1973. S. 7.

73 Ernst Bruche: Was ist eigentlich Forschung? In: Physikalische Blätter, Volume 29, Issue 3. Mosbach: Physik Verlag, 1973. S. 141. <https://onlinelibrary.wiley.com/doi/pdf/10.1002/phbl.19730290307> [22.5.2020].

74 Anke Haarmann: Künstlerische Praxis als methodische Forschung. Düsseldorf: 2011. S. 7. <http://www.dgae.de/wp-content/uploads/2011/09/Haarmann.pdf> [22.5.2020].

75 Anke Haarmann: Künstlerische Praxis als methodische Forschung. Düsseldorf: 2011. S. 1. <http://www.dgae.de/wp-content/uploads/2011/09/Haarmann.pdf> [22.5.2020].

76 Anke Haarmann: Künstlerische Praxis als methodische Forschung. Düsseldorf: 2011. S. 7. <http://www.dgae.de/wp-content/uploads/2011/09/Haarmann.pdf> [22.5.2020].

77 Corinna Carduff: Literatur und künstlerische Forschung. In: Zürcher Jahrbuch der Künste 2009, Band 6 - Kunst und künstlerische Forschung. Zürich: Verlag Scheidegger & Spiess AG, 2010. S. 115 - 116.

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich meine Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Berlin, 9. Juni 2020

Jonathan Frank

